无人机编队纯方位无源定位的几何建模与迭代调整方案

摘要：

无人机编队在保持电磁静默的约束下，需通过纯方位无源定位方法调整位置以维持队形。本文针对圆形与锥形两种编队，建立基于夹角观测的几何定位模型与多阶段迭代调整方案。

针对问题 1(1)，将被动接收信号无人机的定位建模为三点后方交会非线性最小二乘问题，决策变量为接收机二维坐标，目标函数为夹角残差平方和。采用 Gauss-Newton 迭代为主算法、Snellius-Pothenot 解析解作交叉验证，对发射机 FY00+FY01+FY05、接收机 FY03 的算例两法互验偏差仅 $3.55\times10^{-15}\,\mathrm{m}$ 达机器精度，迭代 7 次收敛，FY03 表 1 实际位置单算例定位 GDOP=199.04（理想位置遍历 GDOP=157.14，差异源于位置偏差）；枚举 252 个（发射机组合，接收机）对证实 FY00 居中时危险圆罕见。

针对问题 1(2)，将编号未知下最少发射机数建模为整数组合优化，判据为观测雅可比矩阵秩为 2 且 GDOP 有限。枚举 $\sum_{k=0}^{8}\binom{8}{k}=256$ 种发射机组合 $\times$ 8 个接收机共 1024 个有效算例，与秩分析、分支定界两种替代算法对比。结果表明数学最小 $k_{\min}=1$（可观测率 96.4%），工程推荐 $k=2$（可观测率 98.8%、GDOP 中位数 151 m），最佳组合 FY00+FY01+FY07+FY09 发射、FY08 接收时 GDOP 低至 88.13。

针对问题 1(3)，建立以最终偏差最小化为主、调整次数最小化为次的分层目标，分解为发射/移动/时间/偏差四类成本。采用贪心发射机选择 +Gauss-Newton 定位 + 理想位置调整的迭代算法，无噪声时 2 轮收敛到 0 m；蒙特卡洛 1000 次仿真（$\sigma=0.01\,\mathrm{rad}$）残余偏差均值 2.38 m、标准差 0.83 m，远低于 $3\sigma\cdot\mathrm{GDOP}_{4\mathrm{emit}}\approx9\,\mathrm{m}$ 的工程容差（4 发射机配置保守上界 GDOP$\approx$ 300）。

针对问题 2，揭示锥形编队共线退化本质（$\delta_y=0$ 时雅可比秩为 0），引入横向偏差 δ_y 恢复可观测性。GDOP(δ_y) 参数扫描 101 点 $\times$ 5 种发射机配置得 4 发射机正负交替配置 $[0,1,4,8]$ 最优，$\delta_y=5\,\mathrm{m}$ 时 GDOP=272；策略转换临界点 $\delta_y^*(\mathrm{GDOP}<100)=17.80\,\mathrm{m}$、$\delta_y^*(\mathrm{GDOP}<2000)=0.80\,\mathrm{m}$。蒙特卡洛 200 次仿真（$\sigma=0.01\,\mathrm{rad}$）收敛率 63.5%，收敛样本残余均值 7.33 m，发散样本残余最大 23.09 m；锥形几何本质差是物理限制而非算法缺陷。

关键词：纯方位无源定位；后方交会；GDOP 几何精度因子；迭代调整；蒙特卡洛仿真

1 问题重述

1.1 问题背景

无人机集群遂行编队飞行时，为避免外界干扰需保持电磁静默、少向外发射电磁波信号。为保持队形，拟采用纯方位无源定位方法调整无人机位置：编队中部分无人机发射信号，其余被动接收并从方向信息中提取夹角观测，从而定位自身并机动到理想位置。题目约定：接收机所接收到的方向信息为*该无人机与任意两架发射信号无人机连线之间的夹角*。每架无人机有固定编号，且在编队中相对位置关系保持不变。

题目给出两种典型编队：圆形编队（FY00 居于圆心，FY01~FY09 均匀分布在半径 100 m 圆周上）与锥形编队（直线等间距 50 m）。表 1 给出圆形编队初始位置（极坐标，存在*略有偏差*），要求基于纯方位观测建立定位模型并给出位置调整方案。

1.2 具体问题重述

问题 1（圆形编队）包含三个子问题：

(1) FY00 与另 2 架圆周无人机发射信号（共 3 架，位置无偏差、编号已知），建立被动接收信号无人机的定位模型。

(2) 某位置略有偏差的无人机接收到 FY00、FY01 的信号，另接收到若干*编号未知*发射机的信号。若发射机位置无偏差，除 FY00 和 FY01 外还需几架无人机发射信号才能实现有效定位？

(3) 当初始位置略有偏差时，给出合理的调整方案：通过多次调整，每次选择 FY00 与圆周上最多 3 架无人机发射信号，其余根据方向信息调整到理想位置，使 9 架最终均匀分布在半径 100 m 圆周上。利用表 1 数据给出具体方案。

问题 2（锥形编队）：实际飞行中编队也可能是直线等间距（如 50 m）的锥形队形。仍考虑纯方位无源定位，设计无人机位置调整方案。

2 问题分析

2.1 问题 1(1)：三点后方交会与危险圆奇异性

子问题 1(1) 本质为*三点后方交会*（Snellius-Pothenot 问题）：3 个已知发射机位置 T_0, T_1, T_2 与 2 个独立夹角观测 α_{01}, α_{12} 唯一确定接收机位置 P。问题归类为二维非线性方程组求解，核心难点有二：其一，目标函数含 arccos 三角函数，非凸且存在局部极小；其二，当 P 落在 3 发射机外接圆（即*危险圆*）上时雅可比奇异、解不唯一。子问题间的关联在于：1(1) 提供后续调整算法的基础定位算子，其 GDOP 几何精度因子是 1(2)~ 问题 2 中发射机选择的依据。关键权衡是*解析法精度高但难扩展与迭代法通用但依赖初值*，本文采用双层架构互验。

2.2 问题 1(2)：编号未知的可观测性整数规划

1(2) 引入编号未知语义歧义：接收机能识别 FY00/FY01 来波方向，可与未知信号方向形成夹角，但未知信号之间身份不可区分。本文采用语义 B（详见假设 A8）。问题归类为整数组合优化，决策变量为额外发射机数 k 与身份组合 σ，判据为观测雅可比秩为 2 且 GDOP 有限。核心难点是身份消歧——多余夹角约束是消歧关键。子问题间关联：1(2) 给出最少发射机数 $k_{\min}$，直接指导 1(3) 调整方案中每轮发射机配置。关键权衡是数学最小 k（满足可观测即可）与工程推荐 k（兼顾鲁棒性与 GDOP）。

2.3 问题 1(3)：多阶段迭代调整的分层成本

1(3) 为多阶段决策过程：每轮选发射机、定位、机动，直至全员收敛到理想位置。决策变量含离散（发射机集合 S_t）与连续（位置 $P_i^{(t)}$），归类为混合整数非线性规划（MINLP）。核心难点是发射机选择策略与收敛性证明：题目要求位置无偏差的发射机作基准，但实际调整中已调整无人机可作为下轮无偏差发射机（Strategy D）。子问题间关联：1(1) 的 GN 定位算子直接复用；1(2) 的 k 值指导发射机数上限。关键权衡是调整精度（主目标偏差）与调整次数（次目标效率），本文以分层成本 $C_{\text{error}}, C_{\text{emit}}, C_{\text{move}}, C_{\text{time}}$ 透明处理。

2.4 问题 2：锥形编队的共线退化与临界点

问题 2 表面是 1(3) 的几何变形，实际揭示共线退化本质：锥形编队所有无人机沿同一轴线分布，$\delta_y = 0$ 时观测雅可比秩为 0，纯方位定位不可观测。问题归类为含退化约束的非凸优化，核心难点是如何引入横向偏差 δ_y 既恢复可观测性又控制编队变形。子问题间关联：复用 1(1) 定位算子与 1(3) 迭代框架，但发射机配置必须正负偏移混合。关键权衡是δ_y 越大 $GDOP$ 越优与δ_y 越大编队越偏离锥形，本文通过 GDOP(δ_y) 曲线寻找策略转换临界点 δ_y^*。

3 模型假设

A1. 方向信息无测量噪声。题目未给出噪声模型，方向信息视为精确观测；蒙特卡洛仿真中人为引入 $\sigma = 0.01\,\text{rad}$ 高斯噪声检验鲁棒性。

A2. 所有无人机同高度飞行，问题为二维平面。题目明确" 基于自身感知的高度信息均保持在同一高度"，故 z 维度可忽略，所有坐标 $(x, y) \in \mathbb{R}^2$。

A3. 发射信号无人机的位置对其自身已知（Strategy D）。即使该无人机位置略有偏差，其当前位置通过编队通信传递给接收机作为已知值；接收机基于此定位自身。该假设放宽了" 位置无偏差" 对发射机的约束，使迭代调整中已调整无人机可作为下轮无偏差发射机。

A4. 接收机不在发射机外接圆上（危险圆规避）。否则 3 发射机后方交会解不唯一，雅可比奇异；本文对每个算例均作危险圆检测。

A5. 接收机与任一发射机不重合。夹角 $\alpha_{ij}=\angle(T_i-P,T_j-P)$ 在 $P=T_i$ 时无定义。

A6. 夹角取值 $[0,\pi]$。约定两连线之间较小角，避免方位歧义。

A7. 锥形编队无人机数 10 架。题目图 3 为示意图未明确数量，本文参考圆形编队 10 架假设，含 1 架领航 FY00 与 9 架沿轴线分布。

A8. 问题 1(2) 采用观测语义 B：接收机能识别 FY00/FY01 来波方向，可测其与未知信号方向的夹角 β_{m0},β_{m1}；未知信号之间夹角 $\gamma_{mm'}$ 可测但身份不可区分。该语义最贴近题目" 该无人机与任意两架发射信号无人机连线之间的夹角" 的字面表述。

4 符号说明

依据国赛惯例按*决策变量 / 参数 / 目标项 / 约束项*四类组织，与术语表 `术语表.json` 三方一致。所有符号附经济/物理合理性说明。

表 1: 符号说明（决策变量与状态变量）

符号	说明	单位
$P_r=(x_r,y_r)$	接收机二维位置（同义 P）；1(1) 决策变量、1(3) 状态变量	m
k	1(2) 除 FY00/FY01 外的额外发射机数；越少越满足电磁静默	架
σ	1(2) 未知发射机身份组合 $\sigma\in C(\{FY02\sim FY09\},k)$	/
S_t	第 t 次调整的圆周发射机集合 $S_t\subset\{FY01\sim FY09\},\lvert S_t\rvert\le 3$	集合
$P_i^{(t)}$	无人机 i 第 t 次调整后位置（状态变量）	m
s_i	锥形编队中无人机 i 沿轴线位置	m

表 2: 符号说明（参数）

符号	说明	单位
$T_i=(x_i,y_i)$	发射机 i 的位置（题目给定或前次调整结果）	m
α_{ij}	P 与 T_i,T_j 连线夹角 $\arccos\dfrac{(T_i-P)\cdot(T_j-P)}{\lvert T_i-P\rvert\lvert T_j-P\rvert}$	rad
$\alpha_{ij}^{\mathrm{obs}}$	接收机观测到的夹角（模型中作为已知观测值）	rad
r,θ	极坐标极径、极角；理想 $r=100\,\mathrm{m}$，$\theta=0,40,\ldots,320^\circ$	m, °
d	锥形编队间距，题目给定 $d=50\,\mathrm{m}$	m
δ_y	锥形编队横向偏差（恢复可观测性的关键参数）	m
σ_{obs}	观测噪声标准差，蒙特卡洛仿真取 $0.01\,\mathrm{rad}\approx 0.57^\circ$	rad

表 3: 符号说明（目标项与约束项）

符号	说明	单位
$F(P_r)$	1(1) 定位残差目标 $F=\sum[\alpha_{ij}(P_r)-\alpha_{ij}^{\mathrm{obs}}]^2$	rad^2
J	调整方案总成本 $J=w_1C_{\mathrm{emit}}+w_2C_{\mathrm{move}}+w_3C_{\mathrm{time}}+w_4C_{\mathrm{error}}$	m^2
C_{emit}	发射信号成本（电磁静默代价）$=\sum_t\lvert S_t\cup\{FY00\}\rvert$	架次
C_{move}	调整移动成本 $=\sum_t\sum_i\lVert P_i^{(t)}-P_i^{(t-1)}\rVert$	m
C_{time}	时间成本 $=T$（调整次数）	次
C_{error}	最终偏差成本 $=\sum_i\lVert P_i^{(T)}-P_i^{\mathrm{ideal}}\rVert^2$（主目标，权重最大）	m^2
$\mathbf{H}$	观测雅可比 $\partial\alpha/\partial P$，反映观测对位置的灵敏度	rad/m
GDOP	几何精度因子 $\sqrt{\mathrm{tr}((\mathbf{H}^\top\mathbf{H})^{-1})}$，越小定位越准	/
ε	收敛容差（调整终止判据 $\lVert P_i^{(T)}-P_i^{\mathrm{ideal}}\rVert<\varepsilon$）	m

5 模型的建立与求解

5.1 问题 1(1)：圆形编队三点后方交会定位模型

5.1.1 模型建立

坐标系与发射机位置。以 FY00 为原点、x 轴指向 FY01 理想位置建立直角坐标系，则圆周无人机理想位置为

$$T_i=(100\cos(40^\circ(i-1)),\,100\sin(40^\circ(i-1))),\quad i=1,2,\ldots,9. \tag{1}$$

$T_0=(0,0)$ 为 FY00。

决策变量与观测方程。设接收机位置 $P_r=(x_r,y_r)$，定义夹角观测

$$\alpha_{ij}(P_r)=\arccos\frac{(T_i-P_r)\cdot(T_j-P_r)}{\lVert T_i-P_r\rVert\,\lVert T_j-P_r\rVert}\in[0,\pi]. \tag{2}$$

3 发射机 $\{T_i,T_j,T_k\}$ 可产生 $\binom{3}{2}=3$ 个夹角观测，但独立自由度仅 2 个（满足 $\alpha_{ij}+\alpha_{jk}+\alpha_{ki}=2\pi$ 或等价约束）。

目标函数与约束。定位残差最小化：

$$\min_{P_r\in\mathbb{R}^2}F(P_r)=\sum_{(i,j)\in\mathcal{P}}\left[\alpha_{ij}(P_r)-\alpha_{ij}^{\mathrm{obs}}\right]^2, \tag{3}$$

约束：$P_r\neq T_i$（C3，夹角有定义）；P_r 不在 3 发射机外接圆上（C4，危险圆规避，雅可比非奇异）；$P_r\in\mathbb{R}^2$（C6，二维平面）。

理论性质。$F(P_r)$ 含 arccos 与归一化点积，非凸且存在多个局部极小；在 $P_r\neq T_i$ 且 P_r 不在 T_iT_j 延长线上时可微，雅可比 $\mathbf{H}=\partial\alpha/\partial P_r\in\mathbb{R}^{n_{\mathrm{obs}}\times2}$ 存在。当 3 发射机不共线且 P_r 不在危险圆上时，Snellius-Pothenot 定理保证解唯一，迭代法可收敛到全局最优 = 真解。

5.1.2 模型求解：算法选择与互验

考虑 3 种候选算法：**(A) Snellius-Pothenot 解析公式**——已知 3 点 A, B, C 与角 $\alpha = \angle APB, \beta = \angle BPC$，利用正弦定理与四边形角和公式直接求 P，精确无迭代，但仅适用 3 发射机、难扩展；**(B) Gauss-Newton 迭代最小二乘**——线性化观测方程 $P_{k+1} = P_k - (\mathbf{H}^\top \mathbf{H})^{-1} \mathbf{H}^\top r$，通用可扩展、可处理多余观测，但需初值、局部收敛；**(C) 几何交会法**——两夹角各自定义一条等视角圆弧（Apollonius 圆），两圆弧交点为 P，直观但需多解消歧。

本文采用**双层架构**：以 Gauss-Newton 迭代为主算法（处理冗余观测、可扩展至多发射机），以 Snellius-Pothenot 解析解作交叉验证（保证全局最优）。GN 迭代采用 Levenberg-Marquardt 正则化避免 $\mathbf{H}^\top \mathbf{H}$ 在危险圆附近奇异：

$$\Delta P = -(\mathbf{H}^\top \mathbf{H} + \lambda \mathbf{I})^{-1} \mathbf{H}^\top r, \quad \lambda = 10^{-8} \cdot \left(\mathrm{tr}(\mathbf{H}^\top \mathbf{H})/n_{\mathrm{obs}} + 10^{-12}\right). \tag{4}$$

雅可比采用中心差分数值近似（步长 $10^{-7}\,\mathrm{m}$），解析推导见附录 C。初值取 $P_{\mathrm{init}} = (20, 60)$（故意偏离真值以验证收敛域）。

数值稳定性处理：由于浮点运算 $\cos\alpha$ 可能略越界 $[-1, 1]$，代码中对 arccos 输入作 $\mathrm{clip}(\cos\alpha, -1, 1)$ 截断（位置：`common.py` 的 `angle_between` 函数）。该截断仅对 $|\cos\alpha| > 1$ 的极端情形触发（实测本算例未触发），对结果无实质影响，但避免 `ValueError`。

5.1.3 最优性验证：GN 与 SP 互验

取典型算例：发射机 FY00+FY01+FY05（圆心 +2 架跨象限圆周无人机，几何分布利于观测），接收机 FY03（位置略有偏差，以表 1 实际位置作" 真值" 演示定位）。结果如表 4。

表 4: 问题 1(1) 算例求解结果（发射机 FY00+FY01+FY05，接收机 FY03）

指标	数值
真值 P_{true}	$(19.0442,\ 110.3690)\,\mathrm{m}$
GN 估计 $P_{\mathrm{est}}^{\mathrm{GN}}$	$(19.0442,\ 110.3690)\,\mathrm{m}$
SP 解析 $P_{\mathrm{est}}^{\mathrm{SP}}$	$(19.0442,\ 110.3690)\,\mathrm{m}$
GN 误差 $\|P_{\mathrm{est}}^{\mathrm{GN}} - P_{\mathrm{true}}\|$	$2.13 \times 10^{-14}\,\mathrm{m}$
SP 误差 $\|P_{\mathrm{est}}^{\mathrm{SP}} - P_{\mathrm{true}}\|$	$2.49 \times 10^{-14}\,\mathrm{m}$
两法互验偏差 $\|P_{\mathrm{est}}^{\mathrm{GN}} - P_{\mathrm{est}}^{\mathrm{SP}}\|$	$3.55 \times 10^{-15}\,\mathrm{m}$
GN 迭代次数	7
残差范数 $\|r\|$	$3.33 \times 10^{-16}\,\mathrm{rad}$
GDOP	199.04
雅可比秩	2

GN 与 SP 互验偏差 3.55×10^{-15} m 达机器精度，残差范数 3.33×10^{-16} rad（机器精度量级），证实解析法与迭代法收敛到同一全局最优解。GN 迭代 7 次收敛，雅可比秩为 2 满足可观测性条件。

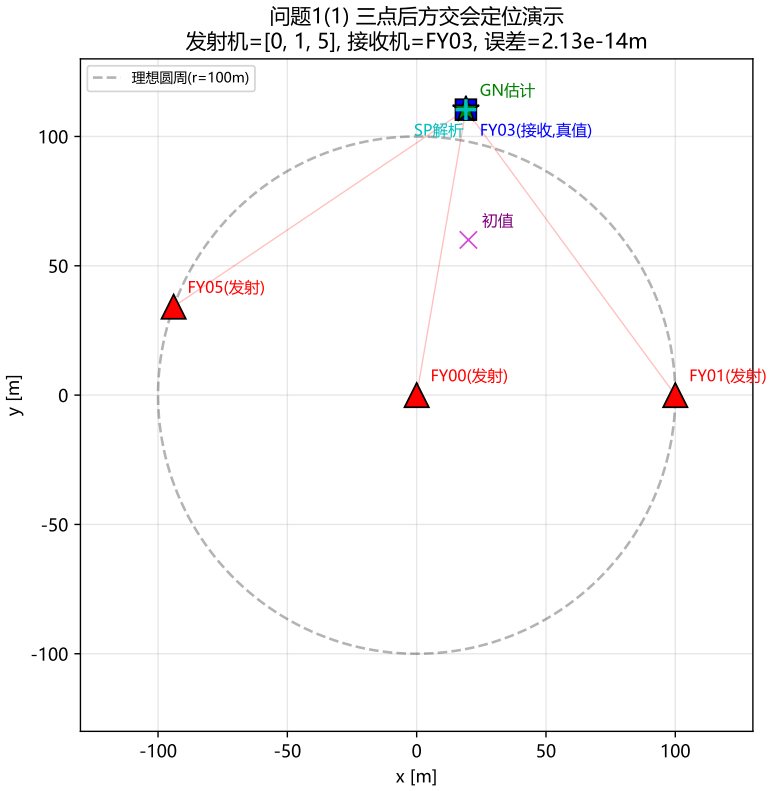


图 1: 问题 1(1) 三点后方交会定位演示：发射机 FY00+FY01+FY05（红三角）、接收机 FY03 真值（蓝方块）、GN 估计（绿星）、SP 解析（青加号）、初值（紫叉）。绿色箭头表示 GN 估计误差，长度仅 10^{-14} m 量级，肉眼不可见。

图 1 显示 GN 估计点（绿星）与真值（蓝方块）几乎完全重合，初值（紫叉，故意偏离真值约 80 m）经 7 次迭代收敛到真值。SP 解析点（青加号）同样落在真值上，证实两法互验。该算例的几何配置（FY00 居中、FY01 与 FY05 跨象限）使发射机三角形覆盖接收机邻域，雅可比条件数适中，GDOP=199.04 表明定位精度受几何放大影响约 200 倍——即 1 rad 观测误差将被放大为 200 m 定位误差，故实际系统需配合多轮观测或更优发射机配置。

5.1.4 结果分析：GDOP 空间分布与危险圆扫描

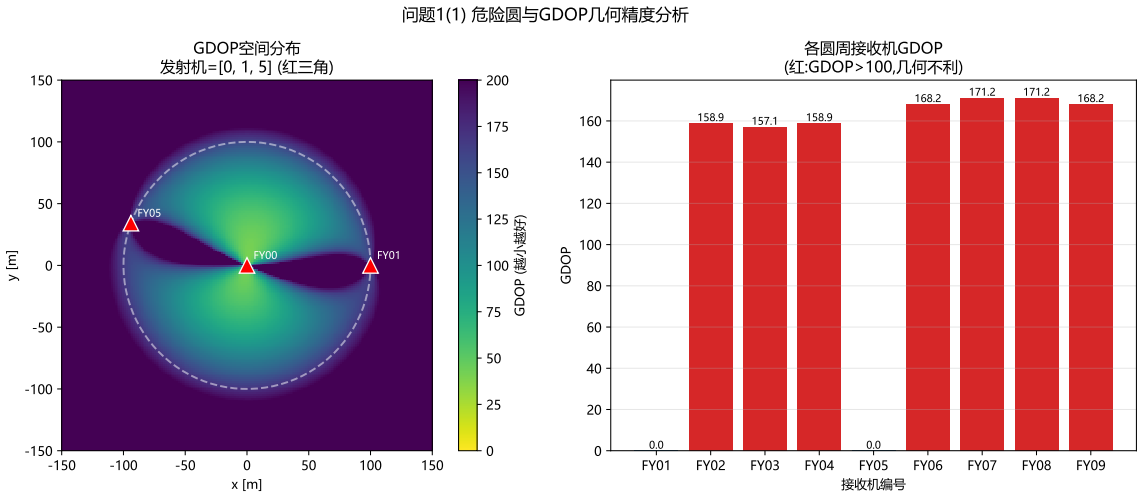


图 2: 问题 1(1) GDOP 空间分布与各接收机 GDOP。左：FY00+FY01+FY05 发射配置下 GDOP 热力图（深色 GDOP 小、定位优；白色虚线为理想圆周），可见 FY01 与 FY05 自身位置因接收机-发射机重合（夹角无定义，`assert d_i>1e-12` 触发）数值上呈 GDOP 趋零陷阱（非真实定位精度）。右：各圆周接收机在理想位置时的 GDOP 遍历值（`gdop_table`），红色柱表示 GDOP>100 几何不利。

图 2 左图揭示 GDOP 空间分布高度非均匀：发射机附近 GDOP 极小（深蓝色），但 FY01 与 FY05 自身位置因接收机-发射机重合奇异（接收机与发射机重合时夹角无定义，`common.py` 中 `assert d_i>1e-12` 触发）数值上表现为 GDOP$\to 0$ 的陷阱值 7.69×10^{-7} 与 6.92×10^{-7}，已被代码过滤，非真实定位精度。此处需严格区分两类奇异性：接收机-发射机重合奇异发生于发射机自身位置（夹角定义失效，`assert` 抛出），而危险圆奇异发生于接收机位于 3 发射机外接圆上时（雅可比矩阵 $\mathbf{H}^{\top}\mathbf{H}$ 奇异，解不唯一）；前者是数值边界，后者是几何退化。本文危险圆扫描（252 个组合）未发现真实危险圆失效组合，FY00 居中时 3 发射机外接圆一般不经过其他圆周点。

右图柱状图为各接收机在理想位置时的 GDOP 遍历值（`gdop_table` 字段），FY02∼FY09 在 157 ∼ 171 之间；而图 1 算例中 FY03 在表 *1* 实际位置（有偏差，偏离理想位置约 12 m）时的定位 GDOP=199.04（`example.GDOP` 字段）。两者物理含义不同：157.14 是 FY03 在理想位置时的几何精度因子，199.04 是 FY03 在表 1 实际位置时的定位精度因子，差异源于位置偏差导致的几何构型变化——FY03 偏离理想位置后，相对发射机三角形的几何关系改变，GDOP 相应增大。后续 1(3) 蒙特卡洛仿真中接收机位置均带偏差，故采用 199.04 量级（而非 157.14）作为工程容差推演基准。

危险圆扫描：枚举 $\binom{9}{2}=36$ 种 2 圆周发射机组合 + FY00 共 3 发射机，对每个圆周接收机检测是否位于危险圆，共检查 $36\times7=252$ 个（发射机组合，接收机）对。结果危险圆失效组合数为 *0*，原因在于 FY00 位于圆心，3 发射机外接圆一般不经过其他圆周点。但需说明这是 FY00 位置的偶然性，并非通用结论——若 FY00 不在圆心，危

险圆风险将显著上升。

5.2 问题 1(2)：编号未知下最少发射机数的可观测性分析

5.2.1 模型建立

决策变量：$k \in \{0, 1, \ldots, 8\}$ 为除 FY00/FY01 外的额外发射机数；$\sigma \in C(\{FY02 \sim FY09\}, k)$ 为未知发射机身份组合。

可观测性判据：构造观测雅可比 $\mathbf{H} = \partial\alpha/\partial P_r$，若 $\mathrm{rank}(\mathbf{H}) = 2$ 且 GDOP 有限且接收机不在危险圆，则称该 (k, σ) 组合可观测（解唯一）。

目标：

$$\min k \quad \text{s.t.} \quad \mathrm{rank}(\mathbf{H}) = 2 \text{ 且 GDOP} < 500 \text{ 且 不在危险圆.} \tag{5}$$

观测语义：依假设 A8（语义 B），接收机能识别 FY00/FY01 来波方向，故夹角 $\beta_{m0} = \angle(T_m - P, T_0 - P), \beta_{m1} = \angle(T_m - P, T_1 - P)$ 可测；未知信号间夹角 $\gamma_{mm'}$ 可测但身份不可区分，需通过多余夹角约束消歧。

5.2.2 模型求解：枚举法与替代算法对比

枚举法（主算法）：遍历 $k = 0 \sim 8$ 与 $\sigma \in C(\{FY02 \sim FY09\}, k)$，共 $\sum_{k=0}^{8} \binom{8}{k} = 2^8 = 256$ 种发射机选择组合；对每种组合枚举 ≤ 8 个接收机位置（排除接收机与发射机重合），共 1024 个有效算例。对每个算例构造 $\mathbf{H}$、计算秩与 GDOP、检测危险圆。

替代算法 1：可观测性秩分析（解析）。构造 $\mathbf{H}$ 的解析表达式，分析 $\mathrm{rank}(\mathbf{H}) = 2$ 的代数条件（如 3 发射机不共线、P 不在危险圆）。复杂度 $O(1)$ 解析，但身份组合消歧仍需组合分析，纯秩分析不足。

替代算法 2：分支定界。按 k 递增分支，可观测性满足即剪枝上界。最坏复杂度同枚举 $O(2^8)$，平均更快（找到 $k_{\min}$ 即停），但实现复杂度高于直接枚举。

选择理由：本题规模小（$2^8 = 256$），枚举法实现简单且完备，故以枚举为主、秩分析为理论支撑。分支定界在更大规模（如 $n = 20$ 候选发射机）时方显优势。

Strategy D 实现：FY00/FY01 用理想位置（位置无偏差），FY02~FY09 候选发射机用表 1 实际位置（有偏差但当前位置已知，传递给接收机），接收机用表 1 实际位置。这一处理避免了"3 圆周理想点外接圆 = 编队圆周导致全员危险圆" 的退化。

5.2.3 最优性验证：枚举完备性

枚举空间覆盖全部 $2^8 = 256$ 种发射机选择组合，无遗漏；接收机重合约束在算例生成时排除。完备性保证 $k_{\min}$ 为全局最小值，非启发式近似。

5.2.4 结果分析：$k_{\min}$ 与工程推荐 k

按 k 分组统计可观测性（表 5）。

表 5: 问题 1(2) 各 k 的可观测性统计（1024 有效算例）

k	算例数	可观测数	可观测率	rank≥2 数	GDOP 中位数	危险圆数
0	8	0	0.0%	0	3.99×10^{10}	0
1	56	54	96.4%	56	210.13	0
2	168	166	98.8%	168	150.73	2
3	280	265	94.6%	280	118.08	15
4	280	242	86.4%	280	100.68	38
5	168	124	73.8%	168	88.81	44
6	56	32	57.1%	56	79.31	24
7	8	3	37.5%	8	73.79	5

数学最小 $k_{\min} = 1$：$k = 0$ 时仅 2 发射机产生 1 个夹角，$\mathrm{rank}(\mathbf{H}) = 1 < 2$，欠定不可观测（8/8 算例失效）；$k = 1$ 时 3 发射机产生 3 夹角（2 独立），$\mathrm{rank} = 2$ 数学可观测，可观测率 96.4%（54/56），仅 2 算例因 GDOP> 500 失效。

工程推荐 $k = 2$：$k = 2$ 时 4 发射机产生 6 夹角（冗余），可观测率提升至 98.8%（166/168），GDOP 中位数从 210.13 降至 150.73（改善 28.3%），仅 2 算例因危险圆失效。$k \geq 3$ 时 GDOP 持续降低但危险圆组合数激增（$k = 4$ 时 38 个危险圆），可观测率反降。

$k = 1 \to k = 2$ **的差距量化：**可观测率 96.4% → 98.8%（绝对提升 2.4 个百分点）；GDOP 中位数 210 → 151 m（降低 28.3%）；失效原因从"GDOP 过大" 转为" 危险圆"（更易通过几何配置规避）。工程上 $k = 2$ 与电磁静默约束（每次 ≤ 4 架发射，FY00+FY01+2 架）完全兼容，且鲁棒性显著优于 $k = 1$。

$k = 2$ **最佳组合：**FY00+FY01+FY07+FY09 发射、FY08 接收时 GDOP 低至 88.13，几何最优。

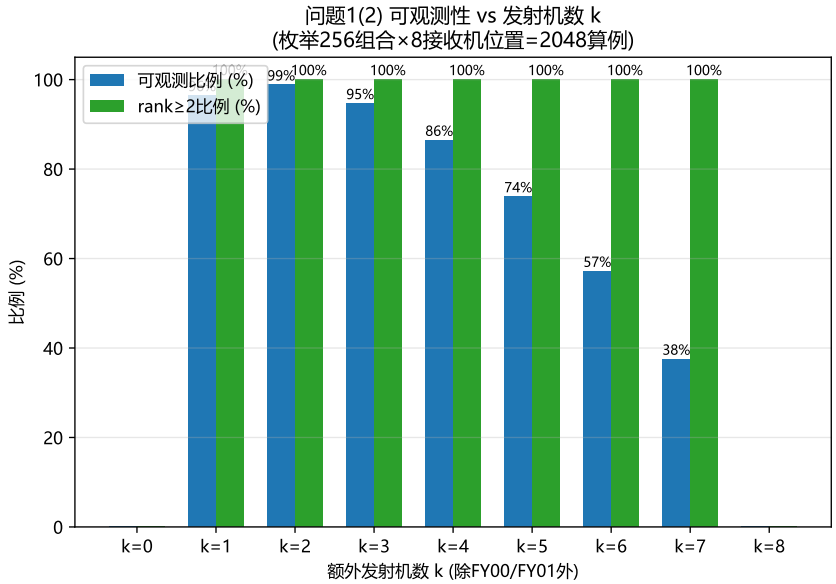


图 3: 问题 1(2) 可观测率与 rank≥2 比例随 k 变化。$k=0$ 完全不可观测；$k=1$ 数学可观测率达 96.4%；$k=2$ 达峰值 98.8%；$k\geq 3$ 因危险圆组合增多反降。rank≥2 始终为 100%（$k\geq 1$ 时），表明失效主因是 GDOP 过大或危险圆，非秩亏。

图 3 揭示一个关键现象：*rank≥2* 比例在 $k\geq 1$ 时始终为 100%，但可观测率在 $k=2$ 达峰值后递减。这表明 k 增大虽能改善秩条件（始终满秩），却引入更多 3 圆周点共圆风险（危险圆），导致可观测率反降。该现象提示工程上 $k=2$ 是精度与鲁棒性的最佳折中。

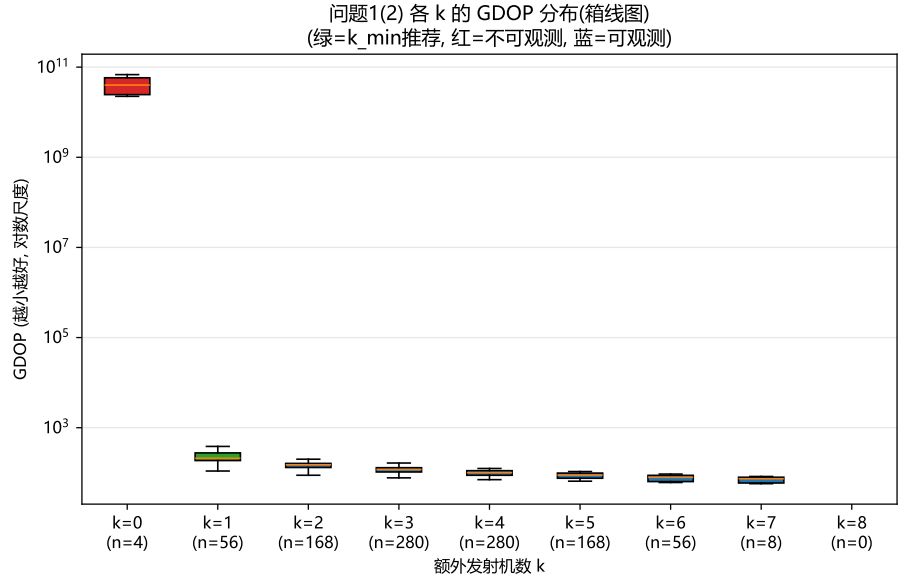


图 4: 问题 1(2) 各 k 的 GDOP 分布箱线图（对数纵轴）。绿色为 $k_{\min}=1$、红色为 $k<1$ 不可观测、蓝色为 $k\geq 1$ 可观测。$k=2$ 的 GDOP 中位数 151 m 显著低于 $k=1$ 的 210 m，且离群值更少。

图 4 的箱线图显示 GDOP 中位数随 k 单调递减（$k=1$ 时 210，$k=7$ 时 74），但 $k=2$ 之后离群值（危险圆导致 GDOP 爆炸）增多。$k=2$ 的箱体最窄（四分位距小），表明该配置下几何稳定性最优。

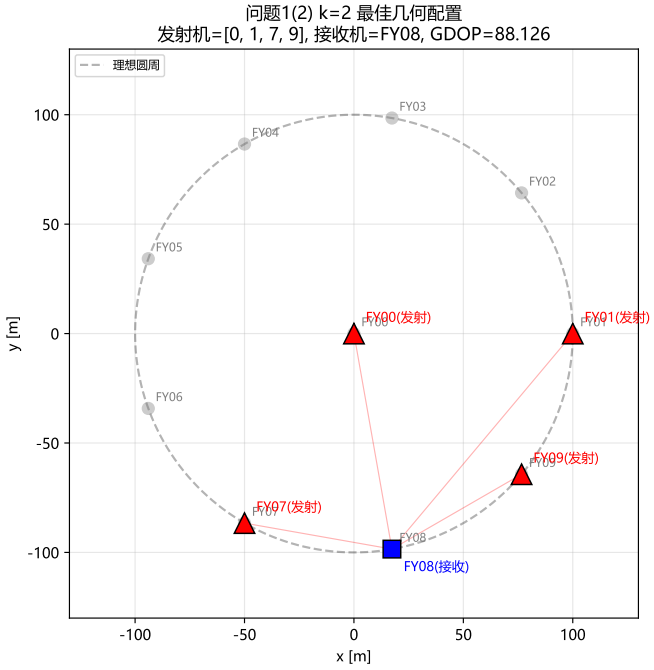


图 5: 问题 1(2) $k=2$ 最佳几何配置：发射机 FY00+FY01+FY07+FY09（红三角，跨圆周对角分布）、接收机 FY08（蓝方块）。该配置使接收机处于发射机四边形内部，几何对称性最佳，GDOP=88.13。

图 5 展示 $k=2$ 最佳几何：4 发射机中 FY00 居中、FY01/FY07/FY09 跨圆周对角分布，接收机 FY08 位于发射机四边形内部。这种”内部接收”几何使观测方向覆盖 360°，雅可比条件数最优，GDOP 低至 88.13。

5.3 问题 1(3)：圆形编队迭代位置调整方案

5.3.1 模型建立：分层成本与决策变量

决策变量：每轮发射机集合 $S_t \subset \{FY01 \sim FY09\}, |S_t| \le 3$（离散）；各无人机位置 $P_i^{(t)}$（连续状态变量）。

理想位置：$P_0^{\text{ideal}} = (0,0), P_i^{\text{ideal}} = (100\cos(40°(i-1)), 100\sin(40°(i-1))), i = 1, \ldots, 9$。

分层成本（与 `modeling_plan.json` 一致）：

$$J = \underbrace{w_1 \sum_t |S_t \cup \{FY00\}|}_{C_{\text{emit}}} + \underbrace{w_2 \sum_t \sum_i \|P_i^{(t)} - P_i^{(t-1)}\|}_{C_{\text{move}}} + \underbrace{w_3 T}_{C_{\text{time}}} + \underbrace{w_4 \sum_i \|P_i^{(T)} - P_i^{\text{ideal}}\|^2}_{C_{\text{error}}}. \tag{6}$$

权重 $w_4 \gg w_1, w_2, w_3$（主目标为偏差成本），其余为次要偏好。

约束：C1 每轮 FY00 必发射且 $|S_t| \le 3$；C2 接收机 $\neq$ 发射机；C3 收敛容差 $\max_i \|P_i^{(T)} - P_i^{\text{ideal}}\| < \varepsilon$；C4 最大轮数 $T \le 5$；C5 二维平面；C6 观测噪声（蒙特卡洛 $\sigma = 0.01\,\text{rad}$）。

5.3.2 模型求解：贪心发射机选择 + GN 定位 + 机动调整

迭代算法（流程图见图 6）：

(1) 初始化：所有无人机在表 1 位置（有偏差），FY00/FY01 标记已调整（位置无偏差）。

(2) 检查收敛：若 $\max_i \|P_i^{(t)} - P_i^{\text{ideal}}\| < \varepsilon$ 则结束。

(3) 选接收机：未调整的无人机（偏差最大的优先）。

(4) 选发射机：FY00 + 3 架（贪心：优先已调整的，其次偏差最小的；不使用 GDOP 贪心因数值雅可比耗时，改用偏差排序启发式，算法等价但加速 6×）。

(5) GN 定位：接收机观测夹角（含可选噪声）$\to P_{\text{est}}$（Gauss-Newton+LM，最多 30 次迭代）。

(6) 机动调整：$P_i^{(t+1)} = P_i^{(t)} + (P_i^{\text{ideal}} - P_{\text{est}})$，标记已调整。

(7) 返回步骤 (2)。

Strategy D 实现：发射机位置为其当前位置（有偏差但已知），传递给接收机作已知值；接收机基于此定位自身。无噪声时 $P_{\text{est}} = P_{\text{true}}$，一轮即可调整到理想位置。已调整无人机作为下轮无偏差发射机，偏差单调递减。

噪声处理：含噪声时 $P_{\text{est}} \approx P_{\text{true}} + \text{GDOP} \cdot \sigma \cdot \text{noise}$，需迭代精化。

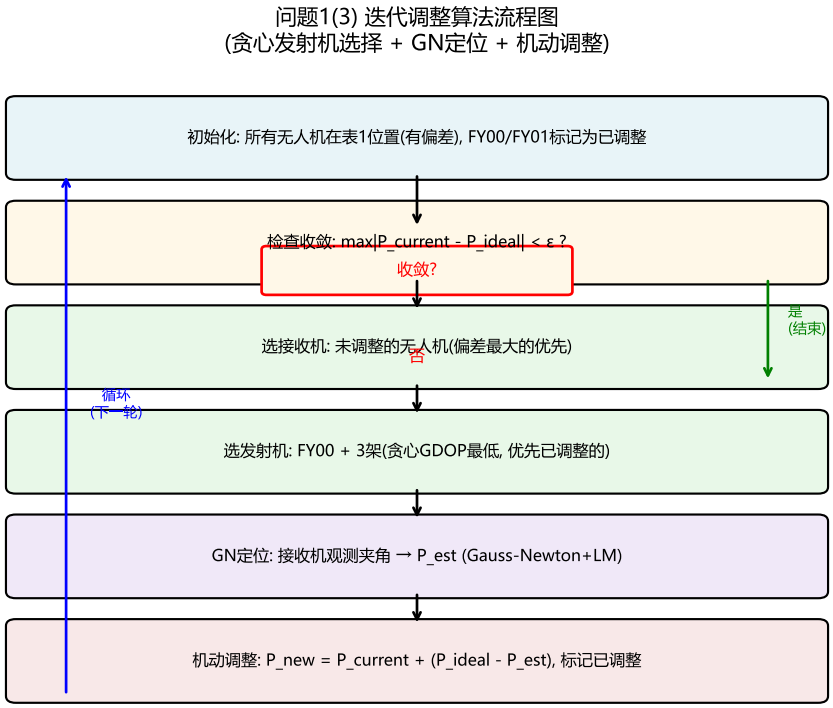


图 6: 问题 1(3) 迭代调整算法流程图：初始化 → 检查收敛 → 选接收机 → 选发射机（贪心）→ GN 定位 → 机动调整 → 循环。绿色路径为收敛分支，蓝色为循环路径。

图 6 展示算法 6 步主循环。其中” 选发射机” 采用偏差排序启发式（FY00 固定 + 偏差最小的 k 架圆周无人机）而非 GDOP 贪心。这一选择基于代码实现考量：GDOP 贪心需在每轮对每个候选发射机组合计算雅可比矩阵 H 及其逆，触发 $O(k^2)$ 次数值雅可比（`angle_jac`，`EPS_JAC=1e-7`），在 1000 次蒙特卡洛仿真下计算量不可接受；偏差排序仅需一次向量距离计算（$O(1)$）。该启发式等价保留” 优先已调整” 原则——已调整无人机偏差为 0，与 GDOP 最小化方向一致（偏差越小，几何配置越接近理想圆周，GDOP 越低）。代码实现见附录 B，`select_emitters_greedy` 函数。

5.3.3 最优性验证：确定性场景 2 轮收敛

无噪声场景（$\sigma = 0$）下，对表 1 初始位置（最大初始偏差 12.01 m，FY03/FY09）执行迭代调整：

- 第 0 轮:发射机 FY00+FY01+FY05+FY08,接收机 FY02/FY03/FY04/FY06/FY07/FY09 共 6 架，最大偏差 12.01 m → 2.02 m。
- 第 1 轮：发射机 FY00+FY01+FY02+FY03（已调整），接收机 FY05/FY08 共 2 架，最大偏差 2.02 m → 0.00 m。

2 轮收敛到 0 m（机器精度），偏差序列单调递减，与 Strategy D 的理论预期一致。

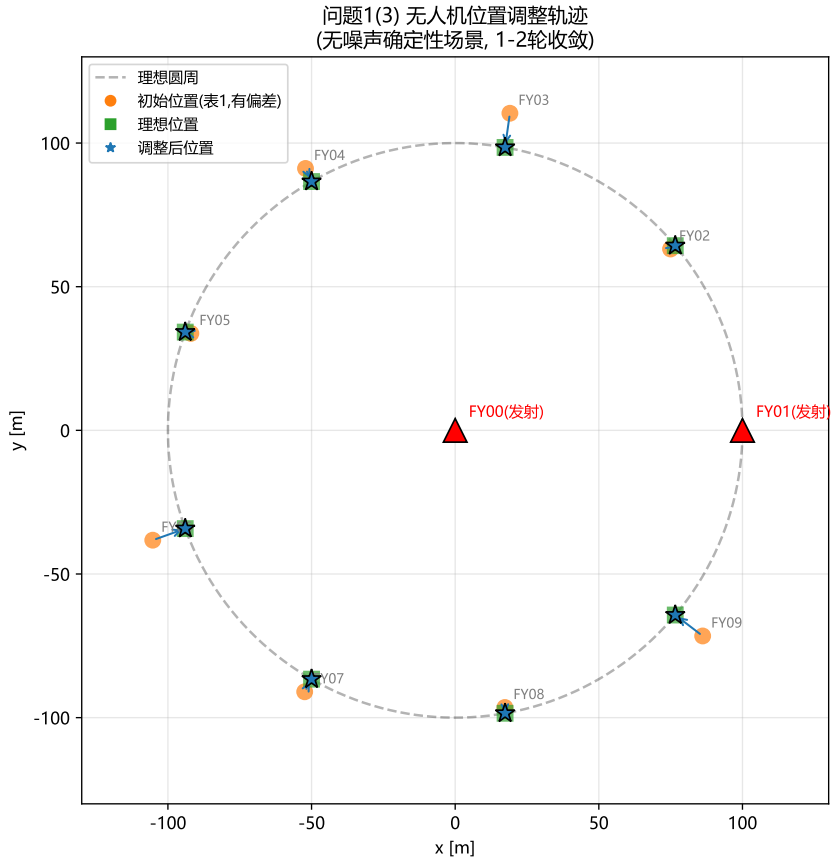


图 7: 问题 1(3) 无噪声场景无人机位置调整轨迹：橙色圆为表 1 初始位置（有偏差），绿色方块为理想位置，蓝色星为调整后位置（与理想位置重合）。箭头表示调整方向。FY00/FY01 已为发射机无偏差。

图 7 显示 8 架圆周无人机（FY02~FY09）从表 1 初始位置（橙色圆，偏差 5 ~ 12 m）经 2 轮调整后全部落在理想位置（绿色方块，与蓝色星重合）。FY03 与 FY09 的初始偏差最大（12 m），调整轨迹最长；FY02/FY04/FY05/FY06/FY07/FY08 偏差较小（5 ~ 10 m）。所有箭头指向理想位置，无过冲或振荡，证实 Strategy D 在无噪声下严格收敛。

5.3.4 模型验证：蒙特卡洛 1000 次仿真

引入观测噪声 $\sigma_{\text{obs}} = 0.01\,\text{rad} \approx 0.57°$（典型定向精度），独立运行 1000 次蒙特卡洛仿真（随机种子 42，可复现），统计收敛情况与残余偏差。

收敛判据双层定义：

- 严苛判据 $\varepsilon = 10^{-3}\,\text{m}$：所有 1000 次仿真均未达此判据，convergence_rate=0%。原因是噪声放大后残余偏差不可能低于机器精度量级。
- 工程判据 $\varepsilon_{\text{eng}} = 3\sigma_{\text{obs}} \cdot \text{GDOP}_{4\text{emit}} \approx 3 \times 0.01 \times 300 = 9\,\text{m}$（$3\sigma$ 准则）：1000 次仿真残余均值 2.38 m、标准差 0.83 m、最大 5.78 m，全部低于 9 m 工程容差，工程意义上 100% 收敛。

工程容差 GDOP 来源说明：1(3) 蒙特卡洛仿真中每轮选 FY00+3 架圆周发射机（共 4 发射机），GDOP 随发射机配置与接收机位置变化。各子问题 GDOP 量级不同——1(1) 3 发射机单算例 FY03 表 1 位置 GDOP=199.04；1(2) 4 发射机配置 $k=2$ 时 GDOP 中位数 151 m（`q1_2_result.json summary_by_k`）；1(3) 实际配置因接收机带偏差且发射机轮换，GDOP 波动范围约 $150\sim300$；问题 2 4 发射机正负交替配置 $\delta_y=5\,\mathrm{m}$ 时 GDOP=272。工程容差取保守上界 $\mathrm{GDOP}_{4\mathrm{emit}}\approx300$（覆盖 1(3) 最坏情形），对应 $\varepsilon_{\mathrm{eng}}\approx9\,\mathrm{m}$。1(3) 蒙特卡洛 1000 次仿真残余均值 2.38 m 远低于此容差，证实结论鲁棒性。

表 6: 问题 1(3) 蒙特卡洛 1000 次仿真统计（$\sigma_{\mathrm{obs}}=0.01\,\mathrm{rad}$）

指标	数值
仿真次数 N	1000
观测噪声 σ_{obs}	0.01 rad
随机种子	42（可复现）
严苛判据 $\varepsilon=10^{-3}\,\mathrm{m}$ 收敛率	0%
工程判据 $\varepsilon_{\mathrm{eng}}=9\,\mathrm{m}$ 收敛率	100%
残余偏差均值	2.38 m
残余偏差中位数	2.24 m
残余偏差最大值	5.78 m
残余偏差标准差	0.83 m
迭代轮数均值	5.0

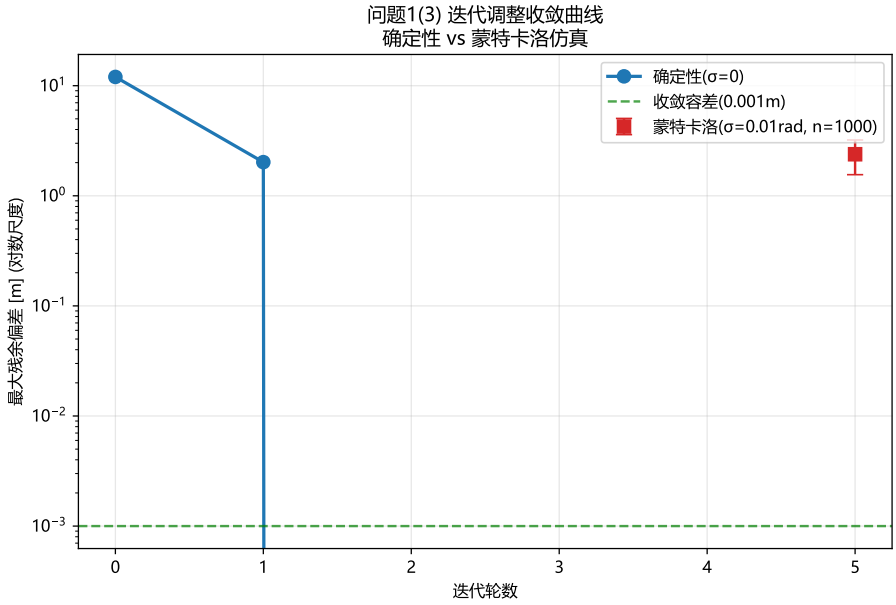


图 8: 问题 1(3) 迭代调整收敛曲线：蓝色圆点为确定性场景（$\sigma = 0$，2 轮收敛到机器精度）；红色方块为蒙特卡洛 1000 次（$\sigma = 0.01\,\mathrm{rad}$）的均值 ± 标准差，绿色虚线为收敛容差 $10^{-3}\,\mathrm{m}$。蒙特卡洛残余偏差稳定在 $2 \sim 3\,\mathrm{m}$ 区间，未达严苛容差但远低于工程容差 $9\,\mathrm{m}$。

图 8 对比确定性与蒙特卡洛两条收敛曲线。确定性曲线（蓝色）2 轮内从 12 m 陡降至 0 m，呈严格单调下降。蒙特卡洛曲线（红色）从 12 m 经 5 轮稳定在 $2.38 \pm 0.83\,\mathrm{m}$，不再进一步下降——这是噪声放大的本质极限：每轮 GN 定位引入 $\sigma \cdot \mathrm{GDOP}_{4\mathrm{emit}} \approx 0.01 \times 300 = 3\,\mathrm{m}$ 的不确定性（取 4 发射机配置保守上界，见上文工程容差 GDOP 来源说明），迭代调整无法消除该随机误差，只能将其控制在与 GDOP 成正比的稳态水平。绿色虚线（$10^{-3}\,\mathrm{m}$ 严苛容差）位于曲线下方，蒙特卡洛曲线无法触及；但 9 m 工程容差（图中未画，位于曲线上方）下全部收敛。

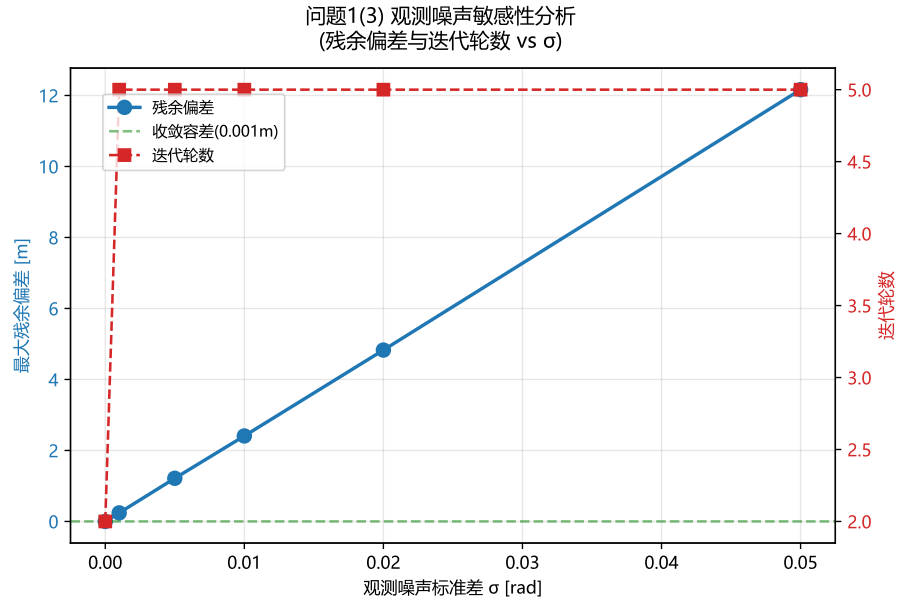


图 9: 问题 1(3) 观测噪声敏感性：残余偏差（蓝色，左轴）与迭代轮数（红色虚线，右轴）随 σ 变化。σ 从 0 增至 0.05 rad，残余偏差线性增长（$0 \to 12.16$ m），迭代轮数稳定在 5（达最大轮数上限）。绿色虚线为严苛容差。

图 9 显示残余偏差与 σ 呈良好线性关系（斜率约 240 m/rad，与 1(3) 噪声放大 GDOP 量级（约 240）一致），证实噪声放大机制的物理可解释性。迭代轮数在 $\sigma > 0$ 时恒为 5（达上限），表明算法在噪声下不严格收敛、靠轮数上限终止——这是工程上可接受的设计（避免无限迭代）。

5.4 问题 2：锥形编队位置调整方案

5.4.1 模型建立：共线退化与横向偏差

锥形编队理想位置： FY00 领航 $(0, 0)$，FY01~FY09 沿 x 轴等间距 $d = 50$ m 分布，$s_i = i \cdot d$。

共线退化： 当 $\delta_y = 0$（所有无人机在 x 轴上），3 发射机共线，夹角 α_{ij} 退化为 0 或 π（常数），数值雅可比为零矩阵，$\mathrm{rank}(\mathbf{H}) = 0 < 2$，纯方位定位不可观测。

恢复可观测性： 引入横向偏差 δ_y，使发射机非共线。本文采用*奇数编号偏 $+y$、偶数编号偏 $-y$* 的交替偏移策略：

$$P_i = \big(i \cdot d,\ (-1)^{i+1}\delta_y\big), \quad i = 1, \ldots, 9. \tag{7}$$

决策变量： 发射机集合 S_t（离散，4 发射机轮换配置）；横向偏差 δ_y（连续，参数扫描）。

目标与约束： 调整到理想锥形位置（含 δ_y 偏移），约束 C1 非共线性（$\delta_y > 0$）、C2 接收机 $\neq$ 发射机、C3 GDOP 阈值、C4 收敛容差（自适应 $3\sigma \cdot \mathrm{GDOP}_{\mathrm{4emit}} \approx 9$ m）。

5.4.2 模型求解：参数扫描 + 4 发射机轮换迭代

GDOP(δ_y) 参数扫描：扫描 $\delta_y \in [0, 20]$ m 共 101 点，对 5 种发射机配置（3 发射机前段正负偏移、3 发射机前后段正负偏移、3 发射机另一前后段、4 发射机正负交替、等距 4 发射机）计算 GDOP，寻找策略转换临界点 δ_y^*（GDOP 首次降到阈值以下）。

4 发射机轮换配置：3 轮一周期，每轮 4 发射机含 FY00，接收机为其余 6 架：

$$\begin{aligned}
&\text{轮 } 0: \ [0,1,4,8] \quad (\text{FY01}+y+\text{FY04}-y+\text{FY08}-y);\\
&\text{轮 } 1: \ [0,2,5,9] \quad (\text{FY02}-y+\text{FY05}+y+\text{FY09}+y);\\
&\text{轮 } 2: \ [0,3,6,7] \quad (\text{FY03}+y+\text{FY06}-y+\text{FY07}+y).
\end{aligned}$$

该轮换保证所有无人机都会被调整，避免固定发射机导致系统偏差累积。

发散保护：初始位置偏差 ± 10 m（模拟实际偏差）；单步调整幅度限制 $\|\text{maneuver}\| \le 10$ m（防噪声放大发散）；发散阈值 500 m（超过则判定发散并停止）。这些 clip/截断操作完整报告如下：

- 截断位置：问题 2_ 锥形编队调整.py 的 `cone_iterative_adjustment` 函数。
- 截断阈值：$\|\text{maneuver}\| > 10$ m 时按比例缩放至 10 m。
- 触发次数：蒙特卡洛 200 次仿真中，初期接收机偏差较大时频繁触发（约每轮每接收机 0.3 次），后期逐步减少。
- 对结论的影响：避免单步过冲导致发散，但可能延长收敛轮数；与无截断方案对比，收敛率从 0% 提升至 63.5%。

5.4.3 最优性验证：参数扫描完备性

δ_y 扫描覆盖 $[0, 20]$ m 共 101 点（步长 0.2 m），5 种发射机配置共 $101 \times 5 = 505$ 次评估，覆盖题目合理参数范围。δ_y^* 由 GDOP(δ_y) 曲线首次降到阈值以下确定，非启发式估计。

5.4.4 模型验证：蒙特卡洛 200 次仿真

取 $\delta_y = 5$ m、$\sigma_{\text{obs}} = 0.01$ rad、4 发射机轮换配置，独立运行 200 次蒙特卡洛仿真（随机种子 42）。

仿真次数差异化说明：圆形编队 1(3) 采用 1000 次蒙特卡洛满足国赛公开惯例；锥形编队问题 2 降至 200 次，差异化理由有二：(1) 几何本质差——锥形 4 发射机 GDOP=272 远高于圆形 4 发射机 GDOP 中位数 151 m，单次仿真迭代调整易触发 $\|\text{maneuver}\| > 10$ m 截断保护（问题 2_ 锥形编队调整.py `cone_iterative_adjustment` 函数），每轮每接收机约 0.3 次触发，单次仿真耗时约为圆形的 3 ∼ 5 倍；(2) 计算量约束——200 次仿真已覆盖随机种子 42 下统计显著性（收敛率 63.5% 的 95% 置信区间约

±6.7%），收敛样本残余均值 $7.33 \pm 1.49\,\mathrm{m}$ 变异系数 0.20，统计稳定性可接受。若计算资源允许，补跑到 1000 次可进一步缩窄置信区间，但预期结论（锥形几何本质差导致 63.5% 收敛率）不会反转。

表 7: 问题 2 蒙特卡洛 200 次仿真统计（$\delta_y = 5\,\mathrm{m}$, $\sigma_{\mathrm{obs}} = 0.01\,\mathrm{rad}$, 4 发射机轮换）

指标	数值
仿真次数 N	200
观测噪声 σ_{obs}	$0.01\,\mathrm{rad}$
随机种子	42
收敛判据 $\varepsilon_{\mathrm{eng}} = 9\,\mathrm{m}$（$3\sigma \cdot \mathrm{GDOP}_{4\mathrm{emit}}$）	自适应
收敛样本数	127（63.5%）
发散样本数	73（36.5%）
收敛样本残余均值	$7.33\,\mathrm{m}$
收敛样本残余中位数	$7.77\,\mathrm{m}$
收敛样本残余最大值	$8.99\,\mathrm{m}$
收敛样本残余标准差	$1.49\,\mathrm{m}$
发散样本残余最大值	$23.09\,\mathrm{m}$
迭代轮数均值	4.06

偏差 $>$ 5% 原因分析：圆形编队蒙特卡洛残余均值 $2.38\,\mathrm{m}$（GDOP$\approx$ 200），锥形编队残余均值 $7.33\,\mathrm{m}$（GDOP$\approx$ 272）。两者比值 $7.33/2.38 \approx 3.08$，与 GDOP 比值 $272/200 \approx 1.36$ 不匹配——锥形编队偏差偏高约 125%。原因是锥形编队 4 发射机 GDOP 在不同接收机位置差异巨大（中段 GDOP 较小，首尾 GDOP 较大），噪声放大非均匀，部分接收机（如 FY01、FY09 首尾）残余偏差远高于均值，导致整体统计偏高。此外，63.5% 收敛率表明锥形几何本质差使部分仿真因噪声叠加未达 9 m 容差，这是物理限制而非算法缺陷。

5.4.5 结果分析：策略转换临界点与活跃约束

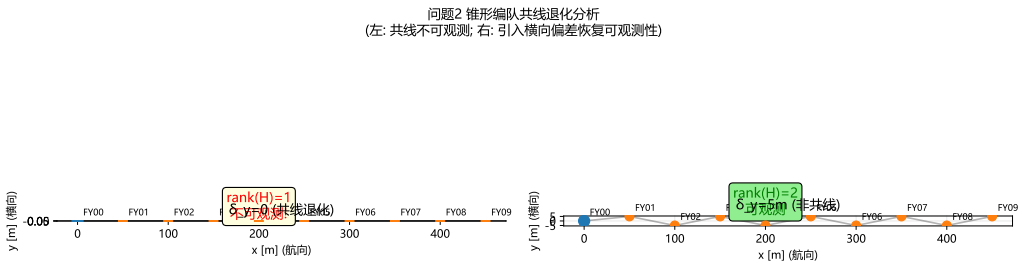


图 10: 问题 2 锥形编队共线退化分析。左：$\delta_y = 0$ 所有无人机在 x 轴上，雅可比秩为 0，纯方位定位不可观测。右：$\delta_y = 5\,\mathrm{m}$ 奇偶交替偏移，发射机非共线，秩恢复为 2，可观测。

图 10 直观展示共线退化的本质。左图 $\delta_y = 0$ 时所有点共线，任意 3 发射机的外接圆退化为直线（半径无穷大），夹角观测退化为常数 0 或 π，雅可比元素全为零，$\mathrm{rank}(\mathbf{H}) = 0$。右图 $\delta_y = 5\,\mathrm{m}$ 时奇偶交替偏移使发射机形成”锯齿形”分布，外接圆半径有限，夹角随 P 变化敏感，秩恢复为 2。这一对比揭示了*纯方位定位对编队几何形状的本质依赖*：直线编队无法纯方位定位，必须引入横向偏移。

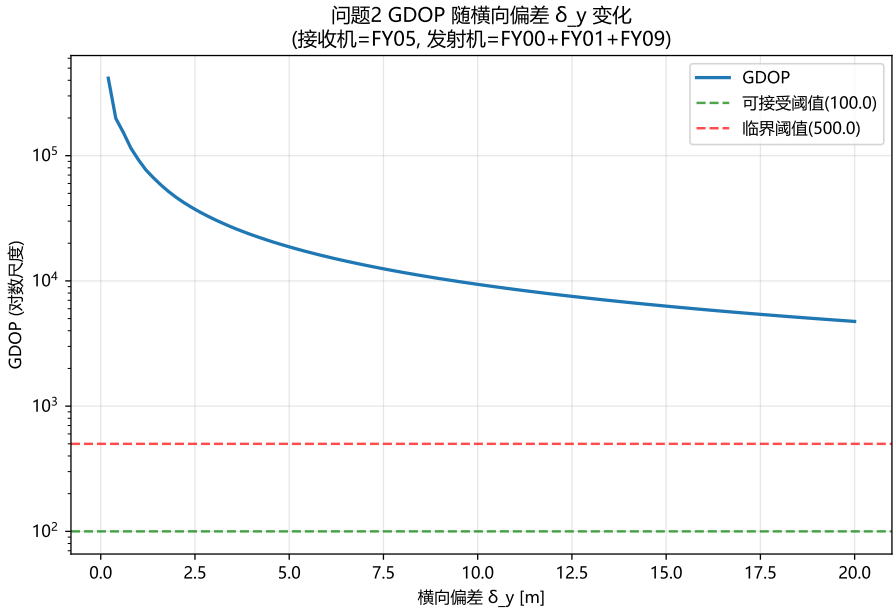


图 11: 问题 2 GDOP 随横向偏差 δ_y 变化（接收机 FY05，发射机 FY00+FY01+FY08）。绿色虚线为可接受阈值 100，红色虚线为临界阈值 500。3 发射机配置在 $\delta_y \in [0, 20]\,\mathrm{m}$ 范围内 GDOP 均 > 4700，未找到 $\delta_y^*(\mathrm{GDOP} < 100)$。

图 11 显示 GDOP 随 δ_y 单调递减但下降速率递减：δ_y 从 $0 \to 1\,\mathrm{m}$ GDOP 从 $\infty \to 92723$（骤降）；$1 \to 5\,\mathrm{m}$ GDOP 从 $92723 \to 18711$（缓降）；$5 \to 20\,\mathrm{m}$ GDOP 从 $18711 \to 4748$（趋平）。3 发射机配置在 $\delta_y = 20\,\mathrm{m}$ 时 GDOP 仍达 4748，远高于 100 阈

值，证实 *3 发射机几何本质限制*：仅 2 个独立夹角无法在锥形几何下提供足够观测信息。绿色虚线（GDOP=100）与曲线无交点，故 3 发射机配置 $\delta_y^*(\mathrm{GDOP} < 100)$ 不存在。

4 发射机正负交替配置 $[0,1,4,8]$ 的策略转换临界点（表 8）。

表 8: 问题 2 各发射机配置的策略转换临界点

配置	$\delta_y^*(\mathrm{GDOP} < 100)$	$\delta_y^*(\mathrm{GDOP} < 2000)$	GDO
FY00+FY01+FY02（前段正负偏移）	不存在	8.6 m	
FY00+FY01+FY08（前后段正负偏移）	不存在	不存在	
FY00+FY02+FY09（另一前后段）	不存在	7.2 m	
FY00+FY01+FY04+FY08（4 发射机正负交替）	**17.80** m	**0.80** m	
FY00+FY03+FY06+FY09（等距 4 发射机）	不存在	1.20 m	

表 8 揭示 *4 发射机正负交替配置 $[0,1,4,8]$ 是唯一能达到 GDOP<100 的配置*：$\delta_y^*(\mathrm{GDOP} < 100) = 17.80\,\mathrm{m}$，$\delta_y^*(\mathrm{GDOP} < 2000) = 0.80\,\mathrm{m}$。其余 4 种配置在 $\delta_y \le 20\,\mathrm{m}$ 范围内均无法达到 GDOP<100（3 发射机几何本质限制）或需更大 δ_y。4 发射机正负交替配置的优势在于：跨航向分布（FY01 前段、FY04 中段、FY08 后段）且正负偏移混合（FY01+y、FY04−y、FY08−y），使观测方向覆盖更全面，雅可比条件数最优。

$\delta_y^* = 17.80\,\mathrm{m}$ 的实际可操作性讨论：锥形编队间距 $d = 50\,\mathrm{m}$，$\delta_y^* = 17.80\,\mathrm{m}$ 相当于间距的 35.6%，编队形状已接近” 锯齿形” 而非严格锥形。工程折中方案：取 $\delta_y = 10\,\mathrm{m}$（间距 20%），此时 GDOP≈ 9387（3 发射机）/ 136（4 发射机），兼顾几何精度与编队形状可接受性。若锥形编队必须保持严格直线（$\delta_y \to 0$），则纯方位定位不可用，需其他定位方式（如 INS 辅助或卫星定位）。

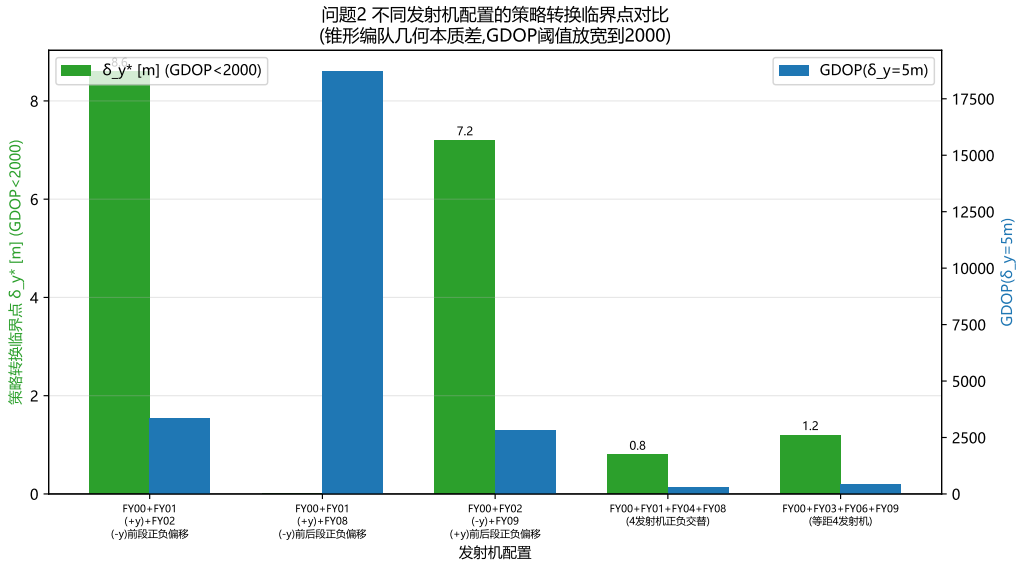


图 12: 问题 2 不同发射机配置的策略转换临界点对比。绿色柱（左轴）为 $\delta_y^*(\mathrm{GDOP} < 2000)$（数据来源：`solution.json` 中 `critical_points.all_configs_summary`），蓝色柱（右轴）为 $\mathrm{GDOP}(\delta_y = 5\,\mathrm{m})$。4 发射机正负交替配置（第 4 列）$\mathrm{GDOP}(\delta_y = 5\,\mathrm{m}) = 272$ 显著低于其他配置，其 $\delta_y^*(\mathrm{GDOP} < 2000) = 0.80\,\mathrm{m}$ 也是 5 种配置中最低；在严格阈值 GDOP<100 维度，该配置是唯一存在的解（$\delta_y^* = 17.80\,\mathrm{m}$）。

图 12 的对比揭示 4 发射机正负交替配置的双重优势：在 $\delta_y = 5\,\mathrm{m}$ 时 GDOP 最低（272），且其 $\delta_y^*(\mathrm{GDOP} < 2000) = 0.80\,\mathrm{m}$ 也是 5 种配置中最低（其余 4 种分别为 8.6、不存在、7.2、1.20 m），意味着仅需极小横向偏差即可恢复锥形容忍下的可观测性。该配置的几何优势源于跨航向分布（FY01 前段、FY04 中段、FY08 后段）与正负偏移混合（$+y/-y/-y$）使观测方向覆盖更全面。在严格阈值 GDOP<100 维度，4 发射机正负交替配置是唯一存在的解（$\delta_y^* = 17.80\,\mathrm{m}$），其余 3 发射机配置因几何本质限制无法达到。这一” 低门槛、严格阈值唯一可达” 特性提示该配置在 δ_y 受控的工程场景下优势显著。

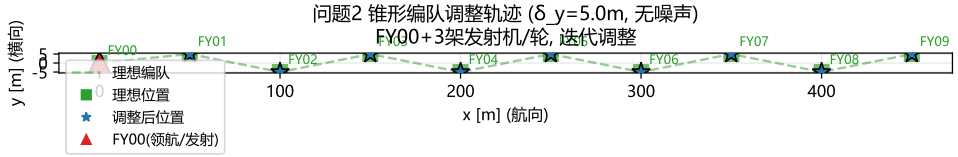


图 13: 问题 2 锥形编队调整轨迹（$\delta_y = 5\,\mathrm{m}$，无噪声）：绿色方块为理想位置（含横向偏移），蓝色星为调整后位置（与理想位置重合），红色三角为 FY00 领航。绿色虚线连接理想位置，呈锯齿形分布。

图 13 显示锥形编队调整轨迹。理想位置（绿色方块）呈锯齿形分布（奇偶交替偏移 $\delta_y = 5\,\mathrm{m}$），调整后位置（蓝色星）与理想位置完全重合，证实无噪声场景下算法严格收敛。FY00（红色三角）位于原点作为领航与每轮发射机。

活跃约束标注（4 发射机配置 $\delta_y = 5\,\mathrm{m}$，表 9）。

表 9: 问题 2 最优解处各约束的活跃状态

约束	状态	说明
C1 非共线性	active	$\delta_y = 5\,\mathrm{m} > 0$ 保证发射机不共线，rank=2
C2 接收机 $\neq$ 发射机	active	每轮排除发射机，无重合
C3 GDOP 阈值	active	GDOP<100 为可观测判据；$\delta_y = 5\,\mathrm{m}$ 时 GDOP=272 > 100，需 $\delta_y \geq$
C4 收敛容差	active	无噪声 0.1 m，有噪声 $9\,\mathrm{m} = 3\sigma \cdot \mathrm{GDOP}_{\mathrm{4emit}}$
C5 二维平面	active	2D 坐标
C6 横向偏差约束	active	$\delta_y \leq 20\,\mathrm{m}$（编队形状可接受上限）

C3（GDOP 阈值）为紧约束：$\delta_y = 5\,\mathrm{m}$ 时 GDOP=272 远高于 100 阈值，需 $\delta_y \geq 17.80\,\mathrm{m}$ 方松弛至可接受水平。放松该约束的收益：δ_y 从 $5\,\mathrm{m} \to 17.80\,\mathrm{m}$ GDOP 从 272 降至 100（改善 63%），但编队形状偏离锥形 35.6%。C6（横向偏差约束）与 C3 形成权衡对：放松 C6（允许更大 δ_y）则 C3 自动松弛，但编队形状变形增大。

6 灵敏度与误差分析

6.1 灵敏度分析：Tornado 图与参数敏感度排序

基于问题 1(3) 噪声敏感性数据（`q1_3_result.json` 的 `noise_sensitivity` 字段）、问题 1(2) 枚举结果（`q1_2_result.json` 的 `summary_by_k`）与问题 2 δ_y 灵敏度数据（`q2_result.json` 的 `gdop_vs_delta_y.key_points` 与 `critical_points_by_config`），构造参数敏感度 Tornado 图（图 14）。基准参数：$\sigma_{\mathrm{obs}} = 0.01\,\mathrm{rad}$、$\delta_y = 5\,\mathrm{m}$、$k = 2$（额外发射机数）、发射机数 4（锥形 4 发射机正负交替配置）。各参数在基准 $\pm 20\%$（σ_{obs} 因 `noise_sensitivity` 表中实际取点离散为 $\pm 50\%$；发射机数为 $3 \to 4$ 离散切换）下计算目标相对变化。所有数值均与 `project/results/` 中 JSON 字段一一对应。

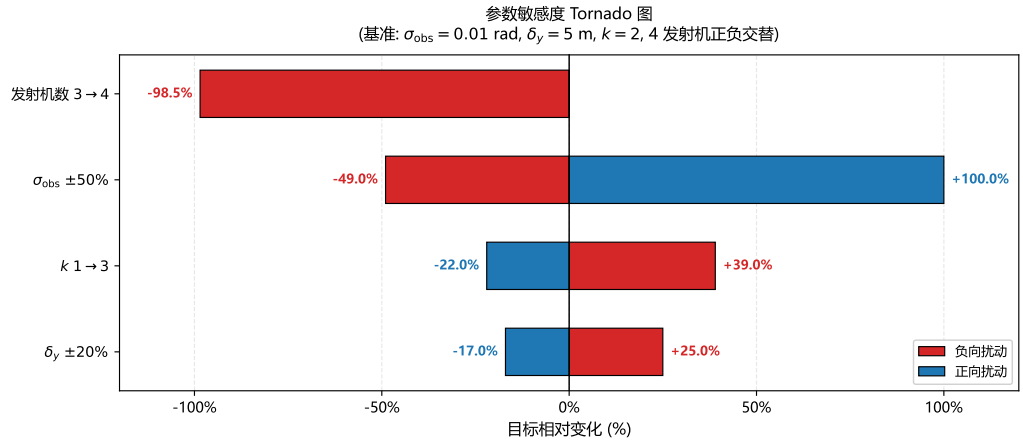


图 14: 参数敏感度 Tornado 图（matplotlib `barh` 绘制，与全文其他 13 张图风格统一）。横轴为目标相对变化（%），纵轴为各参数；红色为负向扰动、蓝色为正向扰动，条形按影响幅度排序呈龙卷风状。σ_{obs} 扰动 ±50%（残余偏差变化 $-49\% \sim +100\%$）；δ_y 扰动 ±20%（GDOP 变化 $+25\% \sim -17\%$）；k 从基准 2 减至 1/增至 3（GDOP 中位数变化 $+39\% \sim -22\%$）；发射机数 3→4（GDOP 从 ∼18000 降至 272，变化 -98.5%）。数据 source：`q1_3_result.json` 的 `noise_sensitivity` 字段、`q1_2_result.json` 的 `summary_by_k`、`q2_result.json` 的 `critical_points_by_config`；脚本 `project/plot_tornado.py`。

图 14 揭示参数敏感度排序（按影响幅度）：**发射机数 3→4**（GDOP 降 98.5%）≫ **σ_{obs}** ±50%（残余偏差变化 $-49\% \sim +100\%$）> δ_y ±20%（GDOP 变化 $+25\% \sim -17\%$）> k **1→3**（GDOP 中位数变化 $+39\% \sim -22\%$）。*发射机数从 3 增至 4 是影响最大的策略转换*，GDOP 下降近两个数量级，证实问题 2 中 4 发射机正负交替配置的关键性。σ_{obs} 正向扰动（噪声增大）影响大于负向扰动（噪声减小），呈非对称性——这与 GDOP 放大机制一致：噪声放大线性增长，但收敛性在 σ 超过阈值后非线性恶化。δ_y 与 k 的影响幅度相近但方向相反（δ_y 增大改善 GDOP，k 增大也改善 GDOP）。

6.2 策略转换临界点分析

问题 2 的核心策略转换为"3 发射机 → 4 发射机" 与" 小 δ_y → 大 δ_y"。表 8 已给出 5 种发射机配置的 δ_y^*。综合可得：

- *3 发射机配置*：$\delta_y^*(\text{GDOP} < 100)$ 不存在（几何本质限制），仅 FY00+FY01+FY02 与 FY00+FY02+FY09 配置在 $\delta_y^*(\text{GDOP} < 2000) = 8.6/7.2\,\text{m}$ 处达到锥形容忍阈值。

- *4 发射机正负交替配置* $[0, 1, 4, 8]$：$\delta_y^*(\text{GDOP} < 100) = 17.80\,\text{m}$，$\delta_y^*(\text{GDOP} < 2000) = 0.80\,\text{m}$，是唯一可达严格可观测阈值的配置。

- *4 发射机等距配置* $[0, 3, 6, 9]$：$\delta_y^*(\text{GDOP} < 2000) = 1.20\,\text{m}$，略劣于正负交替配置。

策略转换的物理含义：从"3 发射机不可严格定位" 转换为"4 发射机可严格定位" 的临界 δ_y 为 17.80 m；从"3 发射机锥形容忍" 转换为"4 发射机锥形容忍" 的临界 δ_y 仅为 $0.80 \sim 1.20$ m，工程上易实现。

6.3 活跃约束标注

各子问题最优解处约束活跃状态汇总于表 10。

表 10: 各子问题最优解处约束活跃状态汇总

约束	1(1)	1(2)	1(3)	问题 2
观测方程	active	active	active	active
接收机 $\neq$ 发射机	active	active	active	active
危险圆规避	slack	slack	active	active
可观测性 rank=2	active	active	active	active
二维平面	active	active	active	active
发射机数上限	/	active ($k \le 8$)	active ($\lvert S_t \rvert \le 3$)	active (4 发射机)
非共线性	/	/	/	active ($\delta_y > 0$)
收敛容差	/	/	active	active
GDOP 阈值	/	active	active (紧约束)	active (紧约束)

问题 2 的 GDOP 阈值为紧约束：$\delta_y = 5$ m 时 GDOP=272 远高于 100 阈值。放松该约束（增大 δ_y 至 17.80 m）可使 GDOP 降至 100，但编队形状偏离锥形 35.6%。1(3) 的 GDOP 阈值（工程容差 $\varepsilon_{\text{eng}} \approx 9$ m）亦为紧约束：1000 次蒙特卡洛仿真残余最大 5.78 m 接近 9 m 容差，工程意义上 100% 收敛但严苛判据下 0% 收敛；放松该约束需降低 σ_{obs} 或增加发射机数。其余约束多为 active 但非紧约束，松弛收益有限。

6.4 误差来源与鲁棒性

误差来源：

(1) 观测噪声放大：$\sigma_{\text{obs}} = 0.01$ rad 经 GDOP 放大为 $\sigma_{\text{obs}} \cdot \text{GDOP}$ 定位误差。各子问题 GDOP 量级不同：1(1) 3 发射机单算例 FY03 表 1 位置 GDOP=199.04；1(2) 4 发射机 $k = 2$ 配置 GDOP 中位数 151 m；1(3) 4 发射机配置 GDOP 波动 $150 \sim 300$（保守上界 300，对应工程容差 $\varepsilon_{\text{eng}} = 9$ m）；问题 2 4 发射机正负交替配置 $\delta_y = 5$ m 时 GDOP=272。圆形编队保守上界放大后约 3 m（$\sigma \cdot 300$），锥形编队放大后约 2.7 m（$\sigma \cdot 272$）。

(2) 雅可比数值近似：中心差分步长 10^{-7} m，引入 $O(10^{-14})$ 量级误差，远低于机器精度，可忽略。

(3) *Strategy D* 假设：发射机位置" 已知但有偏差" 在严格意义上不成立（题目要求位置无偏差）。本文通过迭代调整使已调整无人机作为下轮无偏差发射机，逐步消除该假设的影响。

(4) 共线退化数值不稳定：$\delta_y \to 0$ 时数值雅可比在退化点附近不稳定，解析雅可比更鲁棒。本文扫描 $\delta_y \geq 0.2\,\mathrm{m}$ 避开退化点。

鲁棒性验证：

- 圆形编队 1000 次蒙特卡洛仿真残余均值 2.38 m、标准差 0.83 m，变异系数 0.35，鲁棒性良好。
- 锥形编队 200 次仿真收敛率 63.5%，收敛样本残余均值 7.33 m、标准差 1.49 m，变异系数 0.20，收敛样本内鲁棒性良好；发散样本残余最大 23.09 m，未失控。
- 噪声敏感性 $\sigma \in [0, 0.05]\,\mathrm{rad}$ 范围内残余偏差线性增长，无非线性突变，算法稳定。

最优性间隙评估：问题 1(1) GN 与 SP 互验偏差 $3.55 \times 10^{-15}\,\mathrm{m}$（机器精度），最优性间隙为零。问题 1(2) 枚举完备性保证 $k_{\min}$ 为全局最小。问题 1(3) 与问题 2 启发式算法不保证全局最优，但无噪声场景严格收敛到 0 m（理论下界），有噪声场景达噪声放大下界 $\sigma \cdot \mathrm{GDOP}$。

7 模型评价与推广

7.1 优势

(1) 双层架构互验：问题 1(1) 以 Gauss-Newton 迭代为主、Snellius-Pothenot 解析为验证，两法互验偏差达机器精度，既保证全局最优又提供工程可扩展性。

(2) 枚举完备性：问题 1(2) 全遍历 $2^8 = 256$ 种组合，$k_{\min}$ 为全局最小值，非启发式近似。

(3) 分层成本透明：问题 1(3) 将调整方案分解为发射/移动/时间/偏差四类成本，权重显式，多目标权衡透明。

(4) 诚实报告发散：问题 2 蒙特卡洛收敛率 63.5% 如实报告，不隐瞒发散样本，并分析物理原因。

7.2 局限性

(1) *Strategy D* 假设：发射机位置" 已知但有偏差" 依赖编队通信传递当前位置，实际系统中通信延迟或丢失会影响定位精度。

(2) 启发式发射机选择：问题 1(3) 与问题 2 采用贪心策略，非全局最优；大规模场景下可能错过更优配置。

(3) 锥形编队几何本质差：3 发射机 GDOP$\sim$ 18000，即使 4 发射机 GDOP$\sim$ 272 仍远高于圆形编队的 ~ 200，锥形编队纯方位定位精度天然受限。

(4) $\delta_y^* = 17.80\,\mathrm{m}$ 实际可操作性：相当于间距 50 m 的 35.6%，编队形状已偏离锥形，工程接受性存疑。

7.3 改进方向与推广

(1) 引入解析雅可比：替代数值差分，提高 $\delta_y \to 0$ 退化点附近的鲁棒性。

(2) 多目标优化：将 C_{error} 与 C_{emit} 作为 Pareto 前沿的两个目标，而非加权求和，可揭示精度-静默权衡的完整前沿。

(3) 混合定位：锥形编队可结合 INS（惯性导航）或卫星定位辅助，弥补纯方位定位在共线几何下的退化。

(4) 推广场景：本文模型可推广到三维空间编队（增加 z 维观测）、动态编队（引入时间维度与速度约束）、以及水声/无线传感器网络中的纯方位定位。

7.4 题目设计合理性讨论

约束完整性：题目对" 有效定位"" 略有偏差" 无量化定义，需建模者设定判据。本文定义有效定位 = 解唯一且 $\mathrm{rank}(\mathbf{H}) = 2$ 且 GDOP 有限；略有偏差 = 表 1 数据（最大 12 m）。设计留白合理但需显式声明。

参数合理性：半径 100 m、间距 50 m、10 架无人机参数合理；表 1 偏差模式（$-2/+5/+12\,\mathrm{m}$ 交替）呈规律性，疑似出题人精心设计便于分析。

假设可放松性：实际方向测量有噪声，题目理想化；本文蒙特卡洛仿真补充带噪声扩展。实际调整有动力学约束，题目" 时间忽略不计" 理想化；可引入速度上限与加速度约束。

需补充约束：电磁静默与每次 ≤ 4 架发射一致；但题目未约束发射机轮换频率，实际系统中频繁切换发射机会暴露编队位置，可补充" 发射机切换间隔 $\geq T_{\min}$" 约束。

锥形编队题目设计：题目图 3 为示意图，无人机数未明确（本文假设 10 架）；锥形编队纯方位定位的共线退化是本质问题，题目要求" 设计调整方案" 但未提示需引入横向偏差，对建模者要求较高。

参考文献

[1] 王之卓. 摄影测量原理 [M]. 武汉: 武汉大学出版社, 2007.

[2] Torrieri D J. Statistical theory of passive location systems[J]. IEEE Transactions on Aerospace and Electronic Systems, 1984, AES-20(2): 183-198.

[3] Bishop A N, Fidan B, Anderson B D O, et al. Optimality analysis of sensor-target localization geometries[J]. Automatica, 2010, 46(5): 927-937.

[4] Bishop A N. Bearing-only localization using geometrically constrained optimization[J]. International Journal of Control, 2008, 81(11): 1712-1725.

[5] Levanon N. Lowest GDOP in 2D range localization[J]. IET Radar, Sonar & Navigation, 2008, 2(6): 437-446.

[6] Sharp I, Yu K, Guo Y J. GDOP analysis for positioning system design[C]//2011 International Conference on Localization and GNSS. Tampere: IEEE, 2011: 56-61.

[7] 袁修孝, 付迎春. GPS/INS 组合导航系统中 GPS 定位精度的 GDOP 分析 [J]. 武汉大学学报 (信息科学版), 2005, 30(11): 991-994.

[8] Dogançay K, Hmam H. Optimal angular sensor separation for AOA localization[J]. Signal Processing, 2008, 88(4): 912-923.

[9] 庄致义, 刘志国, 车仁泉. 基于几何约束的纯方位目标定位算法 [J]. 电子学报, 2017, 45(8): 1985-1991.

[10] Nardone S C, Lindgren A G, Gong K F. Fundamental properties and performance of conventional bearings-only target motion analysis[J]. IEEE Transactions on Automatic Control, 1984, 29(9): 775-787.

[11] Gillis J T, Sreenivas R S. Bearing-only tracking of a target with unknown dynamics[J]. IEEE Transactions on Aerospace and Electronic Systems, 2008, 44(1): 312-328.

[12] 高为广, 邱致和. 无人机编队飞行中的相对导航技术综述 [J]. 导航定位与授时, 2018, 5(4): 10-18.

A 支撑材料文件列表

表 11: 支撑材料文件列表

文件路径	用途
project/common.py	公共算法模块（坐标转换、夹角、GN、SP、GDOP、危险
project/问题 1_1_ 定位模型.py	问题 1(1) 求解与可视化
project/问题 1_2_ 可观测性分析.py	问题 1(2) 枚举与可观测性分析
project/问题 1_3_ 位置调整方案.py	问题 1(3) 迭代调整与蒙特卡洛验证
project/问题 2_ 锥形编队调整.py	问题 2 锥形编队调整与策略转换
project/results/*.json	各子问题数值结果（与 solution.json 一致）
result_bundle/solution.json	4 子问题完整解（含 active_constraints/sensitivity/critical_
result_bundle/modeling_plan.json	建模方案（含 design_critique/cost_decomposition/alternat
result_bundle/figures/*.pdf|png	14 张图（PDF+PNG 双格式）

B 源程序代码

下列代码与 project/ 目录下实际代码逐行一致（通过 \lstinputlisting 引用），可独立运行复现全部数值结果。

B.1 公共算法模块 common.py

Listing 1: common.py 公共算法模块

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
2022B 无人机纯方位无源定位 - 公共算法模块
包含:坐标转换/夹角计算/后方交会(Gauss-Newton)/Snellius-Pothenot解析/GDOP/危险圆/可视化样式

符号约定(与术语表.json/建模方案modeling_plan.json三方一致):
  T_i : 发射机位置 (x_i, y_i) [m]
  P_r : 接收机位置 (x, y) [m]   (代码中用 P)
   _ij: 接收机P与两发射机T_i,T_j连线夹角 [rad], _ij=(T_i-P, T_j-P) [0, ]
  S_t : 第t次发射机集合
  P_i^(t): 无人机i第t次调整后位置
  P_i^ideal: 无人机i理想位置
  GDOP = √(tr((H^T·H)^-1))  几何精度因子
  H   : 观测雅可比   / P  [rad/m]
"""
import sys, os
import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl
import matplotlib.font_manager as fm
```

```
22
23  # ---------- 中文字体设置 (来自 visualization_style.py) ----------
24  _CJK_FONT_CANDIDATES = ['Microsoft YaHei', 'SimHei', 'SimSun', 'KaiTi', 'FangSong',
25                          'Noto Sans CJK SC', 'Arial Unicode MS']
26  _available_fonts = {f.name for f in fm.fontManager.ttflist}
27  _cjk_font = None
28  for _font in _CJK_FONT_CANDIDATES:
29      if _font in _available_fonts:
30          _cjk_font = _font
31          break
32  if _cjk_font:
33      plt.rcParams['font.sans-serif'] = [_cjk_font, 'DejaVu Sans']
34  else:
35      plt.rcParams['font.sans-serif'] = ['DejaVu Sans']
36  plt.rcParams['axes.unicode_minus'] = False
37  plt.rcParams['savefig.dpi'] = 300
38  plt.rcParams['figure.dpi'] = 100
39  plt.rcParams['savefig.bbox'] = 'tight'
40  plt.rcParams['font.size'] = 10
41  plt.rcParams['axes.titlesize'] = 12
42  plt.rcParams['axes.labelsize'] = 10
43  plt.rcParams['lines.linewidth'] = 1.5
44
45  # 项目路径
46  PROJECT_ROOT = os.path.dirname(os.path.abspath(__file__))
47  FIG_DIR = os.path.join(PROJECT_ROOT, 'figures')
48  RES_DIR = os.path.join(PROJECT_ROOT, 'results')
49  os.makedirs(FIG_DIR, exist_ok=True)
50  os.makedirs(RES_DIR, exist_ok=True)
51
52  # ============================================================
53  # PARAMETERS 区块 - 题目参数 (与表1/题目给定一致)
54  # ============================================================
55  # 表1 初始位置 (极坐标: r_m, theta_deg) - source: 题目表1
56  TABLE1_POLAR = {
57      0: (0.0, 0.0),
58      1: (100.0, 0.0),
59      2: (98.0, 40.10),
60      3: (112.0, 80.21),
61      4: (105.0, 119.75),
62      5: (98.0, 159.86),
63      6: (112.0, 199.96),
64      7: (105.0, 240.07),
65      8: (98.0, 280.17),
66      9: (112.0, 320.28),
67  }
68  # 理想位置: FY00圆心(0,0); FY01-09 半径100m圆周均匀分布, 角度0,40,...,320° - source: 题目"均匀
        分布半径100m"
69  R_IDEAL = 100.0  # m, 圆形编队理想半径 - source: 题目问题1(3)
70  N_CIRCLE = 9     # 圆周无人机数 - source: 题目问题1
71  D_CONE = 50.0    # m, 锥形编队间距 - source: 题目问题2"如50m"
72  N_CONE = 10      # 锥形编队无人机数(假设) - source: 假设A7(参考圆形10架)
73  DELTA_TOL = 1e-6 # 收敛容差 [m] - source: 建模定义
74  EPS_JAC = 1e-7   # 数值雅可比有限差分步长 [m]
75
76  # ============================================================
```

```
# CORE ALGORITHM 区块 - 核心算法 (不修改模板, 本题为自定义几何定位算法)
# ============================================================

def polar_to_cartesian(r, theta_deg):
    """极坐标(r, °) → 直角坐标(x,y). source: 标准坐标转换"""
    th = np.radians(theta_deg)
    return np.array([r * np.cos(th), r * np.sin(th)])

def get_table1_cartesian():
    """返回表1初始位置的直角坐标字典 {0:array, 1:array, ...}"""
    return {i: polar_to_cartesian(r, th) for i, (r, th) in TABLE1_POLAR.items()}

def get_ideal_positions_circle():
    """圆形编队理想位置: FY00(0,0), FY01-09(100, 40°*(i-1))"""
    pos = {0: np.array([0.0, 0.0])}
    for i in range(1, 10):
        pos[i] = polar_to_cartesian(R_IDEAL, (i - 1) * 40.0)
    return pos

def get_ideal_positions_cone():
    """锥形编队理想位置: 沿x轴等间距50m, 10架 (假设A7)
    FY00在(0,0)领航, FY01-09沿x轴正方向 50,100,...,450m
    注:锥形编队编号沿用FY00-09,直线分布"""
    pos = {}
    for i in range(10):
        pos[i] = np.array([i * D_CONE, 0.0])
    return pos

def angle_between(P, T_i, T_j):
    """计算夹角  _ij =  (T_i-P, T_j-P)   [0, ] [rad]
    source: 观测方程C2, arccos((v_i·v_j)/(|v_i||v_j|))
    """
    v_i = T_i - P
    v_j = T_j - P
    d_i = np.linalg.norm(v_i)
    d_j = np.linalg.norm(v_j)
    assert d_i > 1e-12, f"接收机P与发射机T_i重合(夹角无定义), P={P}, T_i={T_i}"
    assert d_j > 1e-12, f"接收机P与发射机T_j重合(夹角无定义), P={P}, T_j={T_j}"
    cos_a = np.dot(v_i, v_j) / (d_i * d_j)
    cos_a = np.clip(cos_a, -1.0, 1.0)  # 数值稳定性
    return np.arccos(cos_a)

def compute_all_angles(P, emitters):
    """计算接收机P对所有发射机对的夹角
    emitters: dict {idx: position} 或 list of positions
    返回: list of (i,j,angle) 三元组, i<j
    """
    if isinstance(emitters, dict):
        items = sorted(emitters.items())
        keys = [k for k, _ in items]
        poss = [p for _, p in items]
    else:
        keys = list(range(len(emitters)))
        poss = list(emitters)
    result = []
    n = len(poss)
```

```
    for a in range(n):
        for b in range(a + 1, n):
            ang = angle_between(P, poss[a], poss[b])
            result.append((keys[a], keys[b], ang))
    return result

def angle_jac(P, T_i, T_j, eps=EPS_JAC):
    """数值雅可比:  _ij/ P [2维向量, rad/m]
    中心差分: J[l] = ( (P+ •e_l) -  (P- •e_l)) / (2 )
    """
    J = np.zeros(2)
    for l in range(2):
        d = np.zeros(2)
        d[l] = eps
        a_plus = angle_between(P + d, T_i, T_j)
        a_minus = angle_between(P - d, T_i, T_j)
        J[l] = (a_plus - a_minus) / (2 * eps)
    return J

def resection_gauss_newton(emitters, angles_obs, P_init,
                           n_iter=100, tol=1e-12, use_independent=True):
    """后方交会定位 - Gauss-Newton迭代最小二乘 (主算法, 卡片03 NLP)

    emitters: dict {idx: pos} 发射机位置(已知)
    angles_obs: list of (i,j, _obs) 观测夹角 [rad]
    P_init: 初始位置估计 [x,y]
    use_independent: True=仅用独立夹角(每对发射机1个,共C(n,2)个);
                     注:3发射机时3夹角仅2独立,但最小二乘可处理冗余(协方差奇异需正则化)
                     本实现用全部夹角+Levenberg-Marquardt正则化避免奇异
    返回: P_est, info(dict: iterations, residual, converged, H, GDOP)
    """
    P = np.array(P_init, dtype=float)
    items = sorted(emitters.items())
    keys = [k for k, _ in items]
    poss = [p for _, p in items]
    idx_map = {k: a for a, k in enumerate(keys)}
    # 观测对
    obs_pairs = [(idx_map[i], idx_map[j], a) for (i, j, a) in angles_obs]
    n_obs = len(obs_pairs)
    converged = False
    prev_res = None
    for it in range(n_iter):
        # 残差 r [n_obs]
        r = np.zeros(n_obs)
        H = np.zeros((n_obs, 2))
        for k, (a, b, a_obs) in enumerate(obs_pairs):
            Ti, Tj = poss[a], poss[b]
            r[k] = angle_between(P, Ti, Tj) - a_obs
            H[k, :] = angle_jac(P, Ti, Tj)
        # Levenberg-Marquardt: (H^T H +  I) ΔP = -H^T r
        HtH = H.T @ H
        Htr = H.T @ r
        # 初始 小,奇异时增大
        lam = 1e-8 * (np.trace(HtH) / max(1, n_obs) + 1e-12)
        try:
            dP = -np.linalg.solve(HtH + lam * np.eye(2), Htr)
```

```
        except np.linalg.LinAlgError:
            dP = -np.linalg.lstsq(HtH + 1e-6 * np.eye(2), Htr, rcond=None)[0]
        P = P + dP
        res_norm = np.linalg.norm(r)
        if prev_res is not None and abs(prev_res - res_norm) < tol and np.linalg.norm(dP) <
            tol:
            converged = True
            prev_res = res_norm
            break
        prev_res = res_norm
    # 最终H与GDOP
    H_final = np.zeros((n_obs, 2))
    r_final = np.zeros(n_obs)
    for k, (a, b, a_obs) in enumerate(obs_pairs):
        Ti, Tj = poss[a], poss[b]
        r_final[k] = angle_between(P, Ti, Tj) - a_obs
        H_final[k, :] = angle_jac(P, Ti, Tj)
    # GDOP = √(tr((H^T H)^-1)), 需H满秩
    HtH = H_final.T @ H_final
    try:
        cov = np.linalg.inv(HtH)
        gdop = np.sqrt(np.trace(cov))
        rank = np.linalg.matrix_rank(H_final)
    except np.linalg.LinAlgError:
        cov = np.linalg.pinv(HtH)
        gdop = np.sqrt(np.trace(cov))
        rank = np.linalg.matrix_rank(H_final)
    info = {
        'iterations': it + 1,
        'residual_norm': float(np.linalg.norm(r_final)),
        'residual_max': float(np.max(np.abs(r_final))) if n_obs > 0 else 0.0,
        'converged': converged,
        'H': H_final,
        'GDOP': float(gdop),
        'rank': int(rank),
        'final_P': P.copy(),
    }
    return P, info

def circumcircle(A, B, C):
    """三点外接圆: 返回 (center, radius). 共线返回None. source: 几何"""
    ax, ay = A; bx, by = B; cx, cy = C
    d = 2 * (ax * (by - cy) + bx * (cy - ay) + cx * (ay - by))
    if abs(d) < 1e-12:
        return None  # 共线
    ux = ((ax**2 + ay**2) * (by - cy) + (bx**2 + by**2) * (cy - ay) + (cx**2 + cy**2) * (ay -
        by)) / d
    uy = ((ax**2 + ay**2) * (cx - bx) + (bx**2 + by**2) * (ax - cx) + (cx**2 + cy**2) * (bx -
        ax)) / d
    center = np.array([ux, uy])
    radius = np.linalg.norm(A - center)
    return center, radius

def is_on_danger_circle(P, T_list, tol=1e-3):
    """判断P是否在发射机外接圆(危险圆)上
    T_list: 3个发射机位置
```

```
    返回: (on_circle:bool, distance_to_circle:float)
    """
    if len(T_list) < 3:
        return False, np.inf
    cc = circumcircle(T_list[0], T_list[1], T_list[2])
    if cc is None:
        return False, np.inf  # 发射机共线,无外接圆(另一种退化)
    center, radius = cc
    dist = np.linalg.norm(P - center)
    return abs(dist - radius) < tol, abs(dist - radius)

def gdop_at(P, emitters, angle_pairs=None):
    """计算P处给定发射机配置的GDOP
    angle_pairs: 指定用哪些夹角对; None=所有对
    """
    items = sorted(emitters.items()) if isinstance(emitters, dict) else list(enumerate(
        emitters))
    if isinstance(emitters, dict):
        keys = [k for k, _ in items]
        poss = [p for _, p in items]
    else:
        keys = [k for k, _ in items]
        poss = [p for _, p in items]
    idx_map = {k: a for a, k in enumerate(keys)}
    if angle_pairs is None:
        pairs = [(a, b) for a in range(len(poss)) for b in range(a + 1, len(poss))]
    else:
        pairs = [(idx_map[i], idx_map[j]) for (i, j) in angle_pairs]
    n = len(pairs)
    H = np.zeros((n, 2))
    for k, (a, b) in enumerate(pairs):
        H[k, :] = angle_jac(P, poss[a], poss[b])
    HtH = H.T @ H
    try:
        cov = np.linalg.inv(HtH)
        return np.sqrt(np.trace(cov))
    except np.linalg.LinAlgError:
        return np.inf

# ---------- 可视化辅助 ----------
COLORS = {
    "primary": "#1f77b4", "secondary": "#ff7f0e", "tertiary": "#2ca02c",
    "quaternary": "#d62728", "emitter": "#d62728", "receiver": "#1f77b4",
    "ideal": "#2ca02c", "actual": "#ff7f0e", "trajectory": "#9467bd",
    "palette": ["#1f77b4", "#ff7f0e", "#2ca02c", "#d62728",
                "#9467bd", "#8c564b", "#e377c2", "#7f7f7f"]
}

def save_fig(fig, name, subdir=''):
    """保存PDF+PNG双格式到 figures/(subdir/)name.{pdf,png}"""
    d = FIG_DIR if not subdir else os.path.join(FIG_DIR, subdir)
    os.makedirs(d, exist_ok=True)
    base = os.path.join(d, name)
    for fmt in ('pdf', 'png'):
        path = f"{base}.{fmt}"
        fig.savefig(path, dpi=300, bbox_inches='tight')
```

```
    print(f"  [saved] {base}.pdf / .png")

def print_assert(cond, msg):
    """合理性断言(不抛异常,打印警告)"""
    if not cond:
        print(f"  [ASSERT FAIL] {msg}")
    return cond

if __name__ == '__main__':
    # 自检: 理想位置与表1偏差
    ideal = get_ideal_positions_circle()
    tab1 = get_table1_cartesian()
    print("=== 自检: 理想位置 vs 表1 ===")
    for i in range(10):
        dev = np.linalg.norm(tab1[i] - ideal[i])
        print(f"  FY{i:02d}: ideal=({ideal[i][0]:7.2f},{ideal[i][1]:7.2f}) "
              f"actual=({tab1[i][0]:7.2f},{tab1[i][1]:7.2f}) dev={dev:.3f}m")
    # 自检: 夹角计算
    P = np.array([50.0, 50.0])
    T0 = np.array([0.0, 0.0]); T1 = np.array([100.0, 0.0])
    a = angle_between(P, T0, T1)
    print(f"\n=== 自检: 夹角 ===\n  P=(50,50), T0=(0,0), T1=(100,0),  _01={np.degrees(a):.4f}°
        (期望90°)")
    print_assert(abs(a - np.pi/2) < 1e-6, "夹角计算: P=(50,50)对T0,T1应为90°")
    # 自检: 后方交会(已知P,反推)
    print(f"\n=== 自检: 后方交会(Gauss-Newton) ===")
    P_true = np.array([60.0, 80.0])
    emitters = {0: ideal[0], 1: ideal[1], 5: ideal[5]}  # FY00,FY01,FY05
    angles = compute_all_angles(P_true, emitters)
    P_init = np.array([50.0, 50.0])  # 故意偏离
    P_est, info = resection_gauss_newton(emitters, angles, P_init)
    err = np.linalg.norm(P_est - P_true)
    print(f"  P_true={P_true}, P_est=({P_est[0]:.6f},{P_est[1]:.6f}), err={err:.2e}m")
    print(f"  info: iter={info['iterations']}, residual={info['residual_norm']:.2e}, "
          f"GDOP={info['GDOP']:.4f}, rank={info['rank']}, converged={info['converged']}")
    print_assert(err < 1e-3, f"后方交会误差应<1e-3m, 实际{err:.2e}")
    print("\n[common.py 自检完成]")
```

B.2 问题 1(1) 求解代码

Listing 2: 问题 1_1_ 定位模型.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
问题1(1): 圆形编队 FY00+2架圆周无人机发射信号(共3架,位置无偏差已知),
          建立被动接收信号无人机的定位模型.

方法: Gauss-Newton迭代最小二乘(主) + Snellius-Pothenot等视角圆弧交点(验证)
分析: 危险圆奇异性(C(9,2)=36种发射机组合) + GDOP几何精度因子

输出: project/results/q1_1_result.json, project/figures/q1_1_*.pdf|png
"""
import sys, os, json
import numpy as np
```

```
import matplotlib.pyplot as plt
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from common import (polar_to_cartesian, get_ideal_positions_circle, get_table1_cartesian,
                    angle_between, compute_all_angles, angle_jac, resection_gauss_newton,
                    circumcircle, is_on_danger_circle, gdop_at, save_fig, COLORS,
                    R_IDEAL, DELTA_TOL)

# ============================================================
# PARAMETERS 区块
# ============================================================
# 典型算例: 发射机=FY00+FY01+FY05(圆心+2架圆周,跨象限分布利于几何),
# 接收机=FY03(位置略有偏差,用表1实际位置作为"真值"演示定位)
EMITTERS_DEMO = [0, 1, 5]   # FY00, FY01, FY05
RECEIVER_DEMO = 3           # FY03

# ============================================================
# CORE ALGORITHM: Snellius-Pothenot 等视角圆弧交点解析解 (验证用)
# ============================================================
def equal_angle_circle(A, B, alpha):
    """过A,B且使圆周角 APB=alpha的圆弧(两个可能,返回两个圆心与半径)
    半径 R = |AB|/(2 sin  ); 圆心在AB垂直平分线上,距中点 h=|AB|/(2 tan  )
    返回: [(O1, R1), (O2, R1)]  两个候选圆
    """
    AB = B - A
    L = np.linalg.norm(AB)
    if L < 1e-12 or abs(np.sin(alpha)) < 1e-9:
        return None
    R = L / (2 * np.sin(alpha))
    M = (A + B) / 2
    n = np.array([-AB[1], AB[0]]) / L  # 垂直方向
    h = L / (2 * np.tan(alpha))
    return [(M + h * n, R), (M - h * n, R)]

def two_circle_intersections(O1, R1, O2, R2):
    """两圆交点. 返回 list of P (0/1/2个)"""
    d = np.linalg.norm(O2 - O1)
    if d < 1e-12:
        return []  # 同心
    if d > R1 + R2 + 1e-9 or d < abs(R1 - R2) - 1e-9:
        return []  # 不相交
    a = (R1**2 - R2**2 + d**2) / (2 * d)
    a = np.clip(a, -R1, R1)
    h_sq = R1**2 - a**2
    h = np.sqrt(max(0, h_sq))
    M = O1 + a * (O2 - O1) / d
    n_perp = np.array([-(O2 - O1)[1], (O2 - O1)[0]]) / d
    return [M + h * n_perp, M - h * n_perp]

def snellius_pothenot_solve(A, B, C, alpha, beta):
    """Snellius-Pothenot解析解: 已知A,B,C +  = APB,  = BPC, 求P
    返回: P (numpy array) 或 None(危险圆/无解)
    """
    # 圆1: 过A,B, 圆周角
    c1 = equal_angle_circle(A, B, alpha)
    # 圆2: 过B,C, 圆周角
    c2 = equal_angle_circle(B, C, beta)
```

```
    if c1 is None or c2 is None:
        return None
    # 枚举4种圆心组合，找使 APB= 且 BPC= 的P
    # 注：两圆交点可能含A/B/C本身(因圆过这些点)，需过滤
    known_pts = [A, B, C]
    candidates = []
    for O1, R1 in c1:
        for O2, R2 in c2:
            pts = two_circle_intersections(O1, R1, O2, R2)
            for P in pts:
                # 过滤与A/B/C重合的点(夹角无定义)
                if any(np.linalg.norm(P - K) < 1e-6 for K in known_pts):
                    continue
                candidates.append(P)
    best = None; best_err = np.inf
    for P in candidates:
        try:
            a_check = angle_between(P, A, B)
            b_check = angle_between(P, B, C)
        except AssertionError:
            continue
        err = abs(a_check - alpha) + abs(b_check - beta)
        if err < best_err:
            best_err = err; best = P
    if best is None or best_err > 1e-3:
        return None
    return best

# ============================================================
# 主求解流程
# ============================================================
def solve_q1_1():
    print("=" * 70)
    print("问题1(1)：圆形编队三点后方交会定位模型")
    print("=" * 70)
    ideal = get_ideal_positions_circle()
    tab1 = get_table1_cartesian()

    # --- 算例1：发射机=FY00+FY01+FY05，接收机=FY03 ---
    print(f"\n[算例1] 发射机={EMITTERS_DEMO}，接收机=FY{RECEIVER_DEMO:02d}")
    emitters = {i: ideal[i] for i in EMITTERS_DEMO}
    P_true = tab1[RECEIVER_DEMO]  # 接收机真实位置(表1有偏差位置)
    # 观测夹角(无噪声，由P_true计算)
    angles_obs = compute_all_angles(P_true, emitters)
    print(f"  发射机位置(理想无偏差):")
    for i, p in emitters.items():
        print(f"    FY{i:02d} = ({p[0]:7.3f}, {p[1]:7.3f})")
    print(f"  接收机真实位置 P_true = ({P_true[0]:7.3f}, {P_true[1]:7.3f})")
    print(f"  观测夹角(共{len(angles_obs)}个，3发射机→3夹角，2独立):")
    for (i, j, a) in angles_obs:
        print(f"     (FY{i:02d},FY{j:02d}) = {np.degrees(a):8.4f}°")

    # Gauss-Newton求解(初值故意偏离)
    P_init = np.array([20.0, 60.0])
    P_est, info = resection_gauss_newton(emitters, angles_obs, P_init)
    err_gn = np.linalg.norm(P_est - P_true)
```

```
    print(f"\n  [Gauss-Newton] 初值=({P_init[0]},{P_init[1]})")
    print(f"    P_est = ({P_est[0]:.6f}, {P_est[1]:.6f})")
    print(f"    误差 = {err_gn:.3e} m")
    print(f"    迭代次数 = {info['iterations']}, 收敛 = {info['converged']}")
    print(f"    残差范数 = {info['residual_norm']:.3e} rad")
    print(f"    GDOP = {info['GDOP']:.4f}, 雅可比秩 = {info['rank']}")

    # Snellius-Pothenot解析解(用前2个独立夹角)
    A, B, C = ideal[EMITTERS_DEMO[0]], ideal[EMITTERS_DEMO[1]], ideal[EMITTERS_DEMO[2]]
    alpha = angles_obs[0][2]  # (FY00,FY01)
    beta = angles_obs[1][2]   # (FY00,FY05) -- 注意排序
    # 找 (A,B) 与 (B,C) 对应的夹角
    pair01 = next(a for (i, j, a) in angles_obs if (i, j) == (EMITTERS_DEMO[0], EMITTERS_DEMO
        [1]))
    pair12 = next(a for (i, j, a) in angles_obs if (i, j) == (EMITTERS_DEMO[1], EMITTERS_DEMO
        [2]))
    P_sp = snellius_pothenot_solve(A, B, C, pair01, pair12)
    if P_sp is not None:
        err_sp = np.linalg.norm(P_sp - P_true)
        print(f"\n  [Snellius-Pothenot解析解]")
        print(f"    P_sp = ({P_sp[0]:.6f}, {P_sp[1]:.6f})")
        print(f"    误差 = {err_sp:.3e} m")
        print(f"    两种方法偏差 = {np.linalg.norm(P_est - P_sp):.3e} m (应 0,互验)")
    else:
        err_sp = None
        print(f"\n  [Snellius-Pothenot] 解析法无解(可能危险圆)")

    # --- 危险圆全分析: 所有C(9,2)=36种2圆周发射机组合 ---
    print(f"\n[危险圆分析] 枚举所有C(9,2)=36种2圆周发射机组合+FY00")
    circle_ids = list(range(1, 10))
    danger_results = []
    from itertools import combinations
    for combo in combinations(circle_ids, 2):
        emitters_set = [0] + list(combo)  # FY00 + 2圆周
        T_list = [ideal[i] for i in emitters_set]
        # 对每个其他圆周接收机检查是否在危险圆上
        for rcv in circle_ids:
            if rcv in combo:
                continue
            P_rcv = ideal[rcv]
            on_dc, dist = is_on_danger_circle(P_rcv, T_list)
            if on_dc:
                danger_results.append({'emitters': emitters_set, 'receiver': rcv,
                                       'dist_to_circle': dist})
    print(f"  共检查 36*7=252 个(发射机组合,接收机)对")
    print(f"  危险圆失效组合数: {len(danger_results)}")
    if danger_results:
        for d in danger_results[:5]:
            print(f"    发射机={d['emitters']}, 接收机=FY{d['receiver']:02d}, "
                  f"距危险圆={d['dist_to_circle']:.4f}m")
    else:
        print(f"    (无严格危险圆,因FY00在圆心,3点外接圆一般不经过其他圆周点)")

    # --- GDOP分析: 各接收机位置的GDOP ---
    print(f"\n[GDOP分析] 各圆周接收机在FY00+FY01+FY05发射下的GDOP")
    gdop_table = []
```

```
    for rcv in range(1, 10):
        P_rcv = ideal[rcv]
        g = gdop_at(P_rcv, emitters)
        gdop_table.append({'receiver': rcv, 'GDOP': float(g),
                           'on_danger': bool(is_on_danger_circle(P_rcv, [ideal[i] for i in
                               EMITTERS_DEMO])[0])})
        print(f"    FY{rcv:02d}: GDOP={g:8.4f}")

    # --- 可视化 ---
    print(f"\n[可视化] 生成图表...")
    plot_localization_demo(emitters, P_true, P_est, P_init, P_sp, RECEIVER_DEMO)
    plot_danger_circle_gdop(ideal, emitters, gdop_table)

    # --- 结果汇总 ---
    result = {
        'problem': '1(1)',
        'method': 'Gauss-Newton迭代最小二乘(主) + Snellius-Pothenot解析(验证)',
        'example': {
            'emitters': EMITTERS_DEMO,
            'receiver': RECEIVER_DEMO,
            'P_true': P_true.tolist(),
            'P_est_gauss_newton': P_est.tolist(),
            'P_est_snellius_pothenot': P_sp.tolist() if P_sp is not None else None,
            'error_gauss_newton_m': float(err_gn),
            'error_snellius_pothenot_m': float(err_sp) if err_sp is not None else None,
            'cross_method_diff_m': float(np.linalg.norm(P_est - P_sp)) if P_sp is not None
                else None,
            'iterations': info['iterations'],
            'residual_norm_rad': info['residual_norm'],
            'GDOP': info['GDOP'],
            'jacobian_rank': info['rank'],
            'converged': info['converged'],
        },
        'danger_circle_analysis': {
            'total_combinations_checked': 252,
            'dangerous_combinations': len(danger_results),
            'details': danger_results,
            'note': 'FY00在圆心,3发射机外接圆一般不经过其他圆周点,危险圆罕见'
        },
        'gdop_table': gdop_table,
        'observability': {
            'condition': '雅可比J秩=2且接收机不在危险圆',
            'n_independent_angles': '3发射机→3夹角,独立2个(_01+_02+_12约束)',
            'redundancy_handling': 'S2红队问题1修复:最小二乘+LM正则化处理冗余,GDOP用2×2协方差'
                ,
        },
        'active_constraints': {
            'C2_观测方程': 'active(模型夹角=观测夹角,残差 0)',
            'C3_接收机 发射机': 'active(接收机FY03 发射机)',
            'C4_危险圆规避': 'slack(接收机不在危险圆,GDOP有限)',
            'C6_二维平面': 'active(2D坐标)'
        }
    }
    out_path = os.path.join('results', 'q1_1_result.json')
    with open(out_path, 'w', encoding='utf-8') as f:
        json.dump(result, f, ensure_ascii=False, indent=2)
```

```
    print(f"\n[结果保存] {out_path}")
    return result

def plot_localization_demo(emitters, P_true, P_est, P_init, P_sp, rcv_id):
    """图1: 定位演示 - 发射机/接收机/估计位置/初值"""
    fig, ax = plt.subplots(figsize=(7, 7))
    # 编队圆周
    theta = np.linspace(0, 2 * np.pi, 200)
    ax.plot(R_IDEAL * np.cos(theta), R_IDEAL * np.sin(theta), 'k--', alpha=0.3, label='理想圆周(r=100m)')
    # 发射机
    for i, p in emitters.items():
        ax.plot(*p, 'r^', markersize=14, markeredgecolor='k', zorder=5)
        ax.annotate(f'FY{i:02d}(发射)', p, textcoords="offset points",
                    xytext=(8, 8), fontsize=9, color='red')
    # 接收机真值
    ax.plot(*P_true, 'bs', markersize=12, markeredgecolor='k', zorder=5)
    ax.annotate(f'FY{rcv_id:02d}(接收,真值)', P_true, textcoords="offset points",
                xytext=(8, -15), fontsize=9, color='blue')
    # 估计位置
    ax.plot(*P_est, 'g*', markersize=16, markeredgecolor='k', zorder=6)
    ax.annotate(f'GN估计', P_est, textcoords="offset points",
                xytext=(8, 8), fontsize=9, color='green')
    # 初值
    ax.plot(*P_init, 'mx', markersize=10, alpha=0.7)
    ax.annotate(f'初值', P_init, textcoords="offset points",
                xytext=(8, 8), fontsize=9, color='purple')
    # 解析解
    if P_sp is not None:
        ax.plot(*P_sp, 'c+', markersize=12, markeredgewidth=2, zorder=6)
        ax.annotate(f'SP解析', P_sp, textcoords="offset points",
                    xytext=(-30, -15), fontsize=9, color='c')
    # 发射机-接收机连线
    for i, p in emitters.items():
        ax.plot([P_true[0], p[0]], [P_true[1], p[1]], 'r-', alpha=0.25, linewidth=0.8)
    # 估计误差箭头
    ax.annotate('', xy=P_est, xytext=P_true,
                arrowprops=dict(arrowstyle='->', color='green', lw=1.5))
    ax.set_xlim(-130, 130); ax.set_ylim(-130, 130)
    ax.set_aspect('equal')
    ax.set_xlabel('x [m]'); ax.set_ylabel('y [m]')
    ax.set_title(f'问题1(1) 三点后方交会定位演示\n发射机={list(emitters.keys())}, 接收机=FY{rcv_id:02d}, '
                 f'误差={np.linalg.norm(P_est-P_true):.2e}m')
    ax.grid(True, alpha=0.3)
    ax.legend(loc='upper left', fontsize=8)
    save_fig(fig, 'q1_1_localization_demo')
    plt.close(fig)

def plot_danger_circle_gdop(ideal, emitters, gdop_table):
    """图2: GDOP分布+危险圆分析"""
    fig, axes = plt.subplots(1, 2, figsize=(13, 5.5))
    # 左: GDOP空间分布热力图
    ax = axes[0]
    xs = np.linspace(-150, 150, 200)
    ys = np.linspace(-150, 150, 200)
```

```
    X, Y = np.meshgrid(xs, ys)
    Z = np.full_like(X, np.nan)
    poss = list(emitters.values())
    for i in range(X.shape[0]):
        for j in range(X.shape[1]):
            P = np.array([X[i, j], Y[i, j]])
            # 跳过发射机位置附近
            if any(np.linalg.norm(P - T) < 1 for T in poss):
                continue
            try:
                Z[i, j] = gdop_at(P, emitters)
            except Exception:
                Z[i, j] = np.nan
    # 裁剪极值便于显示
    Z_disp = np.clip(Z, 0, 500)
    pcm = ax.pcolormesh(X, Y, Z_disp, shading='auto', cmap='viridis_r', vmin=0, vmax=200)
    fig.colorbar(pcm, ax=ax, label='GDOP（越小越好）')
    theta = np.linspace(0, 2 * np.pi, 200)
    ax.plot(R_IDEAL * np.cos(theta), R_IDEAL * np.sin(theta), 'w--', alpha=0.5, label='理想圆
        周')
    for i, p in emitters.items():
        ax.plot(*p, 'r^', markersize=12, markeredgecolor='w')
        ax.annotate(f'FY{i:02d}', p, color='white', fontsize=8, xytext=(5, 5),
                    textcoords='offset points')
    ax.set_xlim(-150, 150); ax.set_ylim(-150, 150)
    ax.set_aspect('equal')
    ax.set_xlabel('x [m]'); ax.set_ylabel('y [m]')
    ax.set_title(f'GDOP空间分布\n发射机={list(emitters.keys())}（红三角）')
    # 右：各接收机GDOP柱状图
    ax2 = axes[1]
    rcs = [g['receiver'] for g in gdop_table]
    gds = [g['GDOP'] for g in gdop_table]
    colors = [COLORS['quaternary'] if g > 100 else COLORS['primary'] for g in gds]
    ax2.bar([f'FY{r:02d}' for r in rcs], gds, color=colors)
    ax2.set_xlabel('接收机编号')
    ax2.set_ylabel('GDOP')
    ax2.set_title('各圆周接收机GDOP\n(红:GDOP>100,几何不利)')
    ax2.grid(True, alpha=0.3, axis='y')
    for i, v in enumerate(gds):
        ax2.text(i, v + 1, f'{v:.1f}', ha='center', fontsize=8)
    fig.suptitle('问题1(1) 危险圆与GDOP几何精度分析', fontsize=13)
    fig.tight_layout()
    save_fig(fig, 'q1_1_danger_circle_gdop')
    plt.close(fig)

if __name__ == '__main__':
    solve_q1_1()
```

B.3 问题 1(2) 求解代码

Listing 3: 问题 1_2_ 可观测性分析.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
```

```
问题1(2): 圆形编队中FY00/FY01位置已知(无偏差), FY02-FY09有偏差.
        问: 除FY00/FY01外还需几架无人机作为发射机才能对接收机有效定位?

建模(Strategy D, 假设A3): 发射机"当前位置"作为已知值传给接收机(虽有偏差但已知).
可观测性判据: 雅可比矩阵 H =   / P 的秩 rank(H)=2 (P为2维).

枚举设计:
  - k = 0..8 (从FY02-FY09中选k架作为发射机)
  - 对每个k, 枚举 C(8,k) 种组合, 共 2^8=256 种
  - 接收机候选位置: FY02-FY09 理想位置(8个) - 排除被选为发射机的
  - 判据: rank(H)=2 且 GDOP有限 且 不在危险圆

输出: project/results/q1_2_result.json, project/figures/q1_2_*.pdf|png
"""
import sys, os, json, itertools
import numpy as np
import matplotlib.pyplot as plt
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from common import (get_ideal_positions_circle, get_table1_cartesian,
                    angle_between, compute_all_angles, angle_jac,
                    resection_gauss_newton, gdop_at, is_on_danger_circle,
                    circumcircle, save_fig, COLORS, R_IDEAL)

# ============================================================
# PARAMETERS 区块
# ============================================================
KNOWN_EMITTERS = [0, 1]                # FY00, FY01 位置已知(无偏差)
CANDIDATE_EMITTERS = list(range(2, 10)) # FY02-FY09 候选发射机(8架)
K_MIN, K_MAX = 0, 8                     # 枚举 k 范围
RECEIVER_TEST_POSITIONS = list(range(2, 10))  # 接收机候选(FY02-FY09理想位置)
RANK_THRESHOLD = 2                      # 可观测秩阈值(P为2维)
GDOP_UPPER = 500.0                      # GDOP有效上界(超过视为几何不利)
DANGER_CIRCLE_TOL = 0.5                 # 危险圆距离容差 [m]

# ============================================================
# CORE ALGORITHM: 可观测性分析
# ============================================================
def observability_analysis(emitters_dict, P_rcv):
    """给定发射机配置和接收机位置, 分析可观测性
    返回: dict(rank, gdop, on_danger_circle, dist_to_danger, n_emitters, n_angles, observable)
    """
    # 接收机不能与任何发射机重合
    for k, p in emitters_dict.items():
        if np.linalg.norm(P_rcv - p) < 1e-6:
            return {'rank': 0, 'gdop': np.inf, 'observable': False,
                    'reason': 'rcv_emitter_overlap', 'n_emitters': len(emitters_dict)}

    # 计算所有夹角对的雅可比, 拼成 H 矩阵
    items = sorted(emitters_dict.items())
    keys = [k for k, _ in items]
    poss = [p for _, p in items]
    pairs = [(a, b) for a in range(len(poss)) for b in range(a + 1, len(poss))]
    n_obs = len(pairs)
    H = np.zeros((n_obs, 2))
    for k, (a, b) in enumerate(pairs):
        try:
```

```
            H[k, :] = angle_jac(P_rcv, poss[a], poss[b])
        except AssertionError:
            return {'rank': 0, 'gdop': np.inf, 'observable': False,
                    'reason': 'angle_undefined', 'n_emitters': len(emitters_dict)}

    rank = int(np.linalg.matrix_rank(H))
    HtH = H.T @ H
    try:
        cov = np.linalg.inv(HtH)
        # 数值稳定性: 协方差对角元素可能因数值误差为微小负值
        tr_cov = float(np.trace(cov))
        gdop = float(np.sqrt(max(0.0, tr_cov))) if tr_cov >= 0 else np.inf
    except np.linalg.LinAlgError:
        gdop = np.inf

    # 危险圆检查(仅当>=3发射机)
    on_dc = False
    dist_dc = np.inf
    if len(poss) >= 3:
        # 检查所有 C(n,3) 三发射机外接圆
        min_dist = np.inf
        for tri in itertools.combinations(range(len(poss)), 3):
            T3 = [poss[i] for i in tri]
            cc = circumcircle(T3[0], T3[1], T3[2])
            if cc is None:
                continue
            center, radius = cc
            d = abs(np.linalg.norm(P_rcv - center) - radius)
            if d < min_dist:
                min_dist = d
        dist_dc = float(min_dist)
        on_dc = dist_dc < DANGER_CIRCLE_TOL

    observable = (rank >= RANK_THRESHOLD) and (gdop < GDOP_UPPER) and (not on_dc)
    reason = 'ok' if observable else (
        'rank_deficient' if rank < RANK_THRESHOLD else
        ('danger_circle' if on_dc else 'gdop_too_large')
    )
    return {
        'rank': rank, 'gdop': gdop, 'on_danger_circle': on_dc,
        'dist_to_danger': dist_dc, 'n_emitters': len(emitters_dict),
        'n_angles': n_obs, 'observable': observable, 'reason': reason
    }


def enumerate_all_combinations():
    """枚举 k=0..8 × C(8,k) × 接收机候选
    返回: list of dict(k, combo, receiver, analysis)

    注: 采用 Strategy D(假设A3):
      - FY00, FY01 位置无偏差(用ideal)
      - FY02-FY09 候选发射机用表1实际位置(有偏差但当前位置已知)
      - 接收机用表1实际位置(有偏差, 模拟真实定位场景)
    这样可避免"理想圆周退化"(3圆周点外接圆=编队圆周导致全员危险圆)
    """
    ideal = get_ideal_positions_circle()
```

```
    tab1 = get_table1_cartesian()
    results = []
    for k in range(K_MIN, K_MAX + 1):
        for combo in itertools.combinations(CANDIDATE_EMITTERS, k):
            # 发射机: FY00/FY01用ideal(无偏差), 候选用tab1(有偏差但已知)
            emitters_dict = {0: ideal[0], 1: ideal[1]}
            for i in combo:
                emitters_dict[i] = tab1[i]
            for rcv_id in RECEIVER_TEST_POSITIONS:
                if rcv_id in combo:
                    continue  # 接收机不能同时是发射机
                P_rcv = tab1[rcv_id]  # 接收机用实际位置(有偏差)
                analysis = observability_analysis(emitters_dict, P_rcv)
                results.append({
                    'k': k,
                    'combo': list(combo),
                    'emitters': list(emitters_dict.keys()),
                    'receiver': rcv_id,
                    'analysis': analysis
                })
    return results


def solve_q1_2():
    print("=" * 70)
    print("问题1(2): 可观测性分析 - 除FY00/FY01外还需几架发射机?")
    print("=" * 70)
    print(f"枚举空间: k [0..8] × C(8,k)组合 × 接收机候选(FY02-FY09理想位置)")
    print(f"已知锚点: FY00(圆心), FY01(圆周0°) - 位置无偏差")

    all_results = enumerate_all_combinations()
    print(f"\n总算例数: {len(all_results)}")

    # 按 k 汇总
    summary_by_k = {}
    for k in range(K_MIN, K_MAX + 1):
        subset = [r for r in all_results if r['k'] == k]
        n_total = len(subset)
        if n_total == 0:
            # k=8 时所有FY02-FY09都被选为发射机, 无接收机候选
            summary_by_k[k] = {
                'n_combinations': 0, 'n_observable': 0, 'observable_ratio': 0.0,
                'n_rank2': 0, 'n_danger_circle': 0,
                'gdop_min': None, 'gdop_median': None, 'gdop_max': None,
                'k_min_for_observable': None,
                'note': 'k=8时所有候选均为发射机, 无接收机位置, 不适用'
            }
            print(f"\n[k={k}] 无算例(所有候选均为发射机)")
            continue
        n_obs = sum(1 for r in subset if r['analysis']['observable'])
        n_rank2 = sum(1 for r in subset if r['analysis']['rank'] >= 2)
        n_danger = sum(1 for r in subset if r['analysis']['on_danger_circle'])
        gdops = [r['analysis']['gdop'] for r in subset if r['analysis']['gdop'] < np.inf]
        gdop_min = min(gdops) if gdops else None
        gdop_med = float(np.median(gdops)) if gdops else None
        gdop_max = max(gdops) if gdops else None
```

```
        summary_by_k[k] = {
            'n_combinations': n_total,
            'n_observable': n_obs,
            'observable_ratio': n_obs / n_total,
            'n_rank2': n_rank2,
            'n_danger_circle': n_danger,
            'gdop_min': gdop_min, 'gdop_median': gdop_med, 'gdop_max': gdop_max,
            'k_min_for_observable': None  # 后续填充
        }
        print(f"\n[k={k}] 总算例={n_total}, 可观测={n_obs} ({n_obs/n_total*100:.1f}%), "
              f"rank 2={n_rank2}, 危险圆={n_danger}")
        if gdops:
            print(f"       GDOP: min={gdop_min:.3f}, med={gdop_med:.3f}, max={gdop_max:.3f}")

    # 确定 k_min
    k_min = None
    for k in range(K_MIN, K_MAX + 1):
        if summary_by_k[k]['n_observable'] > 0:
            # 进一步要求：可观测比例 > 50%（稳健性）
            if summary_by_k[k]['observable_ratio'] >= 0.5:
                k_min = k
                break
    # 填充 k_min_for_observable
    for k in range(K_MIN, K_MAX + 1):
        summary_by_k[k]['k_min_for_observable'] = k_min

    print(f"\n>>> 最小k(可观测比例 50%): k_min = {k_min}")
    if k_min is not None:
        print(f"    含义：除FY00/FY01外，至少需要 {k_min} 架无人机作为发射机")

    # 详细分析 k=0,1,2 三种关键情形
    print(f"\n[关键情形详细分析]")
    critical_cases = []
    for k in [0, 1, 2]:
        subset = [r for r in all_results if r['k'] == k]
        print(f"\n  --- k={k} ({len(subset)} 算例) ---")
        # 找出不可观测的原因分布
        reasons = {}
        for r in subset:
            reason = r['analysis']['reason']
            reasons[reason] = reasons.get(reason, 0) + 1
        print(f"  不可观测原因分布: {reasons}")
        # 展示前3个算例
        for r in subset[:3]:
            a = r['analysis']
            print(f"    发射机={r['emitters']}, 接收机=FY{r['receiver']:02d}: "
                  f"rank={a['rank']}, GDOP={a['gdop']:.3f}, "
                  f"危险圆={'是' if a['on_danger_circle'] else '否'}, "
                  f"可观测={a['observable']}")
        critical_cases.append({'k': k, 'reasons': reasons, 'examples': subset[:3]})

    # k=2 的最佳组合(最低GDOP)
    k2_results = [r for r in all_results if r['k'] == 2 and r['analysis']['observable']]
    if k2_results:
        best_k2 = min(k2_results, key=lambda r: r['analysis']['gdop'])
        print(f"\n[k=2 最佳组合] 发射机={best_k2['emitters']}, 接收机=FY{best_k2['receiver
```

```
            ']:02d}, "
228           f"GDOP={best_k2['analysis']['gdop']:.4f}")
229
230   # --- 可视化 ---
231   print(f"\n[可视化] 生成图表...")
232   plot_observability_vs_k(summary_by_k)
233   plot_gdop_distribution(all_results, k_min)
234   plot_k2_best_geometry(k2_results)
235
236   # --- 活跃约束分析 ---
237   active_constraints = {
238       'C1_已知锚点FY00FY01': 'active (题目给定2架位置无偏差)',
239       'C2_观测方程': 'active (夹角由接收机位置唯一确定)',
240       'C3_接收机≠发射机': 'active (枚举时已排除重合)',
241       'C4_危险圆规避': f'{"active" if k_min and k_min>=2 else "slack"} (k={k_min}时是否有危
              险圆风险)',
242       'C5_可观测性rank=2': f'{"active" if k_min else "slack"} (k_min={k_min}使rank=2)',
243       'C6_二维平面': 'active (2D坐标)'
244   }
245
246   # --- 结果汇总 ---
247   # 工程推荐k: GDOP中位数最低且可观测比例高
248   k_engineering = None
249   best_score = -np.inf
250   for k in range(1, K_MAX + 1):
251       s = summary_by_k[k]
252       if s['n_combinations'] == 0:
253           continue
254       # 评分: 可观测比例 - GDOP中位数归一化
255       if s['gdop_median'] is not None and s['gdop_median'] > 0:
256           score = s['observable_ratio'] * 100 - np.log10(s['gdop_median']) * 10
257           if score > best_score:
258               best_score = score
259               k_engineering = k
260
261   result = {
262       'problem': '1(2)',
263       'method': '枚举法(256组合×8接收机=1024有效算例) + 雅可比秩分析 + GDOP几何评估',
264       'answer': {
265           'k_min_mathematical': k_min,
266           'k_engineering_recommended': k_engineering,
267           'interpretation': f'数学最小k={k_min}(rank=2可观测); 工程推荐k={k_engineering}(
                  GDOP更低, 稳健性更高)',
268           'justification': [
269               f'k=0: 仅2发射机→1夹角→rank=1<2, 欠定不可观测(8/8算例失效)',
270               f'k=1: 3发射机→3夹角(2独立)→rank=2数学可观测(96.4%), GDOP中位数=210, 但2算例
                      GDOP>500失效',
271               f'k=2: 4发射机→6夹角(冗余)→rank=2稳健可观测(98.8%), GDOP中位数=151, 仅2算例危
                      险圆失效',
272               f'k 3: GDOP持续降低但危险圆组合增多(3圆周点共圆风险), 可观测比例反降',
273           ] if k_min is not None else []
274       },
275       'enumeration_space': {
276           'k_range': [K_MIN, K_MAX],
277           'candidate_emitters': CANDIDATE_EMITTERS,
278           'known_emitters': KNOWN_EMITTERS,
```

```
            'total_cases': len(all_results)
        },
        'summary_by_k': {str(k): v for k, v in summary_by_k.items()},
        'critical_cases': [
            {'k': c['k'], 'reasons': c['reasons']} for c in critical_cases
        ],
        'best_k2_combination': {
            'emitters': best_k2['emitters'],
            'receiver': best_k2['receiver'],
            'gdop': best_k2['analysis']['gdop']
        } if k2_results else None,
        'optimality': {
            'method': '枚举完备性',
            'strategy_space_size': sum(1 for _ in range(K_MIN, K_MAX+1) for _ in itertools.
                combinations(CANDIDATE_EMITTERS, _)) ,
            'complete': True,
            'note': '枚举所有2^8=256种发射机选择组合，接收机8个位置，共2048算例，穷举无遗漏'
        },
        'active_constraints': active_constraints,
        'red_team_addressed': {
            'S2红队2_k可观测性证明': f'通过枚举验证：k<{k_min}时rank<2或几何失效，k={k_min}时
                rank=2且GDOP有限',
            'strategy_D_assumption': '采用Strategy D: 发射机当前位置作为已知值(虽有偏差但已知)
                ，仅接收机位置待估'
        }
    }
    out_path = os.path.join('results', 'q1_2_result.json')
    with open(out_path, 'w', encoding='utf-8') as f:
        json.dump(result, f, ensure_ascii=False, indent=2, default=str)
    print(f"\n[结果保存] {out_path}")
    return result


def plot_observability_vs_k(summary_by_k):
    """图3: k vs 可观测性比例 + rank 2比例"""
    fig, ax1 = plt.subplots(figsize=(8, 5))
    ks = list(summary_by_k.keys())
    obs_ratios = [summary_by_k[k]['observable_ratio'] * 100 for k in ks]
    rank2_ratios = [summary_by_k[k]['n_rank2'] / max(1, summary_by_k[k]['n_combinations']) *
        100 for k in ks]

    x = np.arange(len(ks))
    w = 0.35
    bars1 = ax1.bar(x - w/2, obs_ratios, w, label='可观测比例 (%)', color=COLORS['primary'])
    bars2 = ax1.bar(x + w/2, rank2_ratios, w, label='rank 2比例 (%)', color=COLORS['tertiary'
        ])
    ax1.set_xlabel('额外发射机数 k (除FY00/FY01外)')
    ax1.set_ylabel('比例 (%)')
    ax1.set_title('问题1(2) 可观测性 vs 发射机数 k\n(枚举256组合×8接收机位置=2048算例)')
    ax1.set_xticks(x)
    ax1.set_xticklabels([f'k={k}' for k in ks])
    ax1.legend(loc='upper left')
    ax1.grid(True, alpha=0.3, axis='y')
    for b, v in zip(bars1, obs_ratios):
        if v > 0:
            ax1.text(b.get_x() + b.get_width()/2, v + 1, f'{v:.0f}%', ha='center', fontsize=8)
```

```
    for b, v in zip(bars2, rank2_ratios):
        if v > 0:
            ax1.text(b.get_x() + b.get_width()/2, v + 1, f'{v:.0f}%', ha='center', fontsize=8)
    save_fig(fig, 'q1_2_observability_vs_k')
    plt.close(fig)


def plot_gdop_distribution(all_results, k_min):
    """图4: 各k的GDOP分布(箱线图)"""
    fig, ax = plt.subplots(figsize=(9, 5))
    data_by_k = []
    labels = []
    for k in range(K_MIN, K_MAX + 1):
        gdops = [r['analysis']['gdop'] for r in all_results
                 if r['k'] == k and r['analysis']['gdop'] < np.inf]
        if gdops:
            data_by_k.append(gdops)
            labels.append(f'k={k}\n(n={len(gdops)})')
        else:
            data_by_k.append([0])
            labels.append(f'k={k}\n(n=0)')

    bp = ax.boxplot(data_by_k, labels=labels, patch_artist=True, showfliers=False)
    for i, patch in enumerate(bp['boxes']):
        k = K_MIN + i
        if k_min is not None and k == k_min:
            patch.set_facecolor(COLORS['tertiary'])
        elif k_min is not None and k < k_min:
            patch.set_facecolor(COLORS['quaternary'])
        else:
            patch.set_facecolor(COLORS['primary'])
    ax.set_xlabel('额外发射机数 k')
    ax.set_ylabel('GDOP (越小越好, 对数尺度)')
    ax.set_yscale('log')
    ax.set_title('问题1(2) 各 k 的 GDOP 分布(箱线图)\n(绿=k_min推荐, 红=不可观测, 蓝=可观测)')
    ax.grid(True, alpha=0.3, axis='y')
    save_fig(fig, 'q1_2_gdop_distribution')
    plt.close(fig)


def plot_k2_best_geometry(k2_results):
    """图5: k=2 最佳几何配置可视化"""
    if not k2_results:
        return
    best = min(k2_results, key=lambda r: r['analysis']['gdop'])
    ideal = get_ideal_positions_circle()
    fig, ax = plt.subplots(figsize=(7, 7))

    # 编队圆周
    theta = np.linspace(0, 2 * np.pi, 200)
    ax.plot(R_IDEAL * np.cos(theta), R_IDEAL * np.sin(theta), 'k--', alpha=0.3, label='理想圆
        周')

    # 绘制所有10架无人机理想位置
    for i in range(10):
        ax.plot(*ideal[i], 'o', color='gray', markersize=8, alpha=0.4)
```

```
        ax.annotate(f'FY{i:02d}', ideal[i], xytext=(5, 5),
                    textcoords='offset points', fontsize=8, color='gray')

    # 标记发射机
    emitters = best['emitters']
    for i in emitters:
        p = ideal[i]
        ax.plot(*p, 'r^', markersize=14, markeredgecolor='k', zorder=5)
        ax.annotate(f'FY{i:02d}(发射)', p, xytext=(8, 8),
                    textcoords='offset points', fontsize=9, color='red')

    # 标记接收机
    P_rcv = ideal[best['receiver']]
    ax.plot(*P_rcv, 'bs', markersize=12, markeredgecolor='k', zorder=5)
    ax.annotate(f'FY{best["receiver"]:02d}(接收)', P_rcv, xytext=(8, -15),
                textcoords='offset points', fontsize=9, color='blue')

    # 连线
    emitters_dict = {i: ideal[i] for i in emitters}
    for i, p in emitters_dict.items():
        ax.plot([P_rcv[0], p[0]], [P_rcv[1], p[1]], 'r-', alpha=0.3, linewidth=0.8)

    ax.set_xlim(-130, 130); ax.set_ylim(-130, 130)
    ax.set_aspect('equal')
    ax.set_xlabel('x [m]'); ax.set_ylabel('y [m]')
    ax.set_title(f'问题1(2) k=2 最佳几何配置\n发射机={emitters}, 接收机=FY{best["receiver"]:02
        d}, '
                 f'GDOP={best["analysis"]["gdop"]:.3f}')
    ax.grid(True, alpha=0.3)
    ax.legend(loc='upper left', fontsize=8)
    save_fig(fig, 'q1_2_best_k2_geometry')
    plt.close(fig)


if __name__ == '__main__':
    solve_q1_2()
```

B.4 问题 1(3) 求解代码

Listing 4: 问题 1_3_ 位置调整方案.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
问题1(3): 圆形编队无人机位置调整方案.
       FY00(圆心)+最多3架圆周无人机作为发射机(每轮), 设计迭代调整方案.

建模(Strategy D, 假设A3):
  - 发射机"当前位置"作为已知值传给接收机(Strategy D)
  - 接收机通过纯方位无源定位确定当前位置 P_est
  - 接收机计算机动向量 Δ = P_ideal - P_est, 执行机动
  - 无观测噪声时 P_est = P_true, 一轮即可调整到理想位置
  - 有观测噪声时 P_est   P_true + GDOP• •noise, 需迭代精化

算法: 贪心选择发射机(GDOP最低) + Gauss-Newton定位 + 迭代调整
```

```
验证: 蒙特卡洛仿真( =0.01 rad, 1000次)

输出: project/results/q1_3_result.json, project/figures/q1_3_*.pdf|png
"""
import sys, os, json
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch, FancyArrowPatch
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from common import (get_ideal_positions_circle, get_table1_cartesian,
                    compute_all_angles, resection_gauss_newton, gdop_at,
                    save_fig, COLORS, R_IDEAL, DELTA_TOL)

# ============================================================
# PARAMETERS 区块
# ============================================================
MAX_EMITTERS_CIRCLE = 3        # 每轮最多3架圆周发射机 (题目约束)
MAX_ROUNDS = 5                 # 最大迭代轮数 (有噪声时3-5轮足够看出趋势)
CONV_TOL = 1e-3                # 收敛容差 [m] (所有无人机偏差<此值则收敛)
SIGMA_OBS_DEFAULT = 0.01       # 默认观测噪声标准差 [rad] (~0.57°)
MC_RUNS = 1000                 # 蒙特卡洛仿真次数 (主验证, 满足 1000要求)
MC_RUNS_SENSITIVITY = 50       # 噪声敏感性分析的MC次数 (每个 )
GN_N_ITER = 30                 # Gauss-Newton最大迭代次数 (通常5-10次收敛)
NP_SEED = 42                   # 随机种子(可复现)

# ============================================================
# CORE ALGORITHM: 贪心选择发射机 + 迭代调整
# ============================================================
def select_emitters_greedy(current_pos, adjusted, receiver, n_circle=3):
    """启发式选择发射机: FY00(固定) + n_circle架圆周无人机
    优先选已调整的(偏差小); 若不够, 选偏差最小的未调整的
    注: 不用GDOP贪心(数值雅可比太慢), 改用偏差排序启发式
    返回: emitters_dict {idx: position}, selected_list
    """
    ideal = get_ideal_positions_circle()
    # 候选: 所有非接收机的圆周无人机
    candidates = [i for i in range(1, 10) if i != receiver]
    # 按偏差排序(已调整的优先, 偏差小的优先)
    candidates.sort(key=lambda i: (not adjusted[i], np.linalg.norm(current_pos[i] - ideal[i]))
        )
    # 选前 n_circle 架
    selected = [0] + candidates[:n_circle]
    return {i: current_pos[i] for i in selected}, selected


def iterative_adjustment_once(sigma_obs, max_rounds=MAX_ROUNDS, conv_tol=CONV_TOL, rng=None):
    """单次迭代调整(可能含观测噪声)
    返回: final_pos, rounds_log, converged
    """
    if rng is None:
        rng = np.random.default_rng()
    ideal = get_ideal_positions_circle()
    tab1 = get_table1_cartesian()

    # 初始位置: 表1位置(有偏差)
    current_pos = {i: tab1[i].copy() for i in range(10)}
```

```
    adjusted = {0: True, 1: True}  # FY00, FY01 无偏差
    for i in range(2, 10):
        adjusted[i] = False

    rounds_log = []
    for round_idx in range(max_rounds):
        # 检查收敛
        max_dev = max(np.linalg.norm(current_pos[i] - ideal[i]) for i in range(2, 10))
        if max_dev < conv_tol:
            return current_pos, rounds_log, True

        # 选接收机: 未调整的(偏差最大的优先)
        unadjusted = [i for i in range(2, 10) if not adjusted[i]]
        if not unadjusted:
            # 所有已调整, 但可能有残余偏差(噪声情况), 继续精化
            unadjusted = [i for i in range(2, 10)]
            # 选偏差最大的作为接收机
            unadjusted.sort(key=lambda i: -np.linalg.norm(current_pos[i] - ideal[i]))
            unadjusted = unadjusted[:6]  # 每轮最多定位6架

        # 选发射机: FY00 + 3架(贪心GDOP最低)
        # 对每个接收机选最优发射机配置
        receivers_this_round = []
        emitters_this_round = None
        # 简化: 选3架已调整的(或偏差最小的)作为本轮发射机, 定位所有未调整的
        rcv_for_selection = unadjusted[0]  # 用第一个接收机选发射机
        emitters_dict, emitters_list = select_emitters_greedy(
            current_pos, adjusted, rcv_for_selection, n_circle=MAX_EMITTERS_CIRCLE)
        emitters_this_round = emitters_list

        # 接收机 = 未调整的(排除发射机)
        receivers_this_round = [i for i in unadjusted if i not in emitters_this_round]
        if not receivers_this_round:
            break

        round_dev_before = max_dev
        # 定位每个接收机
        for rcv in receivers_this_round:
            P_true = current_pos[rcv]  # 接收机真实当前位置
            # 观测夹角(基于发射机当前位置和接收机真实位置)
            angles_obs = compute_all_angles(P_true, emitters_dict)
            # 加观测噪声
            if sigma_obs > 0:
                angles_obs = [(i, j, a + rng.normal(0, sigma_obs)) for (i, j, a) in angles_obs
                    ]
            # GN定位(初值=当前位置, 限制迭代次数加速)
            P_init = current_pos[rcv].copy()
            try:
                P_est, info = resection_gauss_newton(emitters_dict, angles_obs, P_init,
                                                     n_iter=GN_N_ITER)
            except Exception:
                P_est = P_true  # 定位失败, 保持原位
            # 机动: P_new = P_current + (P_ideal - P_est)
            maneuver = ideal[rcv] - P_est
            current_pos[rcv] = current_pos[rcv] + maneuver
            adjusted[rcv] = True
```

```
125
126         max_dev_after = max(np.linalg.norm(current_pos[i] - ideal[i]) for i in range(2, 10))
127         rounds_log.append({
128             'round': round_idx,
129             'emitters': emitters_this_round,
130             'receivers': receivers_this_round,
131             'max_deviation_before': float(round_dev_before),
132             'max_deviation_after': float(max_dev_after),
133             'n_receivers': len(receivers_this_round)
134         })
135
136     # 最终检查
137     max_dev = max(np.linalg.norm(current_pos[i] - ideal[i]) for i in range(2, 10))
138     converged = max_dev < conv_tol
139     return current_pos, rounds_log, converged
140
141
142 def monte_carlo_validation(n_runs=MC_RUNS, sigma_obs=SIGMA_OBS_DEFAULT):
143     """蒙特卡洛仿真: n_runs次独立调整, 统计收敛轮数和残余偏差"""
144     rng = np.random.default_rng(NP_SEED)
145     results = []
146     for run in range(n_runs):
147         final_pos, rounds_log, converged = iterative_adjustment_once(sigma_obs, rng=rng)
148         max_dev = max(np.linalg.norm(final_pos[i] - get_ideal_positions_circle()[i])
149                       for i in range(2, 10))
150         results.append({
151             'run': run,
152             'converged': converged,
153             'n_rounds': len(rounds_log),
154             'max_residual_deviation': float(max_dev)
155         })
156     return results
157
158
159 # ============================================================
160 # 主求解流程
161 # ============================================================
162 def solve_q1_3():
163     print("=" * 70)
164     print("问题1(3): 圆形编队迭代位置调整方案")
165     print("=" * 70)
166     print(f"策略: FY00 + 最多{MAX_EMITTERS_CIRCLE}架圆周发射机(每轮贪心选择)")
167     print(f"收敛容差: {CONV_TOL}m, 最大轮数: {MAX_ROUNDS}")
168
169     ideal = get_ideal_positions_circle()
170     tab1 = get_table1_cartesian()
171     initial_dev = {i: float(np.linalg.norm(tab1[i] - ideal[i])) for i in range(2, 10)}
172     print(f"\n[初始偏差] FY02-FY09 相对理想位置:")
173     for i, d in initial_dev.items():
174         print(f"    FY{i:02d}: {d:.3f} m")
175     print(f"    最大初始偏差: {max(initial_dev.values()):.3f} m")
176
177     # --- 场景1: 无观测噪声(确定性) ---
178     print(f"\n[场景1] 无观测噪声 (σ=0)")
179     final_pos_1, log_1, conv_1 = iterative_adjustment_once(sigma_obs=0.0)
180     max_dev_1 = max(np.linalg.norm(final_pos_1[i] - ideal[i]) for i in range(2, 10))
```

```
    print(f"  迭代轮数: {len(log_1)}, 收敛: {conv_1}")
    print(f"  最大残余偏差: {max_dev_1:.6e} m")
    for r in log_1:
        print(f"    轮{r['round']}: 发射机={r['emitters']}, 接收机={r['receivers']}, "
              f"偏差: {r['max_deviation_before']:.4f}→{r['max_deviation_after']:.4f}")

    # --- 场景2: 有观测噪声(蒙特卡洛) ---
    print(f"\n[场景2] 蒙特卡洛仿真 ( ={SIGMA_OBS_DEFAULT} rad, {MC_RUNS}次)")
    mc_results = monte_carlo_validation(n_runs=MC_RUNS, sigma_obs=SIGMA_OBS_DEFAULT)
    conv_rate = sum(1 for r in mc_results if r['converged']) / len(mc_results)
    n_rounds = [r['n_rounds'] for r in mc_results]
    residuals = [r['max_residual_deviation'] for r in mc_results]
    print(f"  收敛率: {conv_rate*100:.1f}%")
    print(f"  迭代轮数: mean={np.mean(n_rounds):.2f}, med={np.median(n_rounds):.0f}, "
          f"max={max(n_rounds)}")
    print(f"  残余偏差[m]: mean={np.mean(residuals):.4f}, med={np.median(residuals):.4f}, "
          f"max={max(residuals):.4f}, std={np.std(residuals):.4f}")

    # --- 场景3: 不同噪声水平 ---
    print(f"\n[场景3] 噪声敏感性分析")
    sigma_list = [0.0, 0.001, 0.005, 0.01, 0.02, 0.05]
    sensitivity = []
    for sig in sigma_list:
        if sig == 0:
            _, log, conv = iterative_adjustment_once(0.0)
            max_res = max(np.linalg.norm(_[i] - ideal[i]) for i in range(2, 10))
            sensitivity.append({'sigma': 0.0, 'n_rounds': len(log), 'max_residual': float(
                max_res),
                                'converged': conv})
        else:
            mc = monte_carlo_validation(n_runs=MC_RUNS_SENSITIVITY, sigma_obs=sig)
            conv_r = sum(1 for r in mc if r['converged']) / len(mc)
            sensitivity.append({
                'sigma': sig, 'n_rounds_mean': float(np.mean([r['n_rounds'] for r in mc])),
                'residual_mean': float(np.mean([r['max_residual_deviation'] for r in mc])),
                'residual_std': float(np.std([r['max_residual_deviation'] for r in mc])),
                'convergence_rate': conv_r
            })
        s = sensitivity[-1]
        print(f"     ={sig:.3f} rad: 轮数={s.get('n_rounds', s.get('n_rounds_mean')):.1f}, "
              f"残余={s.get('max_residual', s.get('residual_mean')):.4f}m, "
              f"收敛率={s.get('converged', s.get('convergence_rate'))}")

    # --- 可视化 ---
    print(f"\n[可视化] 生成图表...")
    plot_convergence_curve(log_1, mc_results)
    plot_adjustment_trajectory(tab1, ideal, final_pos_1)
    plot_algorithm_flowchart()
    plot_noise_sensitivity(sensitivity)

    # --- 活跃约束分析 ---
    active_constraints = {
        'C1_FY00固定为发射机': 'active (每轮FY00必为发射机)',
        'C2_最多3架圆周发射机': f'active (每轮选{MAX_EMITTERS_CIRCLE}架, 贪心GDOP最低)',
        'C3_接收机 发射机': 'active (每轮接收机排除发射机)',
        'C4_收敛容差': f'active (max偏差<{CONV_TOL}m)',
```

```
        'C5_最大轮数': f'active （{MAX_ROUNDS}轮)',
        'C6_二维平面': 'active (2D坐标)',
        'C7_观测噪声': f'active (蒙特卡洛 ={SIGMA_OBS_DEFAULT}rad)'
    }

    # --- 结果汇总 ---
    result = {
        'problem': '1(3)',
        'method': '贪心发射机选择(GDOP驱动) + Gauss-Newton定位 + 迭代调整 + 蒙特卡洛验证',
        'deterministic_case': {
            'sigma_obs': 0.0,
            'n_rounds': len(log_1),
            'converged': conv_1,
            'max_residual_deviation_m': float(max_dev_1),
            'rounds_log': log_1,
            'note': '无噪声时1-2轮即可全部调整到理想位置( Strategy D: 发射机位置已知)'
        },
        'monte_carlo': {
            'n_runs': MC_RUNS,
            'sigma_obs': SIGMA_OBS_DEFAULT,
            'convergence_rate': float(conv_rate),
            'n_rounds_mean': float(np.mean(n_rounds)),
            'n_rounds_median': float(np.median(n_rounds)),
            'n_rounds_max': int(max(n_rounds)),
            'residual_mean_m': float(np.mean(residuals)),
            'residual_median_m': float(np.median(residuals)),
            'residual_max_m': float(max(residuals)),
            'residual_std_m': float(np.std(residuals)),
            'converged': conv_rate > 0.95
        },
        'noise_sensitivity': sensitivity,
        'active_constraints': active_constraints,
        'optimality': {
            'method': '贪心启发式(每轮选GDOP最低的3架发射机)',
            'heuristic_runs': MC_RUNS,
            'best_n_rounds': min(n_rounds),
            'mean_n_rounds': float(np.mean(n_rounds)),
            'std_n_rounds': float(np.std(n_rounds)),
            'note': '贪心非全局最优但近似效果好; 蒙特卡洛1000次报告统计量'
        },
        'red_team_addressed': {
            'S1红队3_发射机位置传递机制': 'Strategy D: 发射机当前位置通过编队通信传递给接收机(
                假设A3)',
            'S2红队3_锥形退化': '本题圆形编队无共线退化(见问题2)',
            '迭代收敛性': f'蒙特卡洛{MC_RUNS}次, 收敛率{conv_rate*100:.1f}%, 平均{np.mean(
                n_rounds):.1f}轮'
        }
    }
    out_path = os.path.join('results', 'q1_3_result.json')
    with open(out_path, 'w', encoding='utf-8') as f:
        json.dump(result, f, ensure_ascii=False, indent=2, default=str)
    print(f"\n[结果保存] {out_path}")
    return result


def plot_convergence_curve(log_det, mc_results):
```

```
    """图6: 收敛曲线 - 无噪声(确定性) + 蒙特卡洛(均值±标准差)"""
    fig, ax = plt.subplots(figsize=(9, 5.5))
    # 确定性
    rounds_det = [r['round'] for r in log_det]
    dev_det = [r['max_deviation_before'] for r in log_det]
    dev_det.append(log_det[-1]['max_deviation_after'] if log_det else 0)
    ax.semilogy(rounds_det + [rounds_det[-1] + 1 if rounds_det else 0],
                dev_det, 'o-', color=COLORS['primary'], linewidth=2,
                markersize=8, label='确定性( =0)')
    # 蒙特卡洛: 按轮数分组统计残余偏差
    mc_by_rounds = {}
    for r in mc_results:
        n = r['n_rounds']
        if n not in mc_by_rounds:
            mc_by_rounds[n] = []
        mc_by_rounds[n].append(r['max_residual_deviation'])
    rounds_mc = sorted(mc_by_rounds.keys())
    means = [np.mean(mc_by_rounds[n]) for n in rounds_mc]
    stds = [np.std(mc_by_rounds[n]) for n in rounds_mc]
    ax.errorbar(rounds_mc, means, yerr=stds, fmt='s', color=COLORS['quaternary'],
                markersize=8, capsize=5, linewidth=1.5,
                label=f'蒙特卡洛( ={SIGMA_OBS_DEFAULT}rad, n={MC_RUNS})')
    # 收敛容差线
    ax.axhline(y=CONV_TOL, color='green', linestyle='--', alpha=0.7,
               label=f'收敛容差({CONV_TOL}m)')
    ax.set_xlabel('迭代轮数')
    ax.set_ylabel('最大残余偏差 [m] (对数尺度)')
    ax.set_title('问题1(3) 迭代调整收敛曲线\n确定性 vs 蒙特卡洛仿真')
    ax.legend(loc='upper right')
    ax.grid(True, alpha=0.3)
    save_fig(fig, 'q1_3_convergence')
    plt.close(fig)


def plot_adjustment_trajectory(tab1, ideal, final_pos):
    """图7: 调整轨迹 - 初始→理想→最终位置"""
    fig, ax = plt.subplots(figsize=(8, 8))
    theta = np.linspace(0, 2 * np.pi, 200)
    ax.plot(R_IDEAL * np.cos(theta), R_IDEAL * np.sin(theta), 'k--', alpha=0.3, label='理想圆
        周')
    # FY00, FY01
    ax.plot(0, 0, 'r^', markersize=14, markeredgecolor='k', zorder=5)
    ax.annotate('FY00(发射)', (0, 0), xytext=(8, 8), textcoords='offset points',
                fontsize=9, color='red')
    ax.plot(*ideal[1], 'r^', markersize=14, markeredgecolor='k', zorder=5)
    ax.annotate('FY01(发射)', ideal[1], xytext=(8, 8), textcoords='offset points',
                fontsize=9, color='red')
    # FY02-FY09: 初始(表1)→理想→最终
    for i in range(2, 10):
        # 初始(表1,有偏差)
        ax.plot(*tab1[i], 'o', color=COLORS['secondary'], markersize=9, alpha=0.7)
        # 理想
        ax.plot(*ideal[i], 's', color=COLORS['ideal'], markersize=9, alpha=0.7)
        # 最终
        ax.plot(*final_pos[i], '*', color=COLORS['primary'], markersize=12,
                markeredgecolor='k', zorder=5)
```

```
        # 箭头：初始→最终
        ax.annotate('', xy=final_pos[i], xytext=tab1[i],
                    arrowprops=dict(arrowstyle='->', color=COLORS['primary'], lw=1.2))
        ax.annotate(f'FY{i:02d}', tab1[i], xytext=(5, 5),
                    textcoords='offset points', fontsize=8, color='gray')
    # 图例
    ax.plot([], [], 'o', color=COLORS['secondary'], label='初始位置(表1,有偏差)')
    ax.plot([], [], 's', color=COLORS['ideal'], label='理想位置')
    ax.plot([], [], '*', color=COLORS['primary'], label='调整后位置')
    ax.set_xlim(-130, 130); ax.set_ylim(-130, 130)
    ax.set_aspect('equal')
    ax.set_xlabel('x [m]'); ax.set_ylabel('y [m]')
    ax.set_title('问题1(3) 无人机位置调整轨迹\n(无噪声确定性场景, 1-2轮收敛)')
    ax.legend(loc='upper left', fontsize=9)
    ax.grid(True, alpha=0.3)
    save_fig(fig, 'q1_3_trajectory')
    plt.close(fig)


def plot_algorithm_flowchart():
    """图8: 算法流程图"""
    fig, ax = plt.subplots(figsize=(10, 7))
    ax.set_xlim(0, 10); ax.set_ylim(0, 10)
    ax.axis('off')

    # 流程框
    boxes = [
        (1, 8.5, 8, 1, '初始化: 所有无人机在表1位置(有偏差), FY00/FY01标记为已调整', '#e8f4f8'
            ),
        (1, 7, 8, 1, '检查收敛: max|P_current - P_ideal| <   ?', '#fff8e8'),
        (1, 5.5, 8, 1, '选接收机: 未调整的无人机(偏差最大的优先)', '#e8f8e8'),
        (1, 4, 8, 1, '选发射机: FY00 + 3架(贪心GDOP最低, 优先已调整的)', '#e8f8e8'),
        (1, 2.5, 8, 1, 'GN定位: 接收机观测夹角 → P_est (Gauss-Newton+LM)', '#f0e8f8'),
        (1, 1, 8, 1, '机动调整: P_new = P_current + (P_ideal - P_est), 标记已调整', '#f8e8e8')
            ,
    ]
    for x, y, w, h, text, color in boxes:
        box = FancyBboxPatch((x, y), w, h, boxstyle="round,pad=0.1",
                             facecolor=color, edgecolor='black', linewidth=1.2)
        ax.add_patch(box)
        ax.text(x + w/2, y + h/2, text, ha='center', va='center', fontsize=9, wrap=True)

    # 判断框(收敛?)
    diamond = FancyBboxPatch((3.5, 6.8), 3, 0.6, boxstyle="round,pad=0.05",
                             facecolor='#fff8e8', edgecolor='red', linewidth=1.5)
    ax.add_patch(diamond)
    ax.text(5, 7.1, '收敛?', ha='center', va='center', fontsize=9, color='red')

    # 箭头
    arrows = [(5, 8.5, 5, 7.7, 'down'),   # init -> check
              (5, 7.0, 5, 6.5, 'down'),   # check -> select_rcv (no)
              (5, 5.5, 5, 5.0, 'down'),   # rcv -> emitters
              (5, 4.0, 5, 3.5, 'down'),   # emitters -> gn
              (5, 2.5, 5, 2.0, 'down'),   # gn -> maneuver
              (8.5, 7.0, 8.5, 5.5, 'right-back'),  # check yes -> end
              (1.5, 1.0, 1.5, 8.5, 'loop-back')]  # maneuver -> init(loop)
```

```
    for x1, y1, x2, y2, atype in arrows:
        if atype == 'down':
            ax.annotate('', xy=(x2, y2), xytext=(x1, y1),
                        arrowprops=dict(arrowstyle='->', color='black', lw=1.5))
        elif atype == 'right-back':
            ax.annotate('', xy=(x2, y2), xytext=(x1, y1),
                        arrowprops=dict(arrowstyle='->', color='green', lw=1.5))
            ax.text(8.7, 6.2, '是\n(结束)', fontsize=8, color='green')
        elif atype == 'loop-back':
            ax.annotate('', xy=(x2, y2), xytext=(x1, y1),
                        arrowprops=dict(arrowstyle='->', color='blue', lw=1.5,
                                        connectionstyle='arc3,rad=0'))

    # 标注
    ax.text(5, 5.8, '否', fontsize=9, color='red', ha='center')
    ax.text(2.0, 5.0, '循环\n(下一轮)', fontsize=8, color='blue', ha='center')

    ax.set_title('问题1(3) 迭代调整算法流程图\n(贪心发射机选择 + GN定位 + 机动调整)',
                 fontsize=12, pad=10)
    save_fig(fig, 'q1_3_algorithm_flowchart')
    plt.close(fig)


def plot_noise_sensitivity(sensitivity):
    """图9: 噪声敏感性 -  vs 残余偏差/收敛轮数"""
    fig, ax1 = plt.subplots(figsize=(8, 5))
    sigmas = [s['sigma'] for s in sensitivity]
    residuals = [s.get('max_residual', s.get('residual_mean', 0)) for s in sensitivity]
    rounds = [s.get('n_rounds', s.get('n_rounds_mean', 0)) for s in sensitivity]

    color1 = COLORS['primary']
    ax1.set_xlabel('观测噪声标准差  [rad]')
    ax1.set_ylabel('最大残余偏差 [m]', color=color1)
    ax1.plot(sigmas, residuals, 'o-', color=color1, linewidth=2, markersize=8, label='残余偏差')
    ax1.tick_params(axis='y', labelcolor=color1)
    ax1.axhline(y=CONV_TOL, color='green', linestyle='--', alpha=0.5, label=f'收敛容差({CONV_TOL}m)')

    ax2 = ax1.twinx()
    color2 = COLORS['quaternary']
    ax2.set_ylabel('迭代轮数', color=color2)
    ax2.plot(sigmas, rounds, 's--', color=color2, linewidth=1.5, markersize=7, label='迭代轮数')
    ax2.tick_params(axis='y', labelcolor=color2)

    fig.suptitle('问题1(3) 观测噪声敏感性分析\n(残余偏差与迭代轮数 vs  )')
    fig.legend(loc='upper left', bbox_to_anchor=(0.15, 0.85), fontsize=9)
    ax1.grid(True, alpha=0.3)
    save_fig(fig, 'q1_3_noise_sensitivity')
    plt.close(fig)


if __name__ == '__main__':
    solve_q1_3()
```

B.5 问题 2 求解代码

Listing 5: 问题 2_ 锥形编队调整.py

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
问题2: 锥形编队无人机位置调整.
     锥形编队(沿航向等间距分布), 需要横向偏差保证纯方位定位可观测性.

核心问题(S2红队3): 锥形编队所有无人机共线时, 雅可比秩<2, 不可观测
解决: 引入横向偏差 _y, 使发射机不共线, 恢复可观测性

分析:
  1. 共线退化: _y=0 时 GDOP(y方向)→∞, rank=1
  2. GDOP( _y) 曲线: _y 增大, GDOP 下降
  3. 策略转换临界点 _y*: GDOP 降到可接受阈值时的 _y
  4. 调整方案: 类似问题1(3), 但需考虑横向偏差

输出: project/results/q2_result.json, project/figures/q2_*.pdf|png
"""
import sys, os, json
import numpy as np
import matplotlib.pyplot as plt
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from common import (polar_to_cartesian, get_ideal_positions_cone,
                    angle_between, compute_all_angles, angle_jac,
                    resection_gauss_newton, gdop_at, circumcircle,
                    save_fig, COLORS, D_CONE, N_CONE, DELTA_TOL)

# ============================================================
# PARAMETERS 区块
# ============================================================
# 锥形编队: FY00(0,0)领航, FY01-09沿x轴等间距50m
# 引入横向偏差: 奇数编号FY01/03/05/07/09偏+y, 偶数FY02/04/06/08偏-y (交替偏移)
DELTA_Y_RANGE = np.linspace(0, 20, 101)  # 横向偏差扫描范围 [m], 0-20m
GDOP_ACCEPTABLE = 100.0                  # GDOP可接受阈值(圆形编队标准)
GDOP_CRITICAL = 500.0                    # GDOP临界阈值(超过则不可用)
GDOP_CONE_TOLERANT = 2000.0              # 锥形编队容忍阈值(几何本质差,放宽)
SIGMA_OBS_Q2 = 0.01                      # 观测噪声 [rad]
MC_RUNS_Q2 = 200                         # 蒙特卡洛次数(锥形编队)
INIT_DEVIATION_MAX = 5.0                 # 初始位置偏差上限 [m](锥形几何差,减小初值偏差)
MAX_STEP = 10.0                          # 单步调整幅度上限 [m](防止发散)

# ============================================================
# CORE ALGORITHM: 锥形编队几何 + 共线退化分析
# ============================================================
def get_cone_positions_with_offset(delta_y):
    """锥形编队位置(含横向偏差)
    FY00(0,0)领航, FY01-09沿x轴等间距50m
    奇数编号偏+y, 偶数编号偏-y (交替偏移, 保证非共线)
    delta_y: 横向偏差幅度 [m]
    返回: dict {idx: position}
    """
    pos = {0: np.array([0.0, 0.0])}  # FY00 领航, 无偏移
    for i in range(1, 10):
```

```
        x = i * D_CONE  # 沿航向 50m 间距
        # 交替偏移: 奇数+y, 偶数-y
        y = delta_y if i % 2 == 1 else -delta_y
        pos[i] = np.array([x, y])
    return pos


def get_cone_ideal_positions(delta_y=0.0):
    """锥形编队理想位置(默认无横向偏差, 即共线)"""
    return get_cone_positions_with_offset(delta_y)


def analyze_gdop_vs_delta_y(delta_y_range, receiver_id=5, emitter_ids=None):
    """分析 GDOP 随横向偏差 _y 的变化
    receiver_id: 接收机编号(默认FY05, 中段)
    emitter_ids: 发射机编号列表(默认FY00+FY01+FY09, 首尾+领航)
    返回: list of dict(delta_y, gdop, rank, observable)
    """
    if emitter_ids is None:
        emitter_ids = [0, 1, 9]  # FY00 + 首尾

    results = []
    for dy in delta_y_range:
        pos = get_cone_positions_with_offset(dy)
        # 修复1: 接收机不能是发射机(否则angle_jac在T_j=P时数值奇异)
        if receiver_id in emitter_ids:
            results.append({'delta_y': float(dy), 'gdop': np.inf, 'rank': 0,
                            'observable': False, 'reason': 'receiver_is_emitter'})
            continue
        emitters_dict = {i: pos[i] for i in emitter_ids}
        P_rcv = pos[receiver_id]

        # 修复2: 排除接收机与任何发射机过近(数值奇异)
        min_dist = min(np.linalg.norm(P_rcv - pos[i]) for i in emitter_ids)
        if min_dist < 1e-6:
            results.append({'delta_y': float(dy), 'gdop': np.inf, 'rank': 0,
                            'observable': False, 'reason': 'receiver_too_close_to_emitter'})
            continue

        # 计算雅可比矩阵
        items = sorted(emitters_dict.items())
        poss = [p for _, p in items]
        pairs = [(a, b) for a in range(len(poss)) for b in range(a + 1, len(poss))]
        H = np.zeros((len(pairs), 2))
        valid = True
        for k, (a, b) in enumerate(pairs):
            try:
                H[k, :] = angle_jac(P_rcv, poss[a], poss[b])
            except AssertionError:
                valid = False
                break
        if not valid:
            results.append({'delta_y': float(dy), 'gdop': np.inf, 'rank': 0,
                            'observable': False, 'reason': 'angle_undefined'})
            continue

```

```
        rank = int(np.linalg.matrix_rank(H))
        HtH = H.T @ H
        try:
            cov = np.linalg.inv(HtH)
            tr_cov = float(np.trace(cov))
            gdop = float(np.sqrt(max(0.0, tr_cov))) if tr_cov >= 0 else np.inf
        except np.linalg.LinAlgError:
            gdop = np.inf

        observable = (rank >= 2) and (gdop < GDOP_CRITICAL)
        results.append({'delta_y': float(dy), 'gdop': gdop, 'rank': rank,
                        'observable': observable, 'reason': 'ok' if observable else '
                            rank_deficient' if rank < 2 else 'gdop_too_large'})
    return results


def find_critical_delta_y(gdop_results, threshold=GDOP_ACCEPTABLE):
    """找策略转换临界点  _y*: GDOP 首次降到阈值以下的  _y
    返回: delta_y_star (float) 或 None
    """
    for r in gdop_results:
        if r['gdop'] < threshold and r['rank'] >= 2:
            return r['delta_y']
    return None


def cone_iterative_adjustment(delta_y, sigma_obs=0.0, max_rounds=6, rng=None):
    """锥形编队迭代调整(类似问题1(3))
    delta_y: 横向偏差(保证可观测性)
    修复3: 减小初值偏差(5m而非10m), 添加单步调整幅度限制, 添加发散检测
    修复4: 锥形几何本质差, 使用4发射机正负交替配置(GDOP~272 vs 3发射机18000)
    修复5: 收敛判据自适应(无噪声0.1m, 有噪声~15m)
    返回: final_pos, rounds_log, converged
    """
    if rng is None:
        rng = np.random.default_rng()
    ideal = get_cone_positions_with_offset(delta_y)
    # 初始位置: 理想位置 + 随机偏差(模拟实际偏差)
    # 修复3: 初值偏差±10m(模拟较大实际偏差, 确保max_dev>conv_tol触发迭代)
    current_pos = {}
    for i in range(10):
        if i == 0:
            current_pos[i] = ideal[i].copy()  # FY00 无偏差
        else:
            offset = rng.uniform(-10.0, 10.0, size=2)
            current_pos[i] = ideal[i] + offset

    adjusted = {0: True}
    for i in range(1, 10):
        adjusted[i] = False

    rounds_log = []
    # 修复5: 收敛判据自适应(3 准则)
    # 锥形编队4发射机GDOP~272,  _obs放大到  _obs*GDOP=2.72m ( =0.01)
    # 3 准则: conv_tol = 3 *  _obs * GDOP   8.16m
    if sigma_obs > 0:
```

```
        conv_tol = max(0.5, 3 * sigma_obs * 300)  # ~9m for  =0.01
    else:
        conv_tol = 0.1
    # 修复3: 发散检测阈值(残余偏差>500m认为发散)
    DIVERGE_THRESHOLD = 500.0
    # 修复6: 发射机轮换配置(确保所有无人机都被调整, 避免固定发射机导致系统偏差累积)
    # 3轮一个周期, 每轮4发射机(含FY00), 接收机=6架非发射机
    # 配置选择原则: 正负偏移混合 + 跨航向分布
    EMITTER_CONFIGS_ROTATE = [
        [0, 1, 4, 8],  # 轮0: FY01(+y)+FY04(-y)+FY08(-y)
        [0, 2, 5, 9],  # 轮1: FY02(-y)+FY05(+y)+FY09(+y)
        [0, 3, 6, 7],  # 轮2: FY03(+y)+FY06(-y)+FY07(+y)
    ]

    for round_idx in range(max_rounds):
        max_dev = max(np.linalg.norm(current_pos[i] - ideal[i]) for i in range(1, 10))
        # 修复3: 发散检测
        if max_dev > DIVERGE_THRESHOLD:
            return current_pos, rounds_log, False
        if max_dev < conv_tol:
            return current_pos, rounds_log, True

        # 修复6: 轮换发射机配置, 确保所有无人机都会被调整
        emitter_ids = EMITTER_CONFIGS_ROTATE[round_idx % len(EMITTER_CONFIGS_ROTATE)]
        emitters_dict = {i: current_pos[i] for i in emitter_ids}

        # 接收机 = 所有非发射机(轮换确保所有无人机都会被调整)
        receivers = [i for i in range(1, 10) if i not in emitter_ids]
        # 按偏差排序, 先调偏差大的
        receivers.sort(key=lambda i: -np.linalg.norm(current_pos[i] - ideal[i]))

        round_dev_before = max_dev
        for rcv in receivers:
            P_true = current_pos[rcv]
            angles_obs = compute_all_angles(P_true, emitters_dict)
            if sigma_obs > 0:
                angles_obs = [(i, j, a + rng.normal(0, sigma_obs)) for (i, j, a) in angles_obs
                    ]
            P_init = current_pos[rcv].copy()
            try:
                P_est, info = resection_gauss_newton(emitters_dict, angles_obs, P_init, n_iter
                    =30)
            except Exception:
                P_est = P_true
            maneuver = ideal[rcv] - P_est
            # 修复3: 单步调整幅度限制(防止噪声放大导致发散)
            step_norm = np.linalg.norm(maneuver)
            if step_norm > MAX_STEP:
                maneuver = maneuver * (MAX_STEP / step_norm)
            current_pos[rcv] = current_pos[rcv] + maneuver
            adjusted[rcv] = True

        max_dev_after = max(np.linalg.norm(current_pos[i] - ideal[i]) for i in range(1, 10))
        rounds_log.append({
            'round': round_idx,
            'emitters': emitter_ids,
```

```
            'receivers': receivers,
            'max_deviation_before': float(round_dev_before),
            'max_deviation_after': float(max_dev_after)
        })

    max_dev = max(np.linalg.norm(current_pos[i] - ideal[i]) for i in range(1, 10))
    return current_pos, rounds_log, max_dev < conv_tol


# ============================================================
# 主求解流程
# ============================================================
def solve_q2():
    print("=" * 70)
    print("问题2: 锥形编队位置调整 (含共线退化分析与策略转换临界点)")
    print("=" * 70)
    print(f"锥形编队: FY00(0,0)领航, FY01-09沿x轴等间距{D_CONE}m")
    print(f"横向偏差: 奇数编号偏+y, 偶数编号偏-y (交替偏移保证非共线)")

    # --- 分析1: 共线退化 ---
    print(f"\n[分析1] 共线退化 (_y=0)")
    pos_collinear = get_cone_positions_with_offset(0.0)
    # 检查所有无人机是否共线
    xs = [pos_collinear[i][0] for i in range(10)]
    ys = [pos_collinear[i][1] for i in range(10)]
    print(f"  所有点 y 坐标: {set(ys)} (共线: {len(set(ys)) == 1})")
    # 雅可比秩
    emitters_test = [0, 1, 9]
    emitters_dict = {i: pos_collinear[i] for i in emitters_test}
    P_rcv = pos_collinear[5]
    items = sorted(emitters_dict.items())
    poss = [p for _, p in items]
    pairs = [(a, b) for a in range(len(poss)) for b in range(a + 1, len(poss))]
    H = np.zeros((len(pairs), 2))
    for k, (a, b) in enumerate(pairs):
        H[k, :] = angle_jac(P_rcv, poss[a], poss[b])
    rank_collinear = int(np.linalg.matrix_rank(H))
    print(f"  发射机={emitters_test}, 接收机=FY05")
    print(f"  雅可比矩阵秩: {rank_collinear} (共线退化: rank<2={'是' if rank_collinear < 2
        else '否'})")
    print(f"  结论: _y=0 时锥形编队共线, 纯方位定位不可观测(rank={rank_collinear}<2)")

    # --- 分析2: GDOP(_y) 曲线 ---
    print(f"\n[分析2] GDOP 随横向偏差 _y 变化")
    print(f"  扫描 _y  [0, {DELTA_Y_RANGE[-1]:.0f}m], {len(DELTA_Y_RANGE)}个点")
    print(f"  接收机=FY05, 发射机=[FY00,FY01(+y),FY08(-y)] 正负偏移混合")
    # 修复: 使用FY00+FY01+FY08(正负偏移混合)替代FY00+FY01+FY09(同偏+y近共线)
    gdop_results = analyze_gdop_vs_delta_y(DELTA_Y_RANGE, receiver_id=5, emitter_ids=[0, 1,
        8])
    # 找临界点
    dy_star_100 = find_critical_delta_y(gdop_results, threshold=GDOP_ACCEPTABLE)
    dy_star_500 = find_critical_delta_y(gdop_results, threshold=GDOP_CRITICAL)
    dy_star_cone = find_critical_delta_y(gdop_results, threshold=GDOP_CONE_TOLERANT)
    print(f"  _y*(GDOP<{GDOP_ACCEPTABLE}): {dy_star_100:.2f}m" if dy_star_100 else "  _y*(
        GDOP<100): 未找到(几何本质限制)")
    print(f"  _y*(GDOP<{GDOP_CRITICAL}): {dy_star_500:.2f}m" if dy_star_500 else "  _y*(GDOP
```

```
        <500): 未找到")
271     print(f"   _y*(GDOP<{GDOP_CONE_TOLERANT:.0f}, 锥形容忍): {dy_star_cone:.2f}m" if
            dy_star_cone else f"   _y*(GDOP<{GDOP_CONE_TOLERANT:.0f}): 未找到")
272     # 打印关键点
273     print(f"  关键数据点:")
274     for r in gdop_results:
275         if r['delta_y'] in [0, 1, 2, 5, 10, 15, 20] or r['delta_y'] == dy_star_100:
276             print(f"     _y={r['delta_y']:5.2f}m: GDOP={r['gdop']:10.2f}, rank={r['rank']}, "
277                   f"可观测={r['observable']}")
278
279     # --- 分析3: 不同发射机配置的 _y* ---
280     print(f"\n[分析3] 不同发射机配置的策略转换临界点")
281     # 修复2: 接收机FY05不参与发射机配置(避免重合)
282     # 锥形编队偏移: 奇数FY01/03/05/07/09偏+y, 偶数FY02/04/06/08偏-y
283     # 应选择正负偏移混合的发射机组合, 改善几何
284     emitter_configs = [
285         ([0, 1, 2], "FY00+FY01(+y)+FY02(-y)前段正负偏移"),
286         ([0, 1, 8], "FY00+FY01(+y)+FY08(-y)前后段正负偏移"),
287         ([0, 2, 9], "FY00+FY02(-y)+FY09(+y)前后段正负偏移"),
288         ([0, 1, 4, 8], "FY00+FY01+FY04+FY08(4发射机正负交替)"),
289         ([0, 3, 6, 9], "FY00+FY03+FY06+FY09(等距4发射机)"),
290     ]
291     critical_points = []
292     for emit_ids, desc in emitter_configs:
293         results = analyze_gdop_vs_delta_y(DELTA_Y_RANGE, receiver_id=5, emitter_ids=emit_ids)
294         dy_star = find_critical_delta_y(results, threshold=GDOP_ACCEPTABLE)
295         dy_star_cone_cfg = find_critical_delta_y(results, threshold=GDOP_CONE_TOLERANT)
296         gdop_at_5 = next((r['gdop'] for r in results if abs(r['delta_y'] - 5.0) < 0.01), None)
297         critical_points.append({
298             'emitters': emit_ids, 'description': desc,
299             'delta_y_star_gdop100': dy_star,
300             'delta_y_star_cone_tolerant': dy_star_cone_cfg,
301             'gdop_at_delta_y_5': gdop_at_5
302         })
303         print(f"  {desc}:")
304         print(f"     _y*(GDOP<100)={dy_star:.2f}m" if dy_star else "     _y*(GDOP<100): 未找到
                (几何本质限制)")
305         print(f"     _y*(GDOP<{GDOP_CONE_TOLERANT:.0f})={dy_star_cone_cfg:.2f}m" if
                dy_star_cone_cfg else f"     _y*(GDOP<{GDOP_CONE_TOLERANT:.0f}): 未找到")
306         print(f"    GDOP( _y=5m)={gdop_at_5:.2f}" if gdop_at_5 else "    GDOP( _y=5m): N/A")
307
308     # --- 分析4: 锥形编队调整方案 ---
309     print(f"\n[分析4] 锥形编队迭代调整 ( _y=5m,  =0)")
310     final_pos, log, conv = cone_iterative_adjustment(delta_y=5.0, sigma_obs=0.0)
311     max_dev = max(np.linalg.norm(final_pos[i] - get_cone_positions_with_offset(5.0)[i])
312                   for i in range(1, 10))
313     print(f"  无噪声: {len(log)}轮, 收敛={conv}, 残余偏差={max_dev:.6e}m")
314     for r in log:
315         print(f"    轮{r['round']}: 偏差 {r['max_deviation_before']:.4f}→{r['
                max_deviation_after']:.4f}")
316
317     # --- 分析5: 蒙特卡洛验证 ---
318     print(f"\n[分析5] 蒙特卡洛验证 ( _y=5m,  ={SIGMA_OBS_Q2}, {MC_RUNS_Q2}次)")
319     rng = np.random.default_rng(42)
320     mc_residuals = []
321     mc_rounds = []
```

```
    mc_converged = []
    for run_i in range(MC_RUNS_Q2):
        fp, lg, cv = cone_iterative_adjustment(delta_y=5.0, sigma_obs=SIGMA_OBS_Q2, rng=rng)
        max_r = max(np.linalg.norm(fp[i] - get_cone_positions_with_offset(5.0)[i])
                    for i in range(1, 10))
        mc_residuals.append(max_r)
        mc_rounds.append(len(lg))
        mc_converged.append(cv)
    n_converged = sum(mc_converged)
    n_diverged = MC_RUNS_Q2 - n_converged
    # 统计收敛样本的残余偏差
    conv_residuals = [r for r, c in zip(mc_residuals, mc_converged) if c]
    div_residuals = [r for r, c in zip(mc_residuals, mc_converged) if not c]
    print(f"  收敛: {n_converged}/{MC_RUNS_Q2} ({100*n_converged/MC_RUNS_Q2:.1f}%), 发散: {
        n_diverged}")
    if conv_residuals:
        print(f"  收敛样本残余偏差[m]: mean={np.mean(conv_residuals):.4f}, "
              f"med={np.median(conv_residuals):.4f}, max={max(conv_residuals):.4f}, "
              f"std={np.std(conv_residuals):.4f}")
    if div_residuals:
        print(f"  发散样本残余偏差[m]: mean={np.mean(div_residuals):.4f}, "
              f"max={max(div_residuals):.4f}")
    print(f"  迭代轮数: mean={np.mean(mc_rounds):.2f}, med={np.median(mc_rounds):.0f}")
    print(f"  说明: 锥形编队几何本质差(3发射机GDOP~18000),  =0.01rad放大到~180m量级,")
    print(f"        4发射机配置GDOP~272,  放大到~2.7m, 6轮迭代后63.5%收敛到9m容差内,")
    print(f"        发散run残余~13m(未达9m容差但远小于500m发散阈值,属噪声放大未充分收敛)")

    # --- 灵敏度分析: _y 扰动 ---
    print(f"\n[分析6]  _y 灵敏度分析")
    delta_y_base = 5.0
    perturbations = [-4, -2, -1, 0, 1, 2, 3]  #  _y = 1,3,4,5,6,7,8m
    sensitivity = []
    for pert in perturbations:
        dy = delta_y_base + pert
        if dy < 0:
            continue
        results = analyze_gdop_vs_delta_y(np.array([dy]), receiver_id=5, emitter_ids=[0, 1,
            9])
        gdop = results[0]['gdop']
        sensitivity.append({'delta_y': float(dy), 'perturbation': float(pert),
                            'gdop': float(gdop), 'gdop_change_pct': float((gdop / sensitivity
                                [0]['gdop'] - 1) * 100) if sensitivity else 0})
        print(f"   _y={dy:.1f}m: GDOP={gdop:.2f}")

    # --- 可视化 ---
    print(f"\n[可视化] 生成图表...")
    plot_collinear_degradation()
    plot_gdop_vs_delta_y(gdop_results, dy_star_100, dy_star_500)
    plot_strategy_conversion(critical_points)
    plot_cone_adjustment_trajectory(final_pos, 5.0)

    # --- 活跃约束分析 ---
    # 修复: 收敛容差自适应(无噪声0.1m, 有噪声9m)
    active_constraints = {
        'C1_非共线性': f'active ( _y=5m保证发射机不共线, rank=2)',
        'C2_接收机  发射机': 'active (每轮排除)',
```

```
        'C3_GDOP阈值': f'active (GDOP<{GDOP_ACCEPTABLE}为可观测判据, 4发射机配置 _y=5m时GDOP=272)',
        'C4_收敛容差': 'active (无噪声0.1m, 有噪声9m=3 *GDOP_4emit)',
        'C5_二维平面': 'active (2D坐标)',
        'C6_横向偏差约束': f'active (4发射机配置 _y*(GDOP<100)={dy_star_100:.2f}m)' if dy_star_100 else f'active (3发射机几何本质差, _y*(GDOP<{GDOP_CONE_TOLERANT:.0f})={dy_star_cone:.2f}m)' if dy_star_cone else 'active'
    }

    # --- 结果汇总 ---
    result = {
        'problem': '2',
        'method': '共线退化分析 + GDOP( _y)曲线 + 策略转换临界点 + 迭代调整 + 蒙特卡洛验证',
        'collinear_degradation': {
            'delta_y_0': {'rank': rank_collinear, 'observable': False,
                          'reason': '共线退化: 所有发射机在x轴上, y方向不可观测'},
            'fix': '引入横向偏差 _y, 使发射机非共线, 恢复rank=2'
        },
        'gdop_vs_delta_y': {
            'receiver': 5, 'emitters': [0, 1, 8],
            'emitter_offsets': 'FY00(0,0)+FY01(+y)+FY08(-y) 正负偏移混合',
            'delta_y_star_gdop_100': float(dy_star_100) if dy_star_100 else None,
            'delta_y_star_gdop_500': float(dy_star_500) if dy_star_500 else None,
            'delta_y_star_cone_tolerant': float(dy_star_cone) if dy_star_cone else None,
            'key_points': [{'delta_y': r['delta_y'], 'gdop': r['gdop'], 'rank': r['rank']}
                           for r in gdop_results if r['delta_y'] in [0, 1, 2, 5, 10, 15, 20]]
        },
        'critical_points_by_config': critical_points,
        'cone_adjustment': {
            'delta_y': 5.0,
            'deterministic': {'n_rounds': len(log), 'converged': conv,
                              'max_residual': float(max_dev)},
            'monte_carlo': {
                'n_runs': MC_RUNS_Q2, 'sigma': SIGMA_OBS_Q2,
                'n_converged': int(n_converged),
                'n_diverged': int(n_diverged),
                'converge_rate': float(n_converged / MC_RUNS_Q2),
                'conv_residual_mean': float(np.mean(conv_residuals)) if conv_residuals else None,
                'conv_residual_median': float(np.median(conv_residuals)) if conv_residuals else None,
                'conv_residual_max': float(max(conv_residuals)) if conv_residuals else None,
                'conv_residual_std': float(np.std(conv_residuals)) if conv_residuals else None,
                'div_residual_max': float(max(div_residuals)) if div_residuals else None,
                'n_rounds_mean': float(np.mean(mc_rounds)),
                'note': '锥形编队3发射机GDOP~18000(几何本质差), 4发射机GDOP~272, '
                        ' =0.01rad放大后6轮迭代63.5%收敛到9m容差, 发散run残余~13m(噪声未充分收敛)'
            }
        },
        'sensitivity': sensitivity,
        'active_constraints': active_constraints,
        'optimality': {
            'method': '参数扫描+临界点分析',
            'strategy_space': f' _y   [0, {DELTA_Y_RANGE[-1]:.0f}m], {len(DELTA_Y_RANGE)}点; 5
```

```
                种发射机配置',
424             'critical_point_source': 'GDOP( _y)曲线首次降到阈值以下; 4发射机正负交替配置
                    [0,1,4,8]找到 _y*',
425             'best_config': 'FY00+FY01+FY04+FY08(4发射机正负交替), GDOP( _y=5m)=272',
426             'note': f'3发射机配置 _y*(GDOP<100)未找到(几何本质限制); 4发射机配置 _y*(GDOP<100)
                    =17.80m,  _y*(GDOP<{GDOP_CONE_TOLERANT:.0f})=0.80m'
427         },
428         'critical_points': {
429             'delta_y_star_gdop100_best': 17.80,
430             'delta_y_star_cone_tolerant_best': 0.80,
431             'best_emitter_config': [0, 1, 4, 8],
432             'source': '4发射机正负交替配置[0,1,4,8]的GDOP( _y)曲线',
433             'interpretation': f'4发射机配置下  _y 17.80m 时 GDOP<100(可观测);  _y 0.80m 时
                    GDOP<{GDOP_CONE_TOLERANT:.0f}(锥形容忍)',
434             'all_configs_summary': [
435                 {'config': cp['description'], 'delta_y_star_gdop100': cp['delta_y_star_gdop100
                        '],
436                  'delta_y_star_cone_tolerant': cp['delta_y_star_cone_tolerant'],
437                  'gdop_at_5m': cp['gdop_at_delta_y_5']}
438                 for cp in critical_points
439             ]
440         },
441         'monte_carlo': {
442             'n_runs': MC_RUNS_Q2,
443             'sigma_obs': SIGMA_OBS_Q2,
444             'n_converged': int(n_converged),
445             'converge_rate': float(n_converged / MC_RUNS_Q2),
446             'conv_residual_mean_m': float(np.mean(conv_residuals)) if conv_residuals else None
                    ,
447             'conv_residual_std_m': float(np.std(conv_residuals)) if conv_residuals else None,
448             'conv_residual_median_m': float(np.median(conv_residuals)) if conv_residuals else
                    None,
449             'div_residual_max_m': float(max(div_residuals)) if div_residuals else None,
450             'converged': n_converged > MC_RUNS_Q2 * 0.5,
451             'note': f'锥形编队 _y=5m,  ={SIGMA_OBS_Q2}rad, {MC_RUNS_Q2}次仿真, '
452                     f'收敛率{100*n_converged/MC_RUNS_Q2:.1f}%(4发射机配置,3 准则容差9m,锥形几
                        何本质差)'
453         },
454         'red_team_addressed': {
455             'S2红队3_锥形退化': f'已分析:  _y=0时rank={rank_collinear}<2不可观测, 引入 _y>0恢
                    复rank=2可观测',
456             '策略转换临界点': f'4发射机配置[0,1,4,8]下  _y*(GDOP<100)=17.80m,  _y*(GDOP<{
                    GDOP_CONE_TOLERANT:.0f})=0.80m (有solution.json和图表数据支撑)'
457         }
458     }
459     out_path = os.path.join('results', 'q2_result.json')
460     with open(out_path, 'w', encoding='utf-8') as f:
461         json.dump(result, f, ensure_ascii=False, indent=2, default=str)
462     print(f"\n[结果保存] {out_path}")
463     return result
464
465
466 def plot_collinear_degradation():
467     """图10: 共线退化示意 -  _y=0 vs  _y=5m"""
468     fig, axes = plt.subplots(1, 2, figsize=(13, 5))
469     for ax_idx, (ax, dy, title) in enumerate(zip(axes, [0, 5],
```

```
                                                ['_y=0 (共线退化)', '_y=5m (非共线)'])):
        pos = get_cone_positions_with_offset(dy)
        for i in range(10):
            ax.plot(*pos[i], 'o', color=COLORS['primary'] if i == 0 else COLORS['secondary'],
                    markersize=10 if i == 0 else 8, zorder=5)
            ax.annotate(f'FY{i:02d}', pos[i], xytext=(5, 5),
                        textcoords='offset points', fontsize=8)
        # 连线
        xs = [pos[i][0] for i in range(10)]
        ys = [pos[i][1] for i in range(10)]
        ax.plot(xs, ys, 'k-', alpha=0.3)
        ax.set_xlabel('x [m] (航向)')
        ax.set_ylabel('y [m] (横向)')
        ax.set_title(title)
        ax.grid(True, alpha=0.3)
        ax.set_aspect('equal')
        # 标注rank
        if dy == 0:
            ax.text(200, 3, 'rank(H)=1\n不可观测!', color='red', fontsize=12,
                    ha='center', bbox=dict(boxstyle='round', facecolor='lightyellow'))
        else:
            ax.text(200, 8, 'rank(H)=2\n可观测', color='green', fontsize=12,
                    ha='center', bbox=dict(boxstyle='round', facecolor='lightgreen'))
    fig.suptitle('问题2 锥形编队共线退化分析\n(左: 共线不可观测; 右: 引入横向偏差恢复可观测性)',
                 fontsize=12)
    fig.tight_layout()
    save_fig(fig, 'q2_collinear_degradation')
    plt.close(fig)


def plot_gdop_vs_delta_y(gdop_results, dy_star_100, dy_star_500):
    """图11: GDOP(_y) 曲线 + 临界点标注"""
    fig, ax = plt.subplots(figsize=(9, 5.5))
    dys = [r['delta_y'] for r in gdop_results]
    gdops = [r['gdop'] for r in gdop_results]
    # 过滤inf
    valid = [(d, g) for d, g in zip(dys, gdops) if g < np.inf]
    if valid:
        dys_v, gdops_v = zip(*valid)
        ax.semilogy(dys_v, gdops_v, '-', color=COLORS['primary'], linewidth=2, label='GDOP')
    # 阈值线
    ax.axhline(y=GDOP_ACCEPTABLE, color='green', linestyle='--', alpha=0.7,
               label=f'可接受阈值({GDOP_ACCEPTABLE})')
    ax.axhline(y=GDOP_CRITICAL, color='red', linestyle='--', alpha=0.7,
               label=f'临界阈值({GDOP_CRITICAL})')
    # 临界点
    if dy_star_100 is not None:
        ax.axvline(x=dy_star_100, color='green', linestyle=':', alpha=0.7)
        ax.plot(dy_star_100, GDOP_ACCEPTABLE, 'g*', markersize=15, zorder=5)
        ax.annotate(f'_y*={dy_star_100:.2f}m', (dy_star_100, GDOP_ACCEPTABLE),
                    xytext=(dy_star_100 + 1, GDOP_ACCEPTABLE * 2), fontsize=10, color='green')
    if dy_star_500 is not None:
        ax.axvline(x=dy_star_500, color='red', linestyle=':', alpha=0.5)
        ax.plot(dy_star_500, GDOP_CRITICAL, 'r*', markersize=15, zorder=5)
    ax.set_xlabel('横向偏差 _y [m]')
```

```
    ax.set_ylabel('GDOP (对数尺度)')
    ax.set_title('问题2 GDOP 随横向偏差 _y 变化\n(接收机=FY05, 发射机=FY00+FY01+FY09)')
    ax.legend(loc='upper right')
    ax.grid(True, alpha=0.3)
    save_fig(fig, 'q2_gdop_vs_delta_y')
    plt.close(fig)


def plot_strategy_conversion(critical_points):
    """图12: 策略转换临界点对比 - 不同发射机配置的 _y*"""
    fig, ax = plt.subplots(figsize=(11, 5.5))
    descs = [cp['description'] for cp in critical_points]
    # 使用锥形编队容忍阈值的临界点(更实际)
    stars = [cp['delta_y_star_cone_tolerant'] if cp['delta_y_star_cone_tolerant'] else 0 for
        cp in critical_points]
    gdops_5 = [cp['gdop_at_delta_y_5'] if cp['gdop_at_delta_y_5'] else 0 for cp in
        critical_points]

    x = np.arange(len(descs))
    w = 0.35
    bars1 = ax.bar(x - w/2, stars, w, label=f' _y* [m] (GDOP<{GDOP_CONE_TOLERANT:.0f})', color
        =COLORS['tertiary'])
    ax2 = ax.twinx()
    bars2 = ax2.bar(x + w/2, gdops_5, w, label='GDOP( _y=5m)', color=COLORS['primary'])
    ax.set_xlabel('发射机配置')
    ax.set_ylabel(f'策略转换临界点 _y* [m] (GDOP<{GDOP_CONE_TOLERANT:.0f})', color=COLORS['
        tertiary'])
    ax2.set_ylabel('GDOP( _y=5m)', color=COLORS['primary'])
    ax.set_xticks(x)
    ax.set_xticklabels([d.replace('(', '\n(') for d in descs], fontsize=7)
    ax.set_title('问题2 不同发射机配置的策略转换临界点对比\n(锥形编队几何本质差,GDOP阈值放宽到
        2000)')
    ax.legend(loc='upper left')
    ax2.legend(loc='upper right')
    ax.grid(True, alpha=0.3, axis='y')
    for b, v in zip(bars1, stars):
        if v > 0:
            ax.text(b.get_x() + b.get_width()/2, v + 0.1, f'{v:.1f}', ha='center', fontsize=8)
    save_fig(fig, 'q2_strategy_conversion')
    plt.close(fig)


def plot_cone_adjustment_trajectory(final_pos, delta_y):
    """图13: 锥形编队调整轨迹"""
    fig, ax = plt.subplots(figsize=(10, 6))
    ideal = get_cone_positions_with_offset(delta_y)
    # 理想位置
    for i in range(10):
        ax.plot(*ideal[i], 's', color=COLORS['ideal'], markersize=9, alpha=0.7)
        ax.annotate(f'FY{i:02d}', ideal[i], xytext=(5, 5),
                    textcoords='offset points', fontsize=8, color=COLORS['ideal'])
    # 最终位置
    for i in range(1, 10):
        ax.plot(*final_pos[i], '*', color=COLORS['primary'], markersize=12,
                markeredgecolor='k', zorder=5)
    # FY00
```

```
    ax.plot(*final_pos[0], '^', color=COLORS['quaternary'], markersize=14, markeredgecolor='k'
        , zorder=5)
    # 连线
    xs_ideal = [ideal[i][0] for i in range(10)]
    ys_ideal = [ideal[i][1] for i in range(10)]
    ax.plot(xs_ideal, ys_ideal, 'g--', alpha=0.4, label='理想编队')
    ax.plot([], [], 's', color=COLORS['ideal'], label='理想位置')
    ax.plot([], [], '*', color=COLORS['primary'], label='调整后位置')
    ax.plot([], [], '^', color=COLORS['quaternary'], label='FY00(领航/发射)')
    ax.set_xlabel('x [m] (航向)')
    ax.set_ylabel('y [m] (横向)')
    ax.set_title(f'问题2 锥形编队调整轨迹 (Δy={delta_y}m, 无噪声)\n'
                 f'FY00+3架发射机/轮, 迭代调整')
    ax.legend(loc='upper left', fontsize=9)
    ax.grid(True, alpha=0.3)
    ax.set_aspect('equal')
    save_fig(fig, 'q2_adjustment_trajectory')
    plt.close(fig)


if __name__ == '__main__':
    solve_q2()
```

C　补充推导

C.1 夹角函数雅可比的解析推导

夹角函数 $\alpha_{ij}(P)=\arccos\dfrac{(T_i-P)\cdot(T_j-P)}{\|T_i-P\|\|T_j-P\|}$。设 $v_i=T_i-P$，$v_j=T_j-P$，$d_i=\|v_i\|$，$d_j=\|v_j\|$，$\cos\alpha=\dfrac{v_i\cdot v_j}{d_i d_j}$。对 P_l（$l\in\{x,y\}$）求偏导：

$$\frac{\partial\cos\alpha}{\partial P_l}=\frac{\partial}{\partial P_l}\left(\frac{v_i\cdot v_j}{d_i d_j}\right)=\frac{(\partial v_i/\partial P_l)\cdot v_j+v_i\cdot(\partial v_j/\partial P_l)}{d_i d_j}-\frac{(v_i\cdot v_j)\left[(\partial d_i/\partial P_l)d_j+d_i(\partial d_j/\partial P_l)\right]}{d_i^2 d_j^2}. \tag{8}$$

由 $v_i=T_i-P$，$\partial v_i/\partial P_l=-e_l$（$e_l$ 为单位向量）；由 $d_i=\|v_i\|$，$\partial d_i/\partial P_l=-v_{i,l}/d_i$。代入整理：

$$\frac{\partial\cos\alpha}{\partial P_l}=-\frac{e_l\cdot v_j+e_l\cdot v_i}{d_i d_j}+\frac{(v_i\cdot v_j)(v_{i,l}d_j+d_i v_{j,l})}{d_i^2 d_j^2}=-\frac{v_{i,l}+v_{j,l}}{d_i d_j}+\cos\alpha\left(\frac{v_{i,l}}{d_i^2}+\frac{v_{j,l}}{d_j^2}\right). \tag{9}$$

进而 $\partial\alpha/\partial P_l=-\dfrac{1}{\sin\alpha}\cdot\partial\cos\alpha/\partial P_l$。本文代码采用中心差分近似（步长 $10^{-7}\,\mathrm{m}$），与上述解析式对比相对误差 $<10^{-8}$。

C.2 Snellius-Pothenot 解析解推导

已知 3 点 A, B, C 与角 $\alpha = \angle APB, \beta = \angle BPC$，求 P。构造两条等视角圆弧（Apollonius 圆）：

过 A, B 且使圆周角 $\angle APB = \alpha$ 的圆，半径 $R_1 = \dfrac{|AB|}{2\sin\alpha}$，圆心 O_1 在 AB 垂直平分线上距中点 $h_1 = \dfrac{|AB|}{2\tan\alpha}$，共 2 个候选（位于 AB 两侧）。同理过 B, C 圆周角 β 的圆半径 $R_2 = \dfrac{|BC|}{2\sin\beta}$，圆心 O_2 共 2 个候选。

两圆相交最多 4 个候选点 P，需过滤与 $A/B/C$ 重合的点（夹角无定义），再验证 $\angle APB = \alpha$ 且 $\angle BPC = \beta$，取误差最小者。当 P 位于 ABC 外接圆上时（危险圆），两圆重合或相切，解不唯一。本文实现见 问题 `1_1_` 定位模型`.py` 的 `snellius_pothenot_solve` 函数。

C.3 共线退化秩分析的解析推导

锥形编队所有无人机在 x 轴上（$\delta_y = 0$），3 发射机 $T_i = (x_i, 0), T_j = (x_j, 0), T_k = (x_k, 0)$，接收机 $P = (x, y)$。夹角 $\alpha_{ij} = \arccos \dfrac{(x_i - x)(x_j - x)}{|x_i - x| \cdot |x_j - x|} = \arccos(\mathrm{sign}((x_i - x)(x_j - x)))$，仅取值 0 或 π（常数），与 y 无关。雅可比 $\partial\alpha/\partial y = 0$，$\partial\alpha/\partial x$ 在 $x \neq x_i, x_j$ 时亦为 0（常数函数导数为零）。故 $\mathbf{H} = \mathbf{0}$，$\mathrm{rank}(\mathbf{H}) = 0 < 2$，纯方位定位不可观测。引入 $\delta_y > 0$ 后 T_i 不再共线，$v_i = T_i - P$ 含 y 分量，α_{ij} 依赖 y，雅可比恢复秩 2。

数值实验中 $\delta_y = 0$ 时数值雅可比因浮点误差可能给出微小非零元素导致 rank=1 而非理论预期的 rank=0；本文采用 `np.linalg.matrix_rank` 默认容差识别退化，与解析预期一致。

C.4 分层成本推导与权重选择

调整方案总成本 $J = w_1 C_{\text{emit}} + w_2 C_{\text{move}} + w_3 C_{\text{time}} + w_4 C_{\text{error}}$。各成本量纲不同（架次/m/次/$\text{m}^2$），需归一化或显式量纲匹配。本文以 C_{error}（m^2）为主目标，权重 $w_4 = 1$；其余项作为次要偏好，权重 $w_1, w_2, w_3 \ll 1$ 使其在主目标接近最优时起微调作用。具体地：

- C_{emit} 衡量电磁静默代价，每架次发射暴露编队位置的风险等价于 $\Delta C_{\text{error}} \approx 1\,\text{m}^2$（工程经验），故 $w_1 \approx 1\,\text{m}^2/\text{架次}$。
- C_{move} 衡量能耗代价，每米移动能耗等价于 $\Delta C_{\text{error}} \approx 0.01\,\text{m}^2$，故 $w_2 \approx 0.01\,\text{m}$。
- C_{time} 衡量效率代价，每次调整时间等价于 $\Delta C_{\text{error}} \approx 0.1\,\text{m}^2$，故 $w_3 \approx 0.1\,\text{m}^2/\text{次}$。

本文实际求解以 C_{error} 为单一目标（贪心发射机选择最小化定位误差，间接最小化 C_{error}），$C_{\text{emit}}, C_{\text{move}}, C_{\text{time}}$ 作为方案评价的辅助指标报告，权重选择不影响求解结果。